

Luiz Carlos Aires de Macêdo

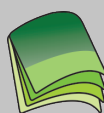
ALGORITMOS E ESTRUTURA DE DADOS I



EdUFERSA

Luiz Carlos Aires de Macêdo

ALGORITMOS E ESTRUTURA DE DADOS I



EDUFERSA

Governo Federal
Ministro de Educação
José Henrique Paim

Universidade Aberta do Brasil
Responsável pela Diretoria da Educação a Distância
João Carlos Teatini de Souza Clímaco

Universidade Federal Rural do Semi-Árido
Reitor
José de Arimatea de Matos

Pró-Reitor de Graduação
Augusto Carlos Pavão

Núcleo de Educação a Distância
Coordenadora UAB
Kátia Cilene da Silva

Equipe multidisciplinar

Antônio Charleskson Lopes Pinheiro – Diretor de
Produção de Material Didático
Ulisses de Melo Furtado – Designer Instrucional
Celeneh Rocha de Castro – Diretora de Formação Continuada
Thiago Henrique Freire de Oliveira – Gerente de Rede
Adriana Mara Guimarães de Farias – Programadora
Camilla Moreira Uchoa – Webdesigner
Ramon Ribeiro Vitorino Rodrigues - Diretor de Arte
Mikael Oliveira de Meneses – Diagramador
Alberto de Oliveira Lima – Diagramador
José Antonio da Silva - Diagramador
Frediano Araújo de Sousa – Ilustrador

Arte da capa

Felipe de Araújo Alves

Equipe administrativa

Rafaela Cristina Alves de Freitas – Assistente em Administração
Iriane Teresa de Araújo – Responsável pelo fomento
Bruno Layson Ferreira leão – Estagiário

Equipe de apoio

Márcio Vinicius Barreto da Silva – Revisão Linguística
Nayra Maria da Costa Lima – Revisão Didática
Josenildo Ferreira Galdino – Revisor Matemático

Serviços técnicos especializados

Life Tecnologia e Consultoria

Edição

EdUFERSA

Impressão

Imprima Soluções Gráfica Ltda/ME

© 2013 by NEaD/UFERSA - Todos os direitos reservados. Nenhuma parte desta publicação poderá ser reproduzida ou transmitida de qualquer modo ou por qualquer outro meio, eletrônico ou mecânico, incluindo fotocópia, gravação ou qualquer outro tipo de sistema de armazenamento e transmissão de informação, sem prévia autorização, por escrito, do NEaD/UFERSA. O conteúdo da obra é de exclusiva responsabilidade dos autores.

Biblioteca Central Orlando Teixeira – BCOT/UFERSA **Setor de Informação e Referência – Ficha Catalográfica**

M141a Macêdo, Luis Carlos Aires de.
 Algoritmos e estrutura de dados / Luis Carlos Aires
 de Macêdo. – Mossoró : EdUFERSA, 2013.
 116 p. : il.

ISBN: 978-85-63145-58-1

1. Algoritmos. 2. Estrutura de dados. I. Título.

RN/UFERSA/BCOT

CDD: 005.1

Bibliotecário-Documentalista
Mário Gaudêncio – CRB-15/476



<http://nead.ufersa.edu.br/>

APRESENTAÇÃO DA DISCIPLINA

Caro(a) aluno(a)!

Nesta disciplina, abordaremos a construção de algoritmos, bem como as estruturas que permitem o desenvolvimento de programas de computadores. Trabalharemos de forma a apresentar os conceitos de estruturas de dados, que nada mais são do que convenções sobre a representação de uma forma que nos permite realizar a comunicação com o computador. Porém, tal forma deve ser organizada por regras e convenções, seguindo um padrão preestabelecido.

Assim, podemos adiantar que escrever algoritmos é uma forma de comunicação entre você e a máquina, fazendo com que o computador entenda sua vontade e realize as operações que você deseja de forma a atender suas necessidades.

Neste caderno, você vai encontrar algumas seções de perguntas e respostas, a fim de enriquecer ainda mais o seu conhecimento. Temos também uma seção denominada “Mais”, onde abordamos assuntos que o ajudarão a entender o que está sendo estudado, os exercícios resolvidos e comentados e os exercícios propostos para a construção do conhecimento.

É importante que você compreenda que algoritmo não se aprende do dia para a noite. Assim sendo, é necessária muita dedicação para a construção do conhecimento neste assunto de forma incremental, ou seja, você deve ter o cuidado de realmente aprender o conteúdo. Sugiro que leia o material didático e realize todos os exercícios, participe intensamente dos chats e, principalmente, não deixe acumular dúvidas.

Você deve ter percebido o quanto esta disciplina é importante para este curso. Ela é a base de toda a programação, a lógica do funcionamento dos computadores.

Então, se você aprende bem algoritmos será um bom programador, porque programar é uma questão de entender a lógica de um computador, seu funcionamento e suas particularidades.

Bons estudos !

SOBRE O AUTOR

Luiz Carlos Aires de Macêdo é Mestre em Ciência da Computação pela UERN/UFERSA, graduado em Sistemas de Informação pela Universidade Potiguar de Natal-RN.

Possui vasta experiência no desenvolvimento de softwares com ênfase em automação industrial, atuando por vários anos no mercado de trabalho.

Atualmente, é professor da UFERSA, desde 2010, lecionando no Campus Caraúbas a disciplina de Informática Aplicada a Engenharia.

SUMÁRIO

UNIDADE I

INTRODUÇÃO A LÓGICA DE PROGRAMAÇÃO

NOÇÕES DE LÓGICA DE PROGRAMAÇÃO E ALGORITMO 13

TIPOS DE ALGORITMO 17

FORMAS DE REPRESENTAÇÃO DOS ALGORITMOS COMPUTACIONAIS 17

- Linguagem natural 17
- Notações gráficas 18
 - Diagrama de Chapin 18
 - Fluxogramas 19
- Português estruturado 22
 - VisuAlg 23

ESTRUTURA BÁSICA DE ALGORITMO 23

VARIÁVEIS 27

- Tipos de variáveis 30
- Definindo variáveis 31

CONSTANTE 32

ENTRADA E SAÍDA DE DADOS 34

ENTRADA DE DADOS 34

SAÍDA DE DADOS 35

TRABALHANDO A CONSTRUÇÃO DE EXPRESSÕES 41

- Atribuição 41
- Instrução e comandos 42
- Operadores e expressões 42

UNIDADE II

ESTRUTURAS DO ALGORITMO

OPERAÇÕES LÓGICAS	49
DESVIO CONDICIONAL SIMPLES E COMPOSTO	50
DESVIO CONDICIONAL ANINHADO	55
COMANDO DE DECISÃO ESCOLHA	60
CONECTORES LÓGICOS	61
• Conector lógico "E"	61
• Conector lógico "OU"	62
• Conector lógico "NÃO"	63
LAÇOS DE REPETIÇÃO	65
• Enquanto	65
• Repita	66
• Para	67
• Comandos de repetição usando o Diagrama de fluxo de dados	68

UNIDADE III

ESTRUTURAS DE ARMAZENAMENTOS E ESTRUTURA DE BLOCOS

MATRIZES	77
• Matrizes com uma dimensão	77
• Matrizes com mais de uma dimensão	85
PROGRAMAÇÃO EM BLOCO (SUBALGORITMOS)	89
• Procedimento	89
• Funções	93
REGISTROS	99

I

INTRODUÇÃO A LÓGICA DE PROGRAMAÇÃO

Nesta unidade, veremos como transformar um problema do mundo real em um problema a ser resolvido no mundo computacional. Veremos, também, como tal representação pode ser feita e quais as melhores ferramentas e técnicas para isso.

Objetivos:

- Apresentar os algoritmos como solução para a modelagem de problemas do cotidiano, através das suas formas de representação.

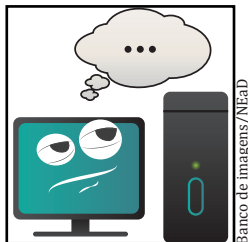
Noções de lógica de programação e Algoritmo

UN 01

A palavra “Lógica” surgiu da filosofia e pode ser definida como a parte da filosofia que estuda o fundamento, a estrutura e as expressões humanas do conhecimento (BRASILESCOLA, 2013). Existem diversos tipos de lógica, dentre elas podemos ter a lógica filosófica, a matemática e a computacional, uma derivação da lógica matemática, objeto de nosso estudo.

Neste curso, vamos estudar a lógica matemática com ênfase na computação. No caso da computação, podemos ver a lógica como uma tentativa de fazer uma máquina raciocinar como um ser humano, pois sabemos que as máquinas não “pensam”.

Sabemos que o *software* é a parte que faz o computador pensar, que são programas e que uma máquina (computador) possui diversos tipos de programas, ou seja, *softwares* de todos os tipos para todas as funções. Assim, conhecer as lógicas matemática e computacional é primordial para desenvolver um programa de computador e entender como ele funciona.



Em nosso cotidiano, nos deparamos com o fato de realizarmos muitas coisas sem pensar! Não pensamos exatamente na atitude tomada em uma situação de perigo, que exige uma decisão rápida. E, simplesmente, dizemos que fazemos algo por ser lógico (óbvio), assim como também temos reações rápidas, sem pensar. Em outras, pensamos muito antes de tomar algumas decisões. Como exemplo, podemos citar:

- Quando encostamos nossa mão em um ferro quente: reagimos rapidamente e logo tiramos nossa mão desse ferro quente!
- Quando nos sentimos extremamente cansados: logo procuramos alguma forma de descansar.

Nos casos em que devemos pensar bastante sobre o que fazer para tomar uma decisão, nem sempre a lógica dos fatos nos levará a uma decisão rápida, como, por exemplo:

- Tenho uma casa velha. Devo comprar uma nova casa, um apartamento ou reformar minha casa velha?
- Minhas contas não fecham, estou devendo muito! Preciso de mais dinheiro. Como fazer para ter mais dinheiro?

As situações listadas acima podem nos demonstrar o quão complexa pode ser uma decisão e o tempo que podemos levar para tomarmos uma decisão baseada em fatos e situações.

Mas o que isso tem a ver com programação? Bom, se vamos ensinar um computador a pensar e realizar tarefas, devemos ensiná-lo como fazer isso! Então, você deve concordar que decisões simples são mais fáceis de serem escolhidas, o que nos leva a concluir logicamente (usamos a lógica de inferência aqui...) que criar programas nem sempre é uma tarefa simples e que tudo dependerá do que queremos resolver.

Podemos adiantar que construir programas de computador é na verdade descrever um problema do mundo real (mundo em que vivemos) para o mundo virtual (de 0's e 1's da vida computacional), de forma que o computador poderá entender.

Assim, usamos a lógica de programação, representando os problemas do mundo real por meio dos algoritmos, seguindo a ordem dos acontecimentos de uma situação do mundo real em uma sequência lógica, ordenada e finita, transcrevendo um problema do mundo real para o computacional a fim de que possamos ensinar o computador a pensar e realizar as tarefas do nosso cotidiano.

Desta forma, um algoritmo pode ser entendido como uma sequência de passos a serem seguidos a partir do entendimento de um problema com o objetivo de transformá-lo em um programa de computador (MANZANO E OLIVEIRA, 2010).



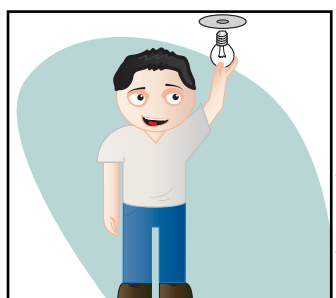
Banco de imagens/NEAD

Entendido o conceito de algoritmo, podemos começar a trabalhar a possibilidade de resolver problemas do mundo real por meio dos algoritmos para então realizarmos a programação dos computadores.

Vamos imaginar a seguinte situação:

Digamos que nos deparamos com uma lâmpada que não acende. Desejamos, então, resolver este problema. Paramos um pouco e pensamos de forma lógica sobre tal problema. Iniciaremos com o entendimento desse problema nos perguntando: o que aconteceu com a lâmpada?

Após esta pergunta, nosso cérebro irá supor várias situações, mas devemos pensar de forma ordenada, lógica. Assim, vamos escrever os passos para a resolução deste problema de forma ordenada, observando sempre uma sequência lógica e correta dos passos a serem seguidos para a resolução deste problema.



Banco de imagens/NEAD

Portanto, após uma análise do problema e entendimento da situação, listamos alguns passos para uma possível correção. Concluímos que a lâmpada está queimada! Usaremos o exemplo da lâmpada para demonstrar que os problemas computacionais devem ser resolvidos baseando-se em sequências de decisões. Na computação, chamamos este processo de algoritmo. Veja:

1. Pego uma escada para subir e chegar à lâmpada;
2. Verifico se a lâmpada está folgada, caso estiver irei ajustá-la;
3. Desço da escada e tento ligá-la novamente. Se acender, problema resolvido;
4. Caso a lâmpada não tenha acendido, subirei na escada novamente e retirarei a lâmpada;
5. Desço da escada e vou à casa de ferragens mais próxima para comprar uma nova lâmpada;
6. Subo novamente na escada e coloco a lâmpada nova no local;
7. Desço da escada e ligo. Caso acenda, problema resolvido!

Com estes sete passos listados de forma ordenada, eu consigo resolver o problema da lâmpada queimada. Mas então por que falei acima que esse seria um possível algoritmo?

Bom, por dois motivos. Primeiro: esta é minha forma de resolver o problema. Você pode ter uma forma diferente para resolvê-lo, concorda? Segundo: no passo 7 de meu algoritmo, pode ser que eu não tenha resolvido o problema, tendo então que verificar outros possíveis problemas que poderiam estar ocorrendo, como a possibilidade de o defeito estar no interruptor, dentre outros problemas que podem ocorrer, chegando até ao ponto de eu ter que chamar um eletricista para resolver este problema.

Outro ponto a observar: será que este algoritmo descreve corretamente todos os problemas de trocas de lâmpadas? Certamente não! Este nosso algoritmo só poderá ser a resolução de um conjunto de problemas específicos daquele tipo de lâmpada.

Na referência, Forbellone e Eberspacher (2005) trazem este problema da lâmpada descrito de forma bastante detalhada. Vale a pena consultar.

Por que é importante aprender algoritmo?

Podemos dizer que qualquer curso de Computação tem a disciplina de algoritmos como uma das mais importantes, pois é por meio desta que poderemos escrever os programas de computador. Portanto, se você quer ser um bom programador, dê uma atenção especial a essa disciplina.



Banco de imagens/Nêad

Por que escrever algoritmo é difícil para muitas pessoas?

Algumas pessoas sentem dificuldades em escrever algoritmos porque passaram toda a sua vida de estudantes condicionadas a resolver os problemas sem precisar pensar muito em como resolver um determinado problema. É natural, portanto, que as pessoas condicionadas a resolver problemas desta forma sintam a dificuldade de, ao escrever um algoritmo, resolver o problema em questão, mas apenas descrever os passos para a resolução do problema, deixando a resolução para o computador. Assim, para algumas pessoas, criar algoritmo será uma tarefa árdua, exigindo que você repense a forma de resolver um problema para, então, escrever um algoritmo.

Algoritmo é uma linguagem de programação?

Não! Algoritmo não é uma linguagem de programação, apesar de você poder escrever algoritmos e executar como se fosse um software com a ajuda de ferramentas. Veremos uma delas nesse caderno. Os códigos escritos usando os códigos de um algoritmo não serão transformados em um programa, mas serão interpretados pelos softwares de auxílio para que você aprenda sobre a lógica de programação, por isso é o algoritmo é importante.

Se eu conseguir escrever bem algoritmos, serei um bom programador?

Acredito que sim! Mas programar vai um pouco além de escrever bons algoritmos, pois você vai precisar de habilidades específicas, como a abstração, capacidade de transcrever um problema do mundo real para o mundo computacional. Precisarás também das habilidades de domínio da linguagem de programação para decidir qual delas usar, pois cada linguagem de programação possui vantagens e limitações.

Assim, podemos concluir que:

1. Algoritmo é a descrição dos passos de forma ordenada para se resolver um problema. Isto não significa a resolução de um problema;
2. Um algoritmo não precisa descrever todas as soluções para resolver um problema, mas deve descrever ao menos uma solução possível;
3. Para escrever algoritmos, devemos pensar logicamente em como resolvemos um problema, escrevendo os passos para tal resolução;
4. Algoritmo exige uma nova forma de pensar sobre o processo de resolução de problemas.



Banco de imagens/Nêad

SAIBA MAIS

Na criação de nossos algoritmos, devemos nos concentrar em como fazer a transformação de um problema do mundo real e torná-lo computacional. Uma das técnicas é a da abstração, que significa ignorar detalhes e nos concentrarmos no que realmente importa (DICABSTRAÇÃO, 2013).

Deste modo, no processo de codificação de nossos algoritmos devemos olhar e nos fixar no que realmente importa para a resolução do programa e implementar o essencial para isto. Depois de nos concentrarmos no que realmente importa para a resolução de um programa, podemos nos preocupar com os detalhes e posteriormente inseri-los em nossos algoritmos, caso seja necessário.

Na prática, este processo de abstração é muito importante para a construção de software, o qual você estudará detalhadamente na disciplina de Engenharia de Software. Também é importante para que você construa algoritmos de forma mais eficiente, usando um método rápido de resolução de problemas. Este método será abordado mais adiante.

O fato de usarmos a abstração não significa que não devamos nos preocupar com os detalhes a serem implementados em um algoritmo, lembrando que um bom software é aquele que se preocupa com todos os detalhes de uma situação, o que evita erros e mantém a integridade e confiança em um software. Inicialmente, ignoraremos muitos detalhes (Abstração!) da resolução de um problema – assim como fizemos na resolução do problema de nossa Lâmpada queimada – para nos preocuparmos em aprender a lógica e construção de algoritmo.

▶ EXERCÍCIO RESOLVIDO

Vou passar para você a receita rápida e prática, além de barata, para se fazer um bolo de chocolate na caneca no forno de micro-ondas...

1. Em uma caneca, junte um ovo inteiro, três colheres de sopa de óleo;
2. Em seguida, adicione quatro colheres de sopa de leite, três colheres de sopa de açúcar e três colheres de chocolate em pó, mexendo bem a massa!
3. Vá adicionando aos poucos farinha de trigo (no máximo quatro colheres de sopa) e mexendo até se tornar uma massa densa;
4. Por fim, acrescente meia colher de chá de fermento em pó e misture novamente;
5. Leve ao forno de micro-ondas por três minutos;
6. Retire do forno micro-ondas e sirva à vontade.

Bom, mas o que isto tem a ver com a nossa disciplina?

Acredite: Este é um exemplo de algoritmo! Um algoritmo para fazer um bolo de chocolate seguindo uma sequência ordenada de passos. Observe que temos os ingredientes e os passos a serem seguidos. Assim, esperamos que ao seguir esses passos você obtenha o sucesso e que seu bolo de caneca seja muito saboroso, assim como o meu foi.

▶ EXERCÍCIO PROPOSTO

1. Escreva um algoritmo com todos os passos que descrevam o que você faz em um dia comum (sugiro que use um pouco de abstração).
2. Escreva um algoritmo para solucionar um problema matemático de cálculo de uma área de um retângulo qualquer.
3. Abstração: Imagine que você é um professor. Escreva um algoritmo para a preparação de uma aula que você terá de ministrar sobre determinado assunto.

Sugestão: Compare seu algoritmo com o algoritmo de um colega de turma e veja as semelhanças e diferenças.

Tipos de algoritmo

UN 01

Até o presente momento, deixamos que você trabalhasse a construção de um algoritmo de forma livre, sem seguir regras, apenas usando a linguagem natural. Mas existem regras que você deverá aprender a partir deste momento, as quais são determinantes para que seu algoritmo seja válido para o mundo computacional. Desta forma, você terá de se adaptar a tais regras sempre que precisar construir um algoritmo.

Com relação às regras, devemos primeiro conhecer as classes às quais um algoritmo pode pertencer, as características, além das regras de cada uma dessas classes. Quanto às classes, podemos ter:

1. **Não Computacionais:** Algoritmos que não podem ser interpretados pelo computador, pois são imprecisos e informais.
2. **Computacionais:** Algoritmos que podem ser interpretados pelo computador, pois seguem as regras para isso.

O exemplo citado no Exercício Resolvido da seção é um exemplo de um algoritmo não computacional, pois a forma como foi escrito não condiz com a realidade computacional: a escrita é livre usando as regras da Língua Portuguesa. Para que os algoritmos sejam considerados computacionais, devem ter características como:

A. Precisão: O algoritmo deve indicar a ordem de realização passo a passo para a correta resolução de um problema, respeitando a hierarquia dos acontecimentos.

B. Não ambiguidade: O algoritmo computacional não pode possuir mais de um sentido, ou seja, palavras que podem possuir um duplo sentido, como: manga (fruta ou camisa), banco (dinheiro ou assento), dados (jogar ou informação), dentre outros que devem ser evitados na linguagem de construção dos algoritmos;

C. Finitude: Todo algoritmo deve ter início e fim, pois não tendo um fim não chegará ao resultado desejado;

D. Permitir entradas e saídas de dados do mundo real: Um algoritmo pode receber dados do mundo real para processar e retornar ao mundo real um valor esperado;

E. Definição: O algoritmo pode ser executado diversas vezes e deve garantir o resultado toda as vezes em que for executado.

Cada uma das categorias possui regras específicas. Nosso foco será nos algoritmos computacionais, mas precisamos usar os não computacionais para entender a construção dos algoritmos. As regras são determinadas também pela forma de representação, de vez que em cada forma de representação pode haver regras específicas para a escrita dos algoritmos computacionais.

Formas de representação dos algoritmos computacionais

UN 01

Assim, na tentativa de construir regras para a representação de algoritmos, foram desenvolvidas e apresentadas várias formas por diversos autores, cada qual com suas características. Veremos apenas as mais citadas nas referências usadas para a construção deste caderno.

Linguagem Natural

Esta técnica de representação, na qual estivemos trabalhando até o presente momento, como no exemplo do Exercício Resolvido 01, consiste em descrever com nossa própria linguagem os passos para resolução de determinado problema. Podemos citar um novo exemplo com a descrição de um algoritmo não computacional escrito em linguagem natural. Quando queremos demonstrar como podemos acessar uma página qualquer da web, os passos são os seguintes:

1. Ligar o computador;
2. Acessar o provedor de acesso à internet por meio do programa do provedor;
3. Abrir o programa de navegação;
4. Digitar o endereço da página que queremos visualizar.

Apesar de a descrição de nosso problema para acessarmos uma página da internet ser clara, percebemos que não atendemos a todas as características de um algoritmo computacional por temos problemas nesta descrição, como de dupla interpretação dos passos a serem seguidos e coesão.

Também há problemas quanto à completude, pois existem muitos outros passos que poderíamos ter acrescentado caso algo desse errado, como o problema de acesso ou tipo do navegador, dentre outros. Assim, para escrevermos um algoritmo de forma completa, devemos escrever muitas outras linhas, e certamente deixar mais problemas como o da dupla interpretação, dentre outros já citados.

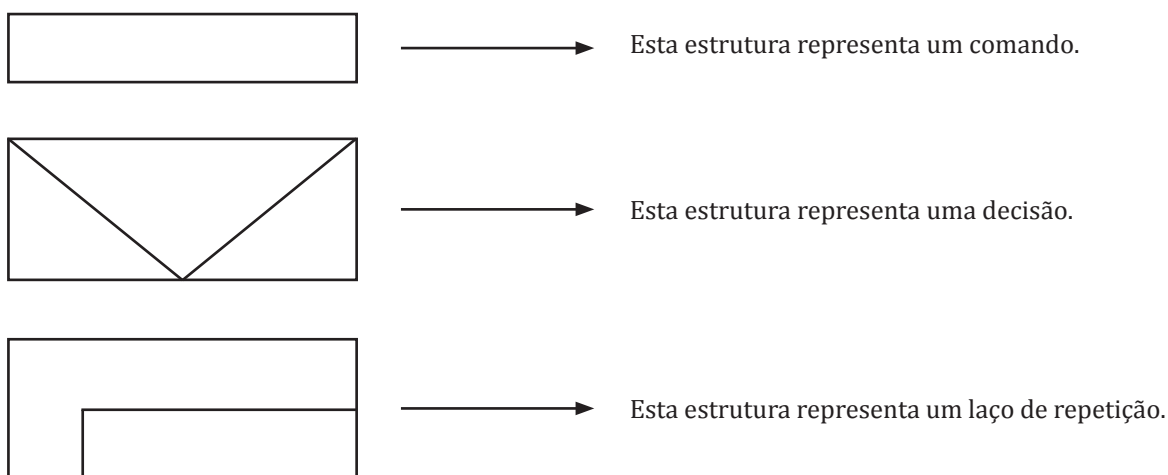
Podemos concluir que apesar de a representação de um algoritmo por meio do uso da linguagem natural ser fácil, usar este tipo de linguagem na representação de um algoritmo computacional não é o ideal.

Notações Gráficas

Nesta classe de representação de algoritmos, podemos encontrar várias formas de representação de um algoritmo por meio de símbolos gráficos, dentre as quais destacamos o Diagrama de Chapin e os Fluxogramas. Falaremos um pouco sobre cada uma delas:

DIAGRAMA DE CHAPIN:

Estes diagramas foram desenvolvidos por Ned Chapin na intenção de criar uma representação gráfica para um programa de forma estruturada a partir de estruturas desenhadas para representação dos principais comandos, como apresentamos a seguir:



Como exemplo, construiremos a representação de um algoritmo de computador no qual deverá ser inserido um valor. Para isso, usaremos o poder da abstração. Representando esse valor como a idade de uma pessoa, o computador deverá analisar a idade inserida e tomar a seguinte decisão: se a idade for superior a 18 anos, a pessoa pode tirar a carteira de motorista e se for inferior a 18 anos, a pessoa não poderá tirar a carteira de motorista. Seguimos então a construção de nosso algoritmo usando a notação de Chapin:



No algoritmo escrito e representado com o Diagrama de Chapin, podemos observar que algumas palavras estão em **negrito**, as quais representam comandos especiais, que veremos detalhadamente mais à frente. São elas:

Início: Determina o início de um algoritmo

Leia: Determina que algum dado seja inserido pelo teclado, sendo este dado armazenado em uma variável, que se chamará “Idade” e armazenará um valor lido.

Escreva: Comando que mostra algo na tela do computador. No caso, as mensagens depois do comando.

Fim: Determina o fim de um programa.

Esta notação para a representação de algoritmos tem como característica as figuras representando a lógica do programa de forma hierárquica e sequencial, como também a característica de que cada figura representará as estruturas básicas de comandos de um algoritmo, como os laços de repetição e tomada de decisão, conforme vimos no exemplo anterior.

O problema da representação por notação de Chapin está no fato de termos que conhecer bem as representações gráficas e as regras de uso destas, como também de termos de montar a representação com os gráficos que podem ocupar bastante espaço para sua representação, o que se agrava quando temos grandes problemas a ser representados.

FLUXOGRAMAS

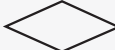
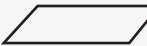
O fluxograma pode ser descrito como um conjunto de notações gráficas para representar a sequência operacional de um processo no qual desejamos que seja representado. Podemos usar diversos formatos e símbolos para compor a representação de um processo ou procedimento, muito utilizados em representação de processos nas áreas administrativas de uma empresa (BRASIL ESCOLA, 2013).

Os fluxogramas são amplamente utilizados na literatura para a representação de processos administrativos ou operacionais, podendo sofrer variações dos símbolos para a representação (questões referentes aos desenhos) e ter uma boa aceitação por vários autores de diferentes áreas.

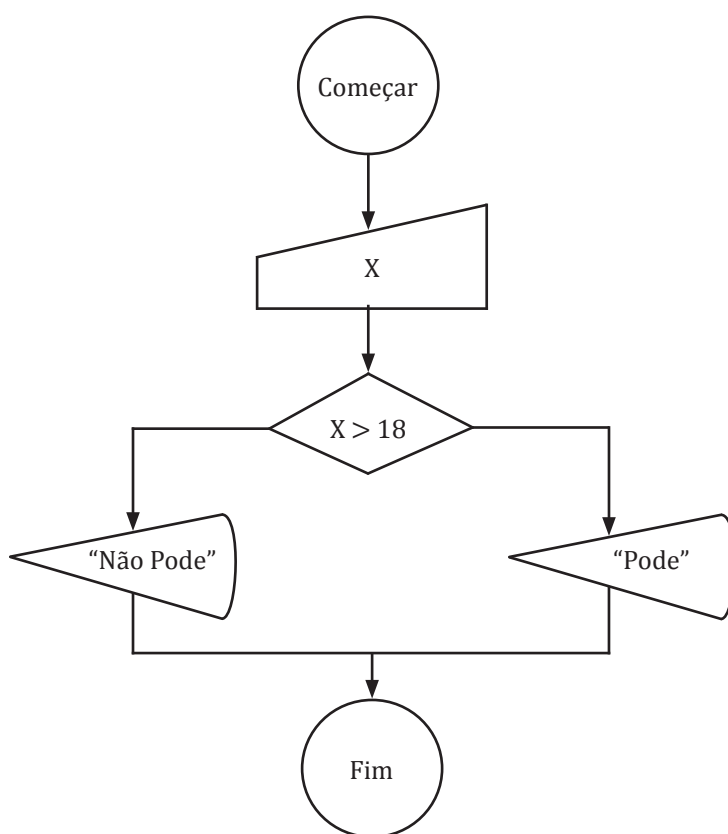
Os fluxogramas também sofreram adaptações para se adequar à representação de algoritmos, tendo a norma ISO 5807:1985(E) como normalizadora para os símbolos e seus significados para a representação dos algoritmos (MANZANO e OLIVEIRA, 2010).

Abaixo são apresentados os principais símbolos que podemos utilizar na composição da representação de um algoritmo usando um fluxograma:

SÍMBOLO	NOME
	terminador
	conector
	fluxo
	processo

SÍMBOLO	NOME
←	atribuição
	sub-rotina
	decisão
	leitura
	exibição

No exemplo abaixo, para representar o uso de um fluxograma, repetiremos o exemplo usado anteriormente com o diagrama de Chapin, onde apresentamos um algoritmo para ler a idade de uma pessoa e dizer se ela pode tirar a carteira de motorista.



O algoritmo inicia e pede que seja digitado um valor a ser representado como a idade da pessoa. No caso, esta idade será armazenada em uma variável (abordaremos mais sobre isto posteriormente) que chamaremos de 'X'. Depois de digitar o valor, o comando seguinte será o de decisão, que perguntará se X é maior do que 18, e em caso positivo mostrará na tela a mensagem "pode", em caso negativo mostrará "não pode", terminando a execução do algoritmo.

Este algoritmo foi desenvolvido baseado no programa FreeDFD. Observe a diferença nos símbolos, em especial o de "Exibição".

Ao contrário da notação de Chapin, esta representação apresenta a sequência dos símbolos com mais clareza por permitir que sejam ligados por setas, as quais definem o fluxo dos acontecimentos.

Ao usarmos os fluxogramas na representação de algoritmos, temos em mãos uma forma eficiente e prática para representar soluções computacionais de tamanho pequeno por meio de algoritmos. Porém, um grande problema surge quando precisamos representar a solução para grandes problemas, pois os símbolos ocuparão muito espaço na representação, tornando difícil o entendimento da solução do problema (criando outro problema).

Por que usar notações gráficas?

A técnica de criação de algoritmos com o uso das notações gráficas facilita a ordenação do pensamento lógico ao resolvermos um algoritmo. Além disso, o uso de desenhos tende a ser de mais fácil entendimento, afinal uma imagem vale mais do que mil palavras, ao menos em princípio.

Usaremos notação gráfica?

Sim, usaremos a notação gráfica do fluxograma até certo ponto com o objetivo de facilitar o entendimento do fluxo e da lógica de um algoritmo, especialmente nos pequenos algoritmos.

SAIBA MAIS

Ferramentas CASE (do inglês Computer-Aided Software Engineering) é uma classificação que abrange todas as ferramentas (softwares) baseadas em computadores que auxiliam em atividades de produção de um software, o que pode se dar desde o projeto de construção do software a simulações, passando inclusive por escrita do código fonte de um programa (PRESSMAN, 2003).

Existem ferramentas case que com poucos cliques conseguem construir um software. Porém, estes softwares serão muito limitados ou com poucas funcionalidades, ao menos até o presente momento. De qualquer forma, é essencial que você conheça bem algoritmos para desenvolver seus programas com ou sem o uso de ferramentas CASE.

Atualmente, é comum usar as ferramentas Case no auxílio à programação, onde esta pode corrigir a sintaxe do código digitado ou criar parte do código, economizando tempo de programação.

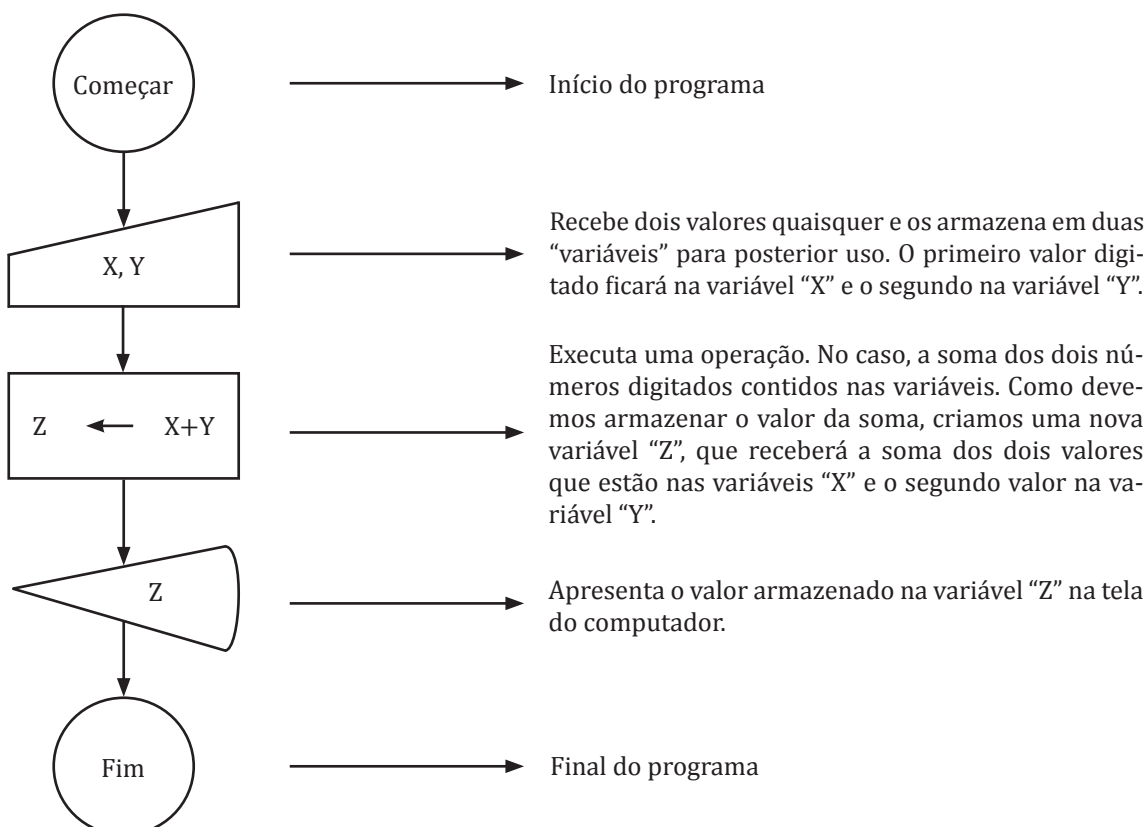
Como forma de auxílio a este caderno, usaremos as ferramentas FreeDFD e VisuAlg para interpretação de nossos algoritmos, sendo estas muito úteis para o processo de aprendizado por permitir os testes de nossos códigos, além de nos auxiliar na compreensão do funcionamento das estruturas que usaremos nos algoritmos.

EXERCÍCIO RESOLVIDO

21

Nota: Estes exercícios resolvidos foram realizados usando o programa FReeDFD Versão 1.1, idioma em Português.

Crie um diagrama de fluxo de dados que receba dois números e apresente a soma desses números.



▶ EXERCÍCIO PROPOSTO

Baixe o programa FreeDFD, instale e teste o algoritmo acima. Observe a interação com o programa. É possível baixar o programa em um dos links abaixo:

<http://www.softpedia.com/progDownload/FreeDFD-Download-137613.html>

ou

<http://freedfd.br.uptodown.com/>.

Apesar de ser um programa simples, existem vídeos no YouTube e diversos tutoriais em sites que ensinam a usar esta ferramenta.



Português estruturado

Nesta forma de representação, é definido um conjunto de palavras que irão possuir significado específico para a interpretação do algoritmo. Tais palavras reservadas são aceitas e adaptadas ao uso computacional, possuindo sintaxe e semântica próprias.

A sintaxe das palavras utilizadas se refere à forma de como tais palavras são escritas. Para isso, devemos seguir regras específicas, tais como o não uso da acentuação e a escrita seguindo um padrão estabelecido e mostrado neste caderno.

A semântica das palavras se relaciona ao significado que cada palavra terá ao ser usada na descrição de um algoritmo, ou seja, os comandos computacionais atrelados à palavra.

Como exemplo, podemos citar as palavras já vistas até o momento, como:

Início: Inicia a construção de um algoritmo;

Fim: Comando que finaliza um algoritmo.

A sintaxe e a semântica do conjunto de palavras escolhidas para representar os algoritmos computacionais darão as características de precisão e não ambiguidade ao algoritmo nesta forma de representação. Além do fato de usarmos palavras de nossa própria linguagem, o que facilitará o processo de compreensão dos algoritmos.

Podemos então concluir que o Português Estruturado é uma forma de representação de um algoritmo por meio de um conjunto finito de palavras.

É como se, a partir de agora, seu vocabulário de palavras se resumisse a um pequeno conjunto de palavras e regras em Português para que você possa representar a construção de um programa.

Então, essa forma de representação em nossa própria língua é realmente muito vantajosa, pois de alguma forma já conhecemos o significado de cada palavra. Porém, apesar de ser bastante vantajoso aprender a lógica de programação utilizando essa linguagem, não se exige uma norma ou convenção formal que estabeleça a sintaxe das palavras, ou seja, como realmente elas devem ser escritas.

Deste modo, temos variação de uma mesma palavra entre os autores de livros sobre lógica e construção de algoritmo e podemos ter algumas palavras específicas de cada autor, podendo gerar problemas na interpretação dos algoritmos caso não seja seguido um autor específico em seus estudos.

Vale lembrar que na construção de algoritmos com os Diagramas de Fluxogramas temos uma norma que rege o uso das notações gráficas, sendo elas padronizadas.

Como exemplo, já podemos perceber que na ferramenta FreeDFD temos a palavra “Começar” e no Português Estruturado usamos a palavra “Iniciar”.

Desta forma, por não haver convenção sobre como escrever, escolhemos escrever nossos algoritmos com o auxílio de uma ferramenta computacional que nos ajudará no aprendizado da construção de algoritmo: o programa VisuAlg. Usaremos este programa até certo ponto deste material. Também tentaremos usar termos comuns entre as literaturas para escrevermos os algoritmos.

Definimos o nome Português Estruturado neste material por convenção, de acordo com os autores nos quais este caderno está baseado. Porém, também podemos encontrar o termo Portugol para esta forma de representação dos algoritmos.

VISUALG

O programa VisuAlg da Apoio Informática é um software gratuito para a construção e teste de algoritmo, sendo esta a ferramenta que usaremos para escrever os algoritmos usando a notação do Português Estruturado.

Você deve baixar o programa gratuitamente por meio do site <http://www.apoioinformatica.inf.br/> e aprender um pouco sobre a forma de usar esta ferramenta.

Um bom tutorial de uso do programa está disponível no seguinte endereço: <http://www.apoioinformatica.inf.br//linguagem.htm>.

Usaremos a convenção do Português Estruturado, adaptando-as, quando necessário, à linguagem do VisuAlg, pois entendemos que é de suma importância testar seus algoritmos. Para isto, nada melhor do que um software para analisarmos o resultado de nossos algoritmos.

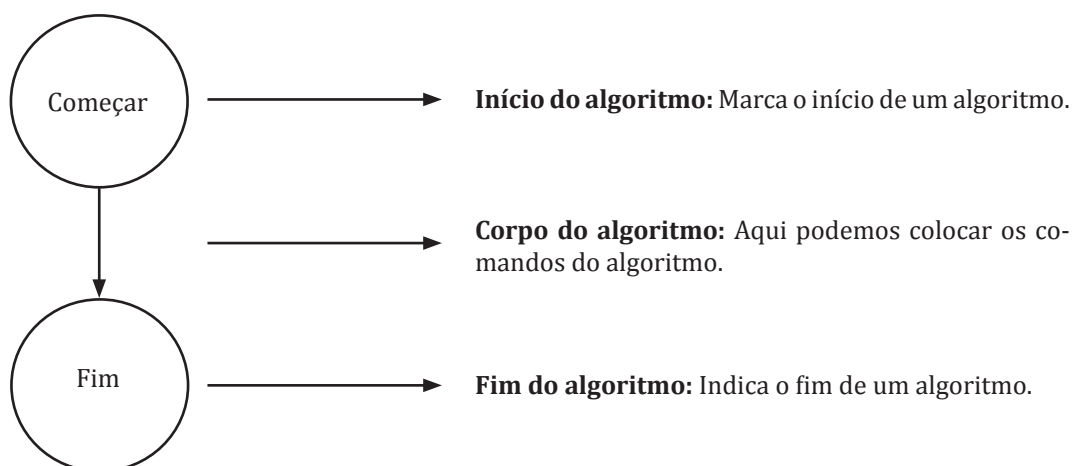


Estrutura básica de um algoritmo

23

UN 01

Podemos definir a estrutura básica de um algoritmo como o corpo do algoritmo, a estrutura mínima que um algoritmo deverá ter. No caso da estrutura de um algoritmo representada por fluxograma, temos:



No caso da representação usando o Português Estruturado, temos a seguinte estrutura mínima:



A palavra determina que o texto abaixo é um algoritmo que deve ter um nome de identificação. Nesta parte, também é possível ter outras identificações para o algoritmo, como: nome do autor, data, versão, dentre outras. A palavra "Algoritmo" é reservada, ou seja, não pode ser utilizada para outros fins no corpo do algoritmo.

Neste ponto do algoritmo, definimos as variáveis que o programa deverá ter para a manipulação dos dados. Também é comum encontrarmos a abreviação dessa palavra como "Var". Ambas são palavras reservadas.

A palavra "início" marca o início dos comandos a ser executados pelo computador. A lógica estará aqui. "Início" também é uma palavra reservada.

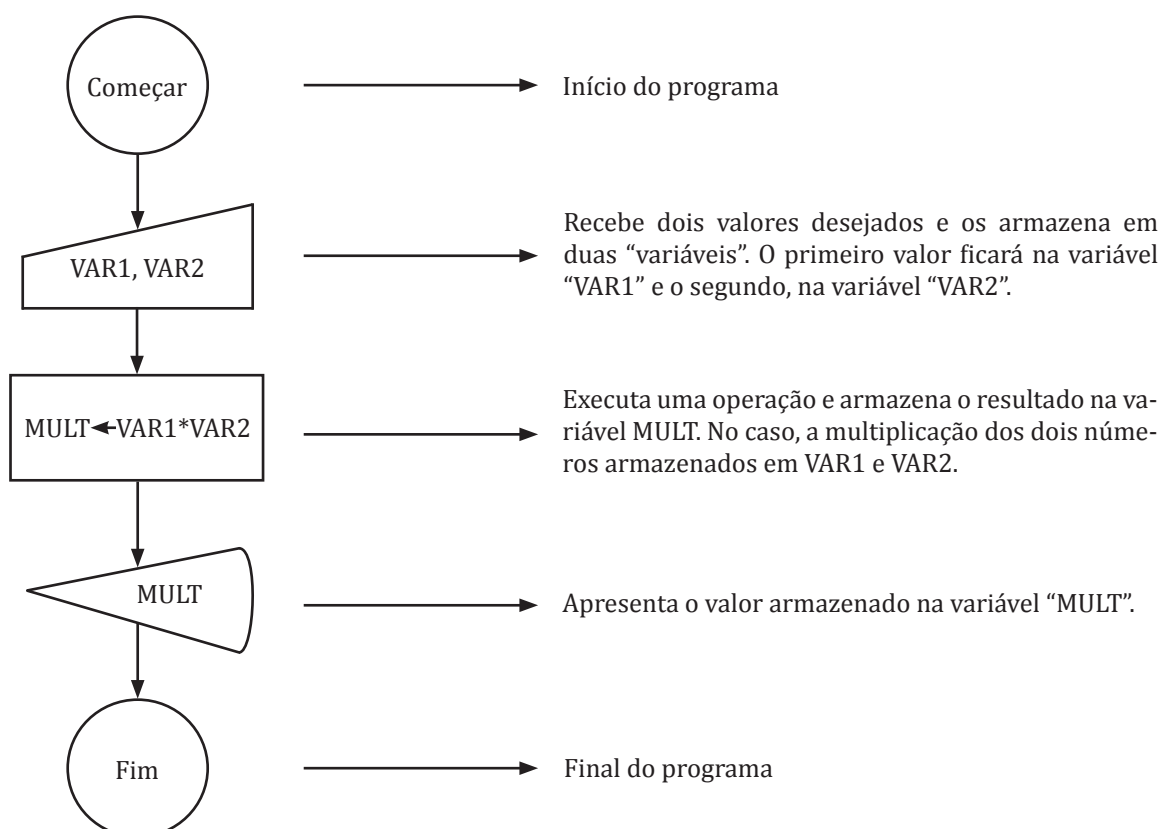
A palavra "FimAlgoritmo" também é uma palavra reservada que pode ser substituída apenas pela palavra "Fim", também reservada, indicando que a escrita do algoritmo terminou neste ponto.

Na escrita do algoritmo por meio do fluxograma, não temos preocupação com relação à escrita e às palavras reservadas como devemos ter ao escrevermos em Português Estruturado, pois esta é substituída por imagens gráficas.

Para escrever bons algoritmos, é necessário que você escreva seus algoritmos seguindo as regras de escrita, como a não acentuação das palavras e, em alguns casos, respeitando as letras maiúsculas e minúsculas. No entanto, nada impede que você escreva usando só letras maiúsculas ou só letras minúsculas. O VisuAlg escreve os comandos somente com letras minúsculas. Tal escolha ficará sob seu critério, sendo importante escrever os comandos com a sintaxe correta, seguindo as demonstrações contidas neste material.

Vamos analisar um exemplo: digamos que queremos escrever um possível algoritmo para multiplicar dois números quaisquer.

Resolvendo o algoritmo com auxílio do FreeDFD



Início do programa

Recebe dois valores desejados e os armazena em duas "variáveis". O primeiro valor ficará na variável "VAR1" e o segundo, na variável "VAR2".

Executa uma operação e armazena o resultado na variável MULT. No caso, a multiplicação dos dois números armazenados em VAR1 e VAR2.

Apresenta o valor armazenado na variável "MULT".

Final do programa

Resolvendo o algoritmo com auxílio do VisuAlg

algoritmo "multiplicar"

// Função : Multiplica dois números

// Autor : Prof. Luiz Carlos

// Data : 10/11/2013

// Seção de Declarações

var

NUMERO1, NUMERO2, MULTIPLICAR: inteiro

Seção de descrição do algoritmo.

“//” significa uma linha de comentário. Isto quer dizer que essa linha não será interpretada pelo computador, sendo apenas informação.

Declaração das variáveis, necessárias ao armazenamento dos valores a serem usados.

inicio

leia (NUMERO1, NUMERO2)

Comando para ler dois valores a armazenar cada valor em uma variável do programa, valor um em “NUMERO1”, valor dois em “NUMERO2”;

MULTIPLICAR: = NUMERO1 * NUMERO2

Execução da multiplicação dos dois valores lidos e armazenamento na variável “MULTIPLICAR”;

escreva (MULTIPLICAR)

Comando que mostra na tela o valor armazenado na variável “MULTIPLICAR”.

fimalgoritmo

Fim da descrição do algoritmo.

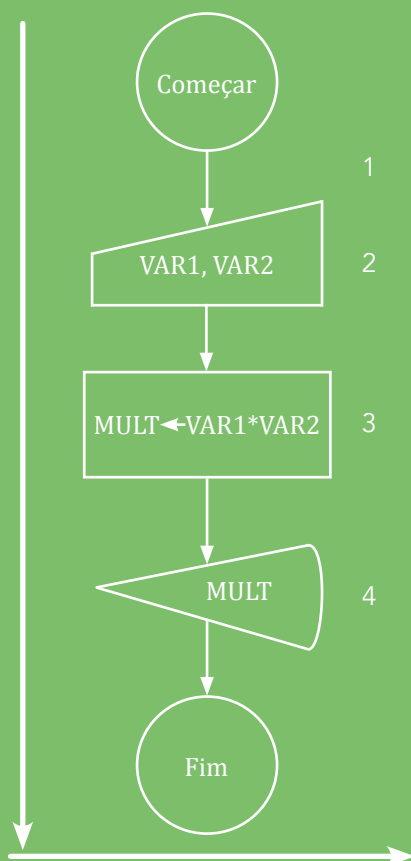
Observe que no algoritmo representado pelo fluxograma definimos nomes de variáveis diferentes. No fluxograma, foram: VAR1, VAR2 e MULT. No Português Estruturado, trabalhamos com: NUMERO1, NUMERO2 e MULTIPLICACAO a fim de melhorar a visualização. Em seguida, você verá que pode atribuir qualquer nome a uma variável.

O objetivo deste é apenas mostrar o corpo do algoritmo, ou seja, a estrutura obrigatória que um algoritmo deve ter. Daqui para frente, discutiremos as estruturas complementares de formação dos algoritmos, bem como os comandos que já apresentamos nos exemplos anteriores, os quais tinham por objetivo fazê-lo pensar e entender a construção de algoritmos em sua lógica.

SAIBA MAIS

Lendo algoritmos - Para que possamos construir bons algoritmos, é necessário que saibamos lê-los e entendê-los, o que possibilitará construir novos algoritmos e testes mentais, que dispensam o auxílio de ferramenta computacional. Para isso, é necessário que você saiba exatamente como o computador trabalha a interpretação de um algoritmo estruturado. Usaremos o exemplo anterior para explicar:

Com o fluxograma, o fluxo da interpretação por meio das setas apontando sempre para baixo já é visível, o que significa que um comando do algoritmo só poderá ser executado após o comando anterior ter sido executado. Desta forma, o algoritmo é sempre executado de cima para baixo e da esquerda para a direita.



Neste caso, o primeiro comando é um comando de entrada² para o qual definimos que entrariam dois valores (VAR1 e VAR2).

Após a entrada, há um comando de processamento³ onde temos a seguinte expressão:

MULT <- VAR1 * VAR2

Isto significa que primeiramente é criada uma variável chamada MULT para posteriormente inserirmos um valor calculado, que será igual à multiplicação da variável VAR1 pela VAR2. Caso não fosse assim, não seria possível armazenar o valor para ser usado posteriormente.

O último comando é um comando de saída⁴, que mostrará na tela o valor da que está armazenado na variável MULT.

Conforme sabemos, há diferenças entre um algoritmo representado em um fluxograma e o algoritmo escrito com o Português Estruturado.

Vamos trabalhar a interpretação de um algoritmo escrito em Português Estruturado. Novamente usaremos o exemplo anterior.

algoritmo "multiplicar"

// Função : Multiplica dois números

// Autor : Prof. Luiz Carlos

// Data : 10/11/2013

// Seção de Declarações

var

NUMERO1, NUMERO2, MULTIPLICAR: inteiro

inicio

leia (NUMERO1, NUMERO2)

MULTIPLICAR: = NUMERO1 * NUMERO2

escreva (MULTIPLICAR)

fimalgoritmo

1 – No Português Estruturado, primeiramente criamos a estrutura que precisaremos utilizar durante a execução, algoritmo e nome. Em seguida, definimos as "variáveis" por meio do espaço denominado "Var".

2 – Criamos as "variáveis" NUMERO1, NUMERO2 e MULTIPLICAR.

3 – Neste ponto, realmente iniciamos os comandos do algoritmo.

4 – Comando para ler dois valores inseridos pelo teclado e armazená-los nas variáveis NUMERO1 e NUMERO2.

5 – Executamos uma operação armazenando seu valor na variável MULTIPLICAR.

6 – Este comando fará com que o computador mostre na tela o valor da variável MULTIPLICAR.

7 – Fim da execução do algoritmo.

Agora realize a seguinte atividade:

Seja o computador. Use papel e caneta e leia o algoritmo passo a passo, interpretando os comandos. Atribua valores aleatórios para cada uma das variáveis e veja o resultado.

EXERCÍCIO PROPOSTO

1. Utilize as ferramentas *FreeDFD* e *VisuAlg* para escrever os algoritmos acima e testá-los. Após os testes, faça com que no comando “escreva” apareça não apenas os valores da multiplicação, como também das duas variáveis com valores armazenados.
2. Relate o que pensa sobre as duas formas de escrita de algoritmos trabalhadas até o momento, assim como a importância dos programas de auxílio à escrita de algoritmos para seu aprendizado nesta disciplina.

Variáveis

UN 01

Uma variável é um espaço de memória no qual podemos armazenar determinado valor para manipulá-lo de acordo com nosso desejo. É uma espécie de recipiente criado para receber dados.



27

Já vimos em exemplos anteriores o uso de variáveis na manipulação dos dados para o armazenamento de valores fornecidos, bem como de valores de cálculos que posteriormente serão mostrados na tela do computador.

A sequência de imagens abaixo apresenta o comportamento das variáveis do algoritmo do exemplo anterior, no qual criamos um algoritmo para multiplicar dois números. Na sequência, discutiremos a execução do algoritmo quanto às variáveis. Vejamos:

Momento 1: Criação das Variáveis.

algoritmo “multiplicar”

// Função : Multiplica dois números

// Autor : Prof. Luiz Carlos

// Data : 10/11/2013

// Seção de Declarações

var

NUMERO1, NUMERO2, MULTIPLICAR: inteiro

inicio

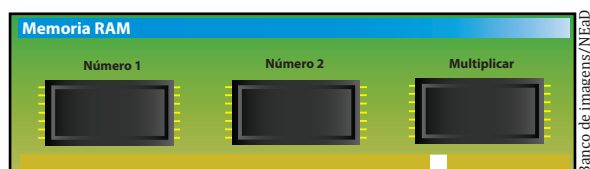
leia (NUMERO1, NUMERO2)

MULTIPLICAR: = NUMERO1 * NUMERO2

escreva (MULTIPLICAR)

fimalgoritmo

Memória RAM do computador: No momento em que iniciamos a execução do algoritmo:



Neste momento, é informado ao computador que devem ser criadas três variáveis do tipo inteiro (NUMERO1, NUMERO2, MULTIPLICAR). Assim, são criados na memória RAM do computador três espaços, cada um com seu nome, ou seja, criam-se três variáveis na memória RAM do computador.

Momento 2: Atribuindo valores às variáveis.algoritmo “multiplicar”

// Função : Multiplica dois números

// Autor : Prof. Luiz Carlos

// Data : 10/11/2013

// Seção de Declarações

var

NUMERO1, NUMERO2, MULTIPLICAR: inteiro

inicio

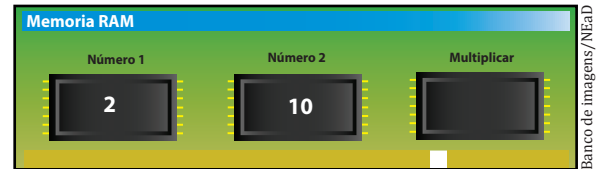
leia (NUMERO1, NUMERO2)

MULTIPLICAR: = NUMERO1 * NUMERO2

escreva (MULTIPLICAR)

fimalgoritmo

Neste momento, o computador aguarda dois números vindos do teclado para armazenar nas variáveis NUMERO1 e NUMERO2. Digamos que o usuário do computador digitou os números “2” e “10”, respectivamente.

**Momento 3:** Atribuindo valor à variável com o cálculo de duas outras variáveis.algoritmo “multiplicar”

// Função : Multiplica dois números

// Autor : Prof. Luiz Carlos

// Data : 10/11/2013

// Seção de Declarações

var

NUMERO1, NUMERO2, MULTIPLICAR: inteiro

inicio

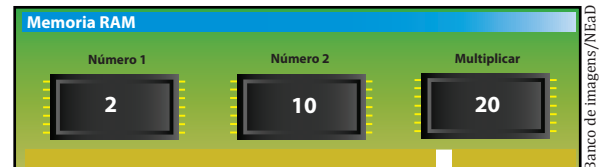
leia (NUMERO1, NUMERO2)

MULTIPLICAR: = NUMERO1 * NUMERO2

escreva (MULTIPLICAR)

fimalgoritmo

O cálculo do valor da variável NUMERO1, que tem o valor dois armazenado, multiplicado pela variável NUMERO2, que tem o valor 10, será armazenado na variável MULTIPLICAR, que não tinha nenhum valor.



Momento 4: Mostrando valores armazenados em uma variável.

algoritmo “multiplicar”

// Função : Multiplica dois números

// Autor : Prof. Luiz Carlos

// Data : 10/11/2013

// Seção de Declarações

var

NUMERO1, NUMERO2, MULTIPLICAR: inteiro

inicio

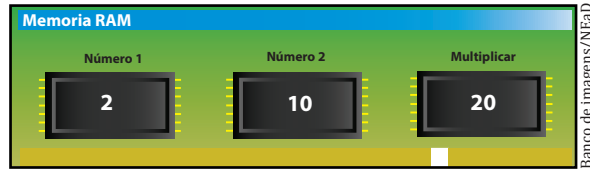
leia (NUMERO1, NUMERO2)

MULTIPLICAR = NUMERO1 * NUMERO2

escreva (MULTIPLICAR)

fimalgoritmo

Neste momento, o comando “escreva” mostrará o valor armazenado na variável MULTIPLICAR. No nosso exemplo, é o valor vinte.



Podemos fazer as seguintes observações:

No momento 1: Ao serem criadas as variáveis, não existe nenhum valor dentro delas. Representamos nenhum valor com a expressão “NULL”.

No momento 2: A ordem de atribuição dos valores das variáveis segue a ordem da interpretação, ou seja, da esquerda para a direita. Assim, o primeiro valor digitado ficará na variável mais à esquerda, e assim sucessivamente.

No momento 3: Todas as variáveis estão com valores armazenados e assim permanecerão até o fim da execução do algoritmo.

No momento 4: As variáveis persistem com os valores armazenados, até que ao fim do algoritmo o programa é destruído na memória RAM, acabando com todos os valores e estruturas criadas até o momento.

As variáveis são essenciais quando queremos trabalhar com valores, e devem ser usadas sempre que precisamos manipular algum valor. Desta forma, usamos as variáveis para três finalidades distintas:

1. Armazenar Dados de Entrada: São variáveis criadas para armazenarmos os valores que o algoritmo precisa receber, oriundos do mundo real, entrada de dados, como números e nomes.

2. Armazenar Dados de Saída: Usamos estas variáveis quando queremos armazenar os resultados das operações, para que possamos manipular ou enviar para o mundo exterior, isto é, prover a saída de dados.

3. Armazenar Dados de Processamento: As variáveis poderão armazenar informações de cálculos que estão sendo executados. Como exemplo, uma soma de números que será necessária para o cálculo da média, para posterior cálculo ou saída de dados.

Podemos concluir, portanto, que as variáveis são responsáveis por duas características de um algoritmo computacional: possibilidade de receber dados do mundo exterior e capacidade de fornecer dados ao mundo exterior, assim como podem auxiliar no processamento de dados por meio do armazenamento de valores resultantes de operações.

Bom, mas para criarmos as estruturas das variáveis usando o Português Estruturado, precisamos definir o tipo de dado que esta variável receberá, o que é assunto do próximo tópico.

Tipos de variáveis

As variáveis podem ser do tipo primitivo, ou seja, pré-definidas por uma linguagem ou notação, e de referências, que referenciam os tipos primitivos e formam estruturas mais complexas para armazenamento de dados (FORBELLONE; EBERSPACHER, 2005). Neste tópico, apresentaremos as principais variáveis do tipo primitivo. Que são:

• Numéricas

São as variáveis que irão armazenar os valores dos números, se dividem em dois grupos:

Inteiro: São variáveis que podem receber valores numéricos que formam o conjunto dos números inteiros. Podemos citar como exemplos:

10

20

-345

1034

Real: Neste tipo de variável, podemos atribuir valores pertencentes ao conjunto dos números reais, por exemplo:

0.23

1.23

1

100

100.78888

• Caractere

Neste tipo de variável, podemos manipular valores do tipo caracteres, que são as letras (a..z; A..Z), os números (0..9) e os símbolos especiais (!, #, ", *...). Dados desse tipo são mais comumente usados na representação de mensagens de texto, e são representados sempre pelo uso das aspas (" ") (MANZANO; OLIVEIRA, 2009). Vejamos alguns exemplos:

"casa"

"123"

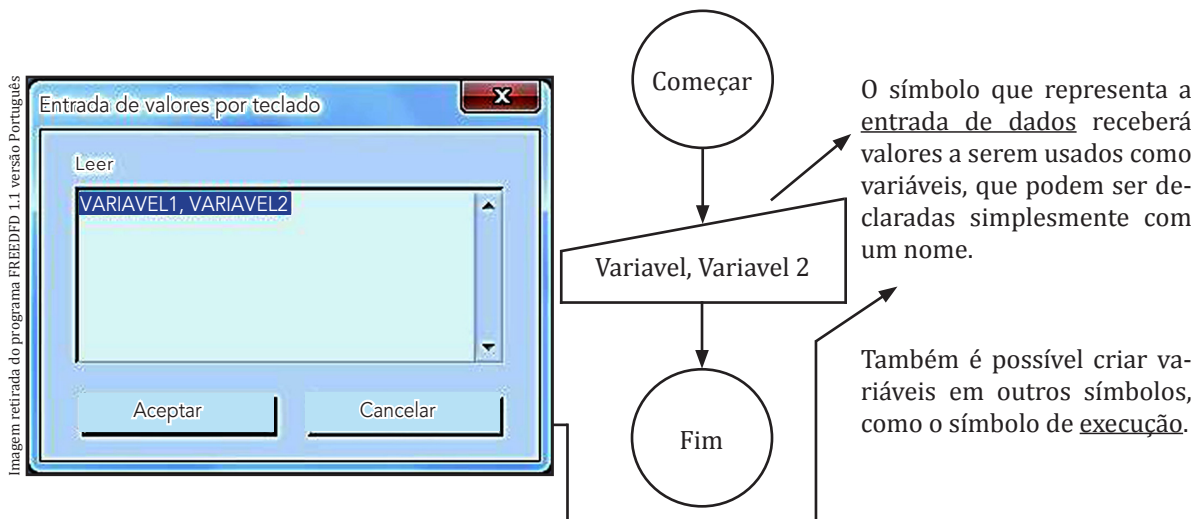
" eu faço computação..."

• Lógico

Este tipo de dado é usado para o armazenamento de apenas dois valores, chamados de valores lógicos, que são: .verdadeiro. e .falso. ou valores como .sim. ou não.; enfim, valores resultantes de operações lógicas. É bastante utilizado para operações de comparações lógicas, conforme veremos adiante.

Definindo variáveis

Em exemplos anteriores, já trabalhamos a definição das variáveis tanto no Português Estruturado quanto nos fluxogramas. No fluxograma, não precisamos definir o tipo de uma variável, apenas damos um nome a variável, conforme exemplo abaixo:



Quando usamos o fluxograma, devemos ter cuidado ao inserir valores em variáveis, respeitando o tipo de dados que poderemos trabalhar, por exemplo: se formos trabalhar com operações matemáticas, usamos apenas números nas variáveis.

No caso do Português Estruturado, já vimos no tópico anterior que existe uma palavra chave para definirmos as variáveis, que é a palavra Variavel ou simplesmente Var, que fica abaixo da seção de identificação do algoritmo. O exemplo abaixo mostra como definimos as variáveis usando o Português Estruturado:

Algoritmo "Declara_Variavel"

Var

<Nome_da_variavel> : <Tipo_da_Variavel>

Início

Fim

O nome da variável se refere ao nome pelo qual desejamos chamá-la, constituindo o identificador do espaço criado na memória. Pode ser qualquer nome desejado, desde que atentemos para as seguintes regras:

1. O nome deve começar por letras e nunca por números.

Exemplo: Errado (1S; 23AB; 1NUMERO) Certo (S1; AB23; NUMERO1);

2. O nome não pode conter caracteres especiais.

Exemplo: Errado (\$A1; @NUM; NUM@1);

3. Não pode haver espaços em branco no nome.

Exemplo: Errado (Maria Luiza; Joao 2) Certo: (MariaLuiza; Joao2).

4. As variáveis não podem ter os mesmos nomes das palavras reservadas.

Exemplo: Leia, escreva, escreval, var...

Uma dica importante é que você sempre use nomes significativos para suas variáveis, ou seja, nomes que façam sentido e dos quais você sempre se lembre, o que ajudará na hora de interpretarmos um algoritmo.

Se desejar, use comentários para melhorar o sentido de suas variáveis. Exemplo:

Algoritmo “DeclaraVariavel”

Var

```
NOME: Caracter;           //Nome do objeto
IDADE: inteiro;           //Idade do objeto
PRECO: real;              //Valor de custo do objeto
PROCEDENCIA: Logico;      //Existe ou não procedência do objeto
```

Inicio

Fim

Constantes

UN 01

Ao contrário de uma variável, uma constante é um valor fixo que pode ser usado durante todo o programa. Assim, podemos ter uma constante simplesmente como um número fixo ou mesmo uma variável de valor predeterminado, que não mudará durante o algoritmo. Vejamos o exemplo a seguir:

Algoritmo DeclaraVariavel

Var

```
NOME: Caracter;           //Nome do objeto
IDADE: inteiro;           //Idade do objeto
PRECO: real;              //Valor de custo do objeto
PROCEDENCIA: Logico;      //Existe ou não procedência do objeto
```

Inicio

```
NOME := “Luiz”           //Atribuindo valores a variável NOME
IDADE:= 34                //Atribuindo o valor a variável IDADE
```

Fim

No exemplo acima, apesar de termos uma variável, definimos um valor e não o alteramos ao longo do algoritmo. Podemos dizer, portanto, que o valor é constante durante todo o algoritmo, não variando, pois não há atribuição de valores no decorrer da execução do algoritmo.

Na literatura, como em Manzano e Oliveira (2010), podemos encontrar a expressão “const” antes da declaração da variável para afirmar que teremos um espaço de memória constante, que não poderá sofrer alterações, e estabelecido com um valor previamente definido, conforme o exemplo a seguir, que mostra como representar a constante PI em um algoritmo:

algoritmo “constante”**const**

PI = 3.1415

var

Area : Real

R: Real

inicio

// Seção de Comandos

R:= PI*200

fimalgoritmo

Já no VisuAlg não temos a opção real de a declaração de uma constante, ficando a representação da constante de PI mais ou menos desta forma:

algoritmo “constante”

// Função : Demonstrar constantes

// Autor : Prof. Luiz Carlos

// Data : 21/01/2014

// Seção de Declarações

var

Area : Real

R, P : Real

inicio

// Seção de Comandos

P: = 3.1415

R: = P * 200

fimalgoritmo

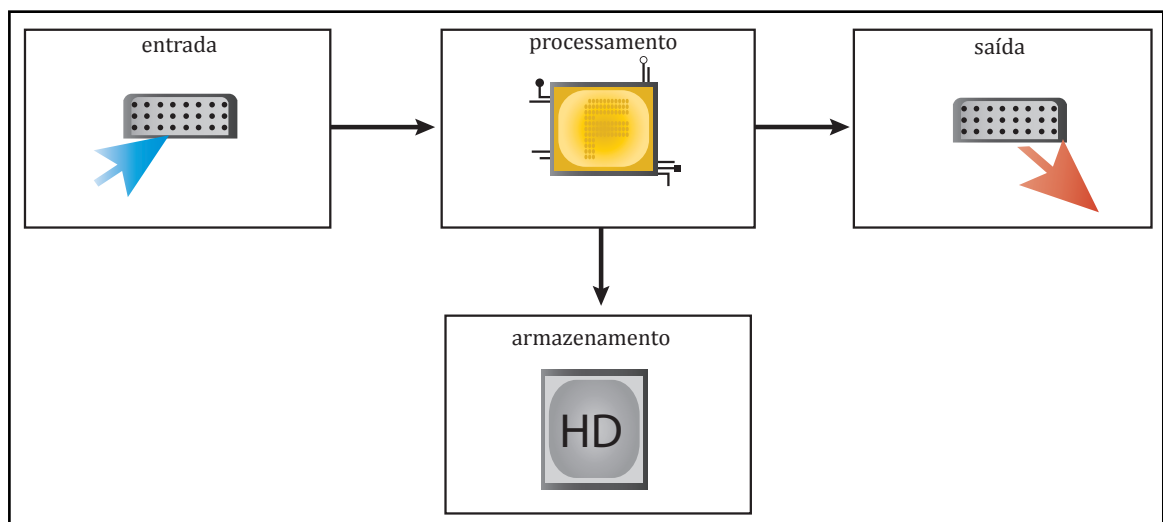
Além de PI, para o qual usamos uma variável do tipo real para armazenar o valor de sua constante, usamos também um valor fixo “200” (constante) na expressão “R:= P*200”, que não deixa de ser a representação de uma constante na expressão que desejamos calcular.

Mas quando realmente devemos armazenar um valor na memória como constante? Bom, sempre que precisarmos utilizar este valor mais de uma vez no algoritmo. Em caso contrário, podemos inserir o valor diretamente, conforme mostrado no exemplo acima.

Entrada e saída de dados

UN 01

Bom, no começo deste caderno afirmamos que um algoritmo deve permitir a entrada de dados do mundo real e a saída de dados para o mundo real. Para isso, usamos os comandos de entrada e saída de dados, conforme já visto. A figura abaixo representa a arquitetura básica do computador:



Banco de imagens/NEaD

34 Entrada de dados

UN 01

O comando de entrada de dados no Português Estruturado é denominado “Leia”, conforme mostrado abaixo:

Algoritmo DeclaraVariavel

Var

NOME: Caracter;	//Nome do objeto
IDADE: inteiro;	//Idadedo objeto
PRECO: real;	//Valor de custo do objeto
PROCEDENCIA: Logico;	//Existe ou não procedência do objeto

Inicio

```

leia (NOME)

leia (IDADE, PRECO)

leia (PROCEDENCIA)

```

Fim

No exemplo acima, temos o comando “leia” atrelado a uma variável. Em:

```
leia (NOME)
```

O comando irá esperar que um dado do tipo “character” seja digitado no teclado. Após a digitação, deverá ser pressionada a tecla enter. Neste momento, o valor digitado passará a ser armazenado dentro da variável referenciada no comando “leia”. No caso, a variável NOME.

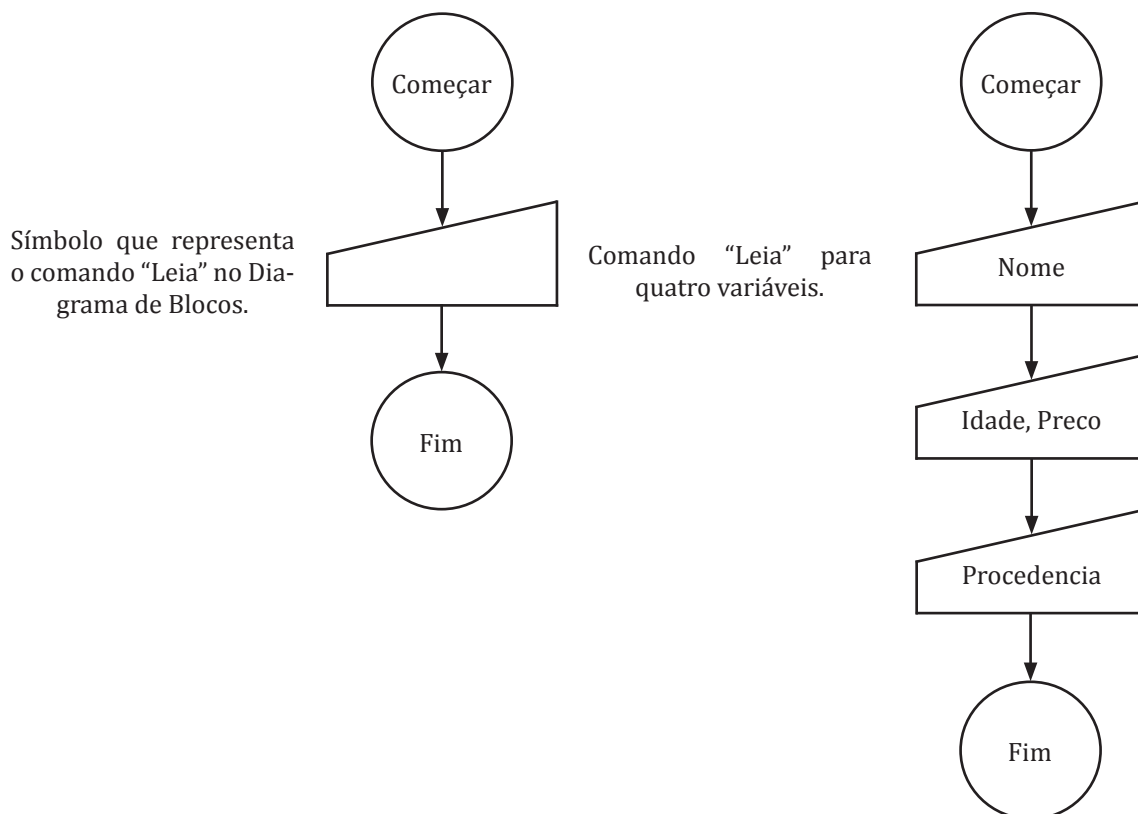
Podemos ler diversos valores a partir do teclado e armazená-los em diversas variáveis, como no exemplo:

leia (IDADE, PRECO)

Neste exemplo, temos dois valores que serão entrados via teclado e armazenados nas referidas variáveis, seguindo a ordem da esquerda para a direita de interpretação dos comandos.

Deste modo, sempre que usarmos o comando “leia”, devemos ter em mente que o comando representa a leitura de determinado valor oriundo do teclado do computador. E se vamos ler um valor, é conveniente que o armazenemos em uma variável.

Quando usamos diagrama de blocos, o bloco que corresponde ao comando “leia” é:



Saída de dados

O comando de saída de dados no Português Estruturado é denominado “Escreva”, e mostra uma mensagem ou valor de uma variável na tela do computador, conforme mostrado abaixo:

Algoritmo DeclaraVariavel

Var

```

NOME: Caracter;           //Nome do objeto
IDADE: inteiro;           //Idade do objeto
PRECO: real;              //Valor de custo do objeto
PROCEDENCIA: Logico;      //Existe ou não procedência do objeto

```

Início

leia (NOME)

leia (IDADE, PRECO)

leia (PROCEDENCIA)

Escreva (NOME)

Escreva("sua idade é: ", IDADE)

Escreva(PRECO, PROCEDENCIA)

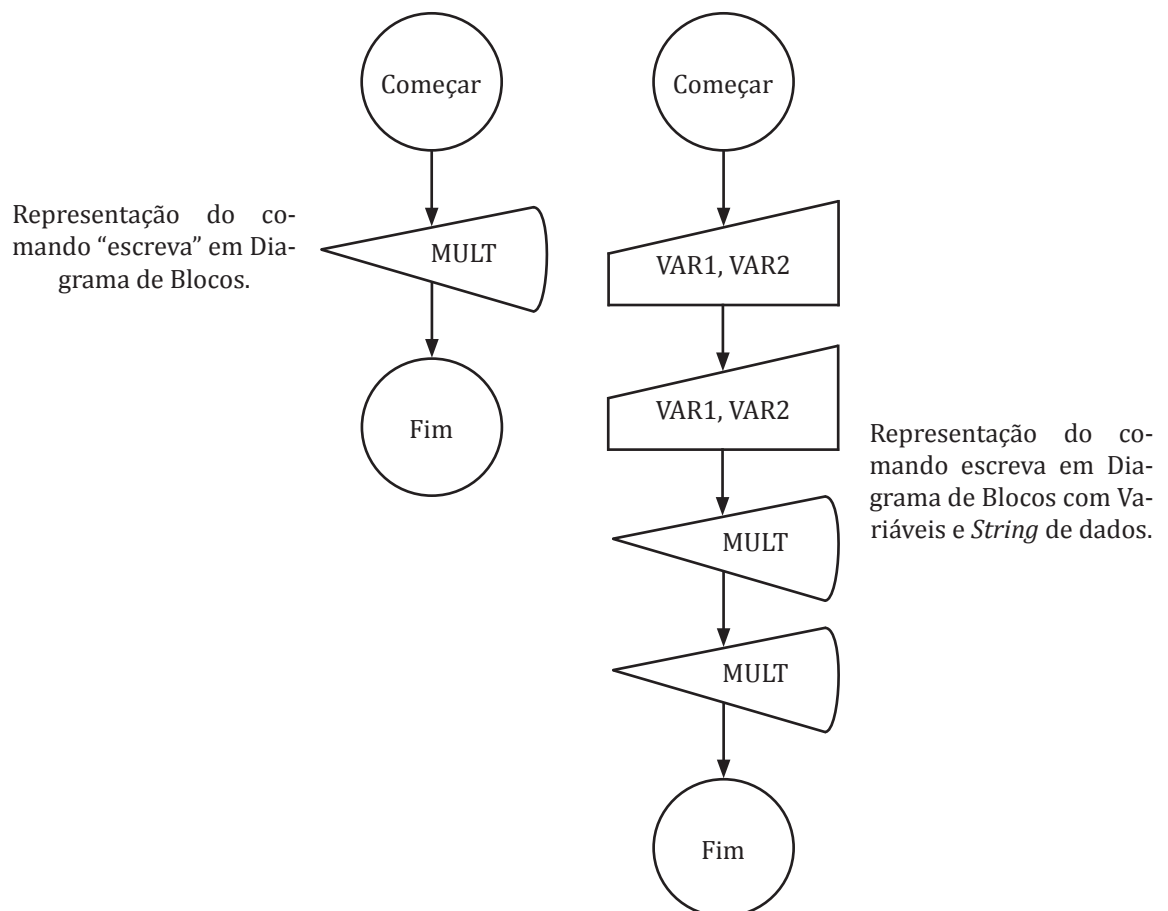
Fim

O comando escreva pode tanto mostrar um valor armazenado em uma variável quanto uma expressão que você deseje apresentar na tela, como no exemplo abaixo:

Escreva("sua idade é: ", IDADE)

Neste exemplo, a expressão entre aspas "sua idade é:" significa que é uma *string* de caracteres e não deve ser interpretada pelo computador, mas apenas mostrada na tela. A vírgula separa a *string* de caracteres de outra palavra, que é uma variável. Neste caso, IDADE, no lugar da qual será apresentado o valor armazenado dentro da variável IDADE.

A figura abaixo apresenta o comando de saída de dados quando usamos o Diagrama de Fluxo de dados:



Também podemos usar o comando escreva para nos apresentar apenas uma informação, uma orientação do que deve ser feito com relação ao algoritmo. Este tipo de uso é comum para orientar sobre a inserção de dados em uma variável, sendo o comando escreva colocado antes do comando leia, apenas com uma *string* de mensagem, conforme veremos na próxima seção.

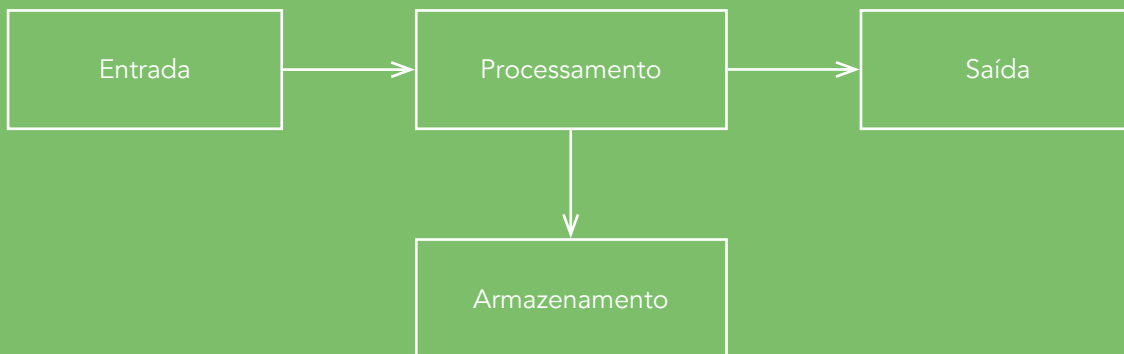
SAIBA MAIS

Resolvendo algoritmos: Identificando Variáveis - As variáveis são os espaços alocados na memória para que possamos armazenar valores. Assim, é necessário que conheçamos bem um problema a fim de identificar as variáveis a partir do enunciado de um problema, como também de sua interpretação lógica.

Vejamos o exemplo resolvido:

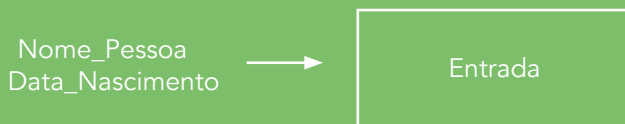
Crie um algoritmo que receba o nome da pessoa e o ano de nascimento e apresente na tela o nome e a idade dessa pessoa.

Propomos que você entenda o algoritmo segundo a arquitetura do computador, como mostrado a seguir:

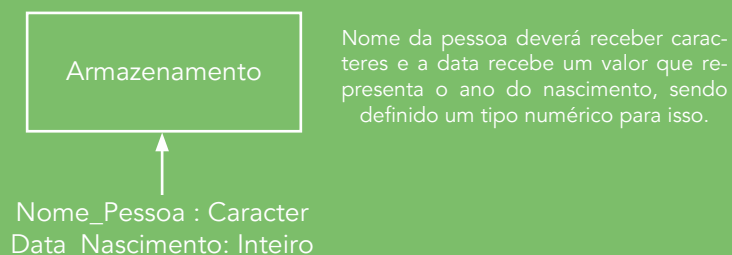


Assim, iniciamos o processo de resolução do enunciado do problema.

Entradas: Será necessário entrarmos com os dados do nome da pessoa e o ano de nascimento. Pois bem, se vamos entrar com valores, devemos armazená-los. Em caso contrário perderemos estes valores ao serem digitados, temos:



Já que precisamos armazenar estes valores, devemos criar variáveis para armazenamento e definir seu tipo:



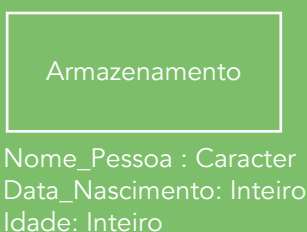
O próximo passo é definir nosso processamento, ou seja, o que será calculado. No exemplo, devemos calcular a idade de uma pessoa a partir do ano de seu nascimento subtraído do ano atual (no caso, 2013).

Idade:= 2013 - Data_Nascimento



Como devemos efetuar um cálculo e este valor deverá ser apresentado posteriormente, criaremos uma nova variável denominada "idade" para armazenar o valor do cálculo, que será um valor numérico do tipo inteiro.

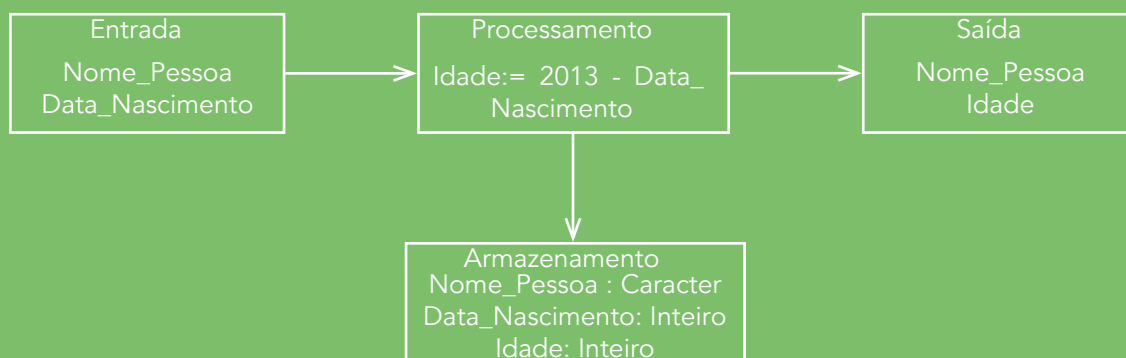
Assim, criaremos uma nova variável denominada de Idade para armazenar o valor do cálculo.



Por fim, devemos definir as saídas do nosso algoritmo, que serão o nome e a idade da pessoa, já armazenados nas variáveis após as entradas de valores e do processamento.



Assim, temos a seguinte representação do algoritmo por meio da arquitetura do computador:



Bom, agora basta construir o algoritmo usando o VisuAlg seguindo as convenções das imagens do diagrama de bloco do hardware do computador com os comandos já vistos:

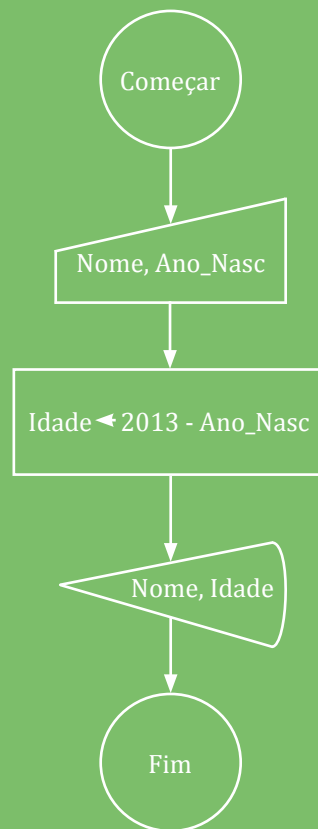
```

algoritmo "calculaidade"
// Função : Calcular a idade de uma pessoa a partir do ano de nascimento
// Autor: Prof. Luiz Carlos
// Data: 21/01/2014
// Seção de Declarações
var
  Nome_Pessoa: Caracter
  Data_Nascimento: Inteiro
  Idade: Inteiro
inicio
// Seção de Comandos
Escreval("Entre com o nome da pessoa")
Leia (Nome_Pessoa)
Escreval("Entre com o ano de nascimento")
Leia (Data_Nascimento)
Idade:= 2013 - Data_Nascimento
Escreva (" A idade de: ", Nome_Pessoa, " é ", Idade)
Fimalgoritmo
  
```

Observação:

No VisuAlg, quando acrescentamos a letra "l" ao comando "Escreva", tornando-o "Escreval", significa que depois de apresentar o que deve ser mostrado, o comando saltará para uma nova linha.

Usando o FreeDFD:



39

EXERCÍCIO PROPOSTO

No algoritmo anterior, para descobrirmos a idade de uma pessoa, diminuimos o ano de nascimento pelo ano atual. No entanto, o algoritmo leva em consideração o ano de 2013. E se estivéssemos em 2010? 2018? 2230? O algoritmo não funcionaria corretamente, concorda? De tal modo, refaça o algoritmo do exemplo acima para resolver sua falha. Teste o algoritmo com as ferramentas FreeDFD e VisuAlg.

1. Supomos que precisamos armazenar os valores abaixo. Indique o tipo de dado mais apropriado para armazenar estes valores em uma variável:

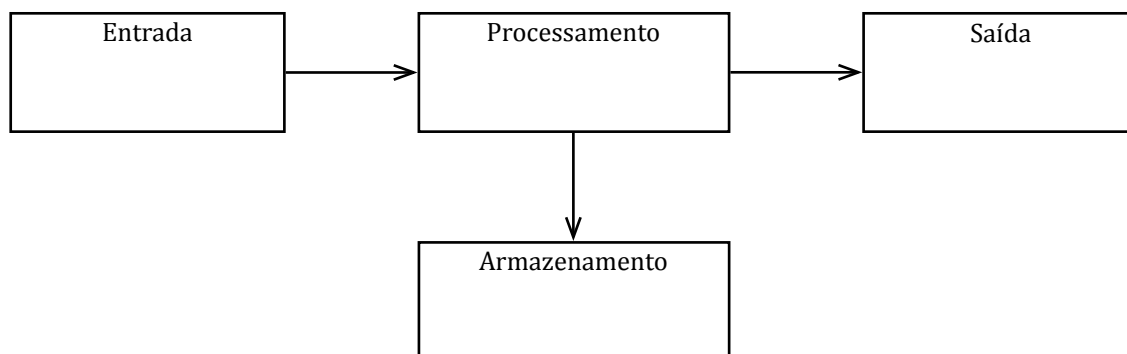
- () Francisco_31
- () 1.345
- () 1,345663
- () #"%
- () 1998
- () 456,654
- () Verdadeiro
- () 1.234Adddh
- () 1,0
- () Falso

Use a seguinte numeração:

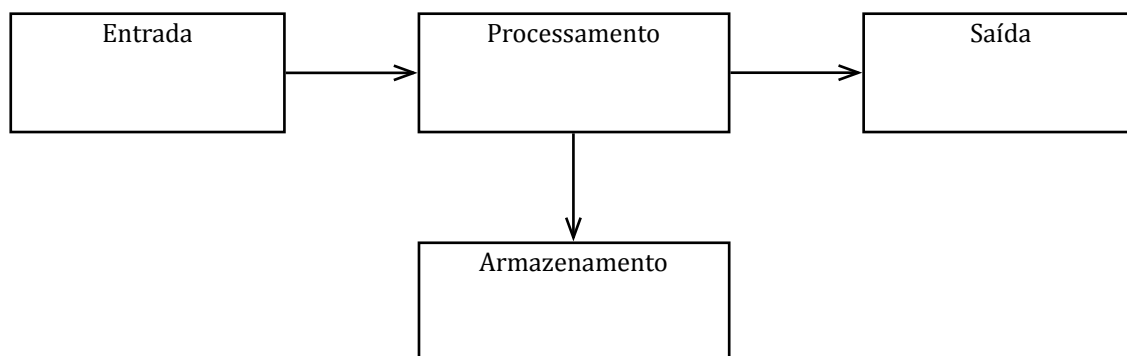
- (1) Tipo Inteiro
- (2) Tipo Real
- (3) Tipo Caracter
- (4) Tipo Booleano

2. Identifique as variáveis e os tipos das entradas e saídas nas descrições dos algoritmos abaixo, alocando-os de acordo com a representação da arquitetura de um computador:

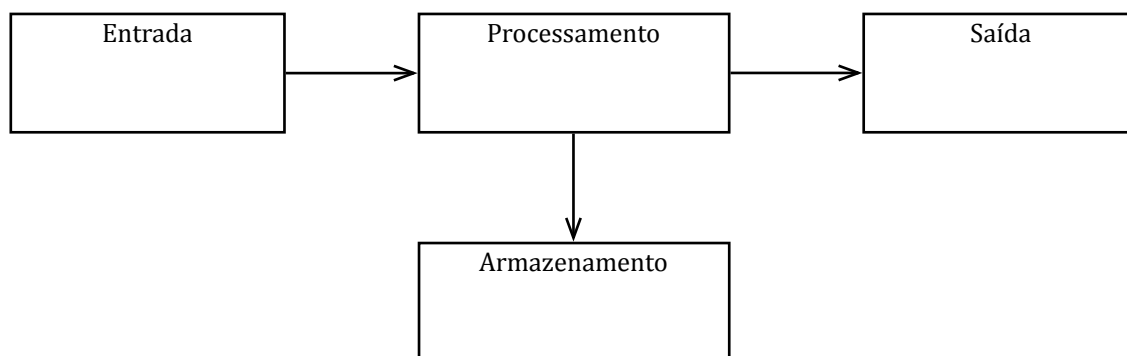
a) Construa um algoritmo que receba o nome, endereço e a data de nascimento de uma pessoa e mostre todos os dados na tela.



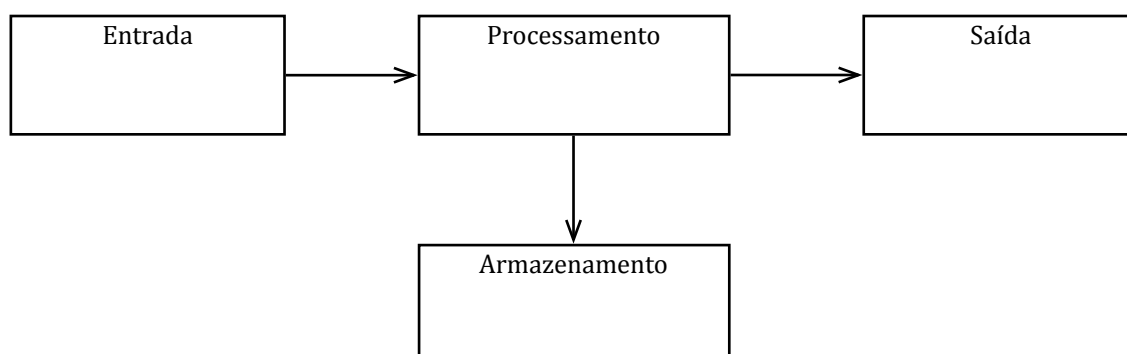
b) Construa um algoritmo para calcular a área de um círculo.



c) Construa um algoritmo para calcular a raiz quadrada de um número qualquer fornecido pelo usuário.



d) Construa um algoritmo que receba dois números e calcule a soma do quadrado destes dois números.



Trabalhando a construção de expressões

UN 01

Até o presente momento, trabalhamos a questão da entrada, armazenamento e saída de dados. Neste ponto, discutiremos as operações que podemos realizar no algoritmo na parte de processamento de comandos.

Atribuição

Atribuir em algoritmo significa designar um valor a algo. Já usamos o comando de atribuição em alguns exemplos neste caderno.

O comando de atribuição pode ser representado de duas formas:

Forma 1: Atribuição com o Símbolo “:=”

<variável> := <valor>

Forma 2: Atribuição com o símbolo “<-”

<variável> <- <valor>

Ambas as formas significam que o valor que temos à direita pertencerá à variável da esquerda.

Vejamos o exemplo de atribuição de valores a uma variável:

algoritmo “Atribuição”

var

Nome : Caracter

Idade: Real

inicio

Nome := “João”

Idade:= 23

Escreval (Nome, “tem: “ , Idade, “ anos de idade”)

fimalgoritmo

Neste algoritmo, temos definidas duas variáveis, às quais atribuímos os valores à variável Nome (que será: “João”) e à variável Idade (que será: 23). Tais valores serão posteriormente apresentados na tela por meio do comando “escreva”.

O comando de entrada de dados via teclado “Leia” atribui o valor digitado à variável que está referenciado. Exemplo:

algoritmo “Atribuição”

var

Nome : Character

Idade: Real

inicio

leia (Nome)

leia (idade)

Escreval(Nome, “tem: “, Idade, “ anos de idade”)

fimalgoritmo

Neste algoritmo, temos definidas duas variáveis, que receberão os valores vindos do teclado, sendo atribuídos tais valores às variáveis e posteriormente apresentados na tela.

Instrução e Comandos

As instruções são as ordens que o computador deve seguir para a realização. Podem ser consideradas uma ordem de execução de alguma tarefa a ser realizada, como um cálculo ou mesmo a atribuição de um valor a uma variável.

Os comandos podem ser definidos como um conjunto de instruções para a realização de determinada função no sistema. Como exemplo, temos os comandos de entrada e saída de dados “leia” e “escreva”, respectivamente.

Operadores e Expressão

Os operadores são os símbolos matemáticos que podemos utilizar para compor uma expressão, que por sua vez é a representação de uma ou várias operações utilizando os símbolos matemáticos.

Podemos utilizar os operadores aritméticos que já conhecemos da Matemática Básica. São eles:

SÍMBOLO	DESCRIÇÃO	EXEMPLO	COMENTÁRIO
+	Adição	A: = 10 + 5	A variável A receberá o valor da expressão 10 + 5
-	Subtração	A: = 10 - 5	A variável A receberá o valor da expressão 10 - 5
*	Multiplicação	A: = 10 * 5	A variável A receberá o valor da expressão 10 x 5
/	Divisão	A: = 10 / 5	A variável A receberá o valor da expressão 10 / 5

Observação:

A Divisão é uma operação crítica, pois se considerarmos os tipos de variáveis e nos lembrarmos de que os tipos devem ser respeitados, será comum encontrarmos a seguinte expressão:

Considere que uma variável X é do tipo inteiro. Veja a seguinte situação:

X:= 10/3

O valor resultante não será um número inteiro, mas um número real, causando um problema com o algoritmo, que não funcionará.

Para resolver este problema, temos os operadores:

SÍMBOLO	DESCRIÇÃO	EXEMPLO	COMENTÁRIO
DIV	Divisão por Inteiros	$X := 10 \text{ DIV } 3$	A variável X receberá o valor inteiro da divisão. No caso, $X = 3$.
MOD	Resto de uma Divisão por Inteiros	$X := 10 \text{ MOD } 3$	A variável X receberá o resto inteiro da divisão. No caso, $X = 1$.

Com esses comandos podemos, portanto, efetuar operações com variáveis do tipo inteiro, que resultarão em números inteiros.

Além dos operadores aritméticos convencionais, temos os seguintes operadores:

SÍMBOLO	DESCRIÇÃO	EXEMPLO	COMENTÁRIO
$\wedge$	Exponenciação	$X := 10 \wedge 3$	A variável X receberá o valor de 10 elevado a 3, $X = 1000$.
RAIZQ^()	Radiciação	$X := \text{RAIZQ}(10)$	A variável X receberá o valor da raiz quadrada de 10. No caso, $X = 3.12$.

01. O que acontece internamente na memória quando inserimos uma variável de tipo diferente do esperado?

No caso das variáveis, espaços na memória específicos para receber um único tipo de dado, quando criamos uma variável é alocado um espaço que pode variar de acordo com o tipo da variável, por exemplo: uma variável do tipo inteiro pode ser representada internamente apenas como um conjunto de 0 e 1's, pois podemos representar um número inteiro.



Banco de Imagens/NiEd

No caso de uma variável do tipo real, a estrutura deve estar preparada para armazenar mais 0 e 1's do que o necessário, devido à técnica de representação por ponto flutuante, por meio da qual podemos definir um número real a partir de números inteiros. Consequentemente, este tipo de variável ocupará mais memória do que as variáveis do tipo inteiro.

02. Por que temos diferentes tipos?

Porque cada tipo necessitará de uma quantidade de espaço na memória. Alguns ocuparão menos espaços e outros mais espaços. Por exemplo, a representação do número 1, se for inteiro, ocupará um espaço X de memória. Se a representação for para um número real, teremos um espaço de 1,2X. Ou seja, ocupará mais espaço que um inteiro.

03. São apenas quatro os tipos de variáveis?

Não, as variáveis dependem muito da linguagem de programação. As apresentadas aqui são as básicas, consideradas primárias. Existem outras variáveis complexas. Veremos algumas adiante neste caderno, que podem ser criadas a partir de estruturas mais elaboradas partindo de variáveis primitivas.

▶ EXERCÍCIO RESOLVIDO

Construa um algoritmo que receba dois números inteiros e apresente as quatro operações básicas com estes números inteiros (adição, subtração, multiplicação e divisão).

Para resolvermos este problema, levantaremos as entradas de dados que o sistema deverá receber. Desta forma, temos duas entradas de dados que serão dois números inteiros. Elas serão chamadas de NUM1 e NUM2.

Após as entradas, vamos trabalhar o processamento dos dados. Teremos que realizar as quatro operações. Assim, devemos ter quatro variáveis para armazenar o valor das quatro operações, que serão chamadas de:

Operação Soma: SOM, que será a soma dos dois números que recebemos na entrada. Ficará: $SOM := NUM1 + NUM2$;

Operação Subtração: SUB, que será a subtração dos dois números que recebemos na entrada. Ficará: $SUB := NUM1 - NUM2$;

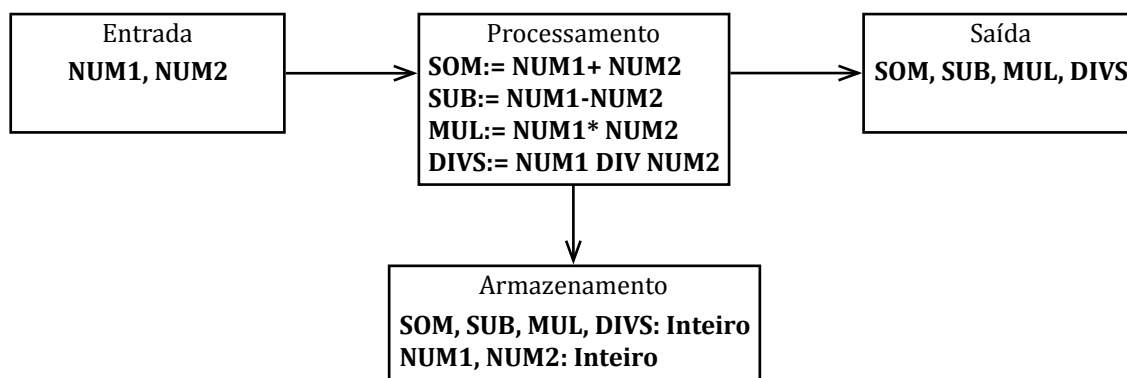
Operação Multiplicação: MUL, que será a multiplicação dos dois números que recebemos na entrada. Ficará: $MUL := NUM1 * NUM2$;

Operação Divisão: DIVS, que deverá ser uma divisão por inteiros, que será a divisão por inteiro dos dois números que recebemos na entrada. Ficará: $DIVS := NUM1 \text{ div } NUM2$.

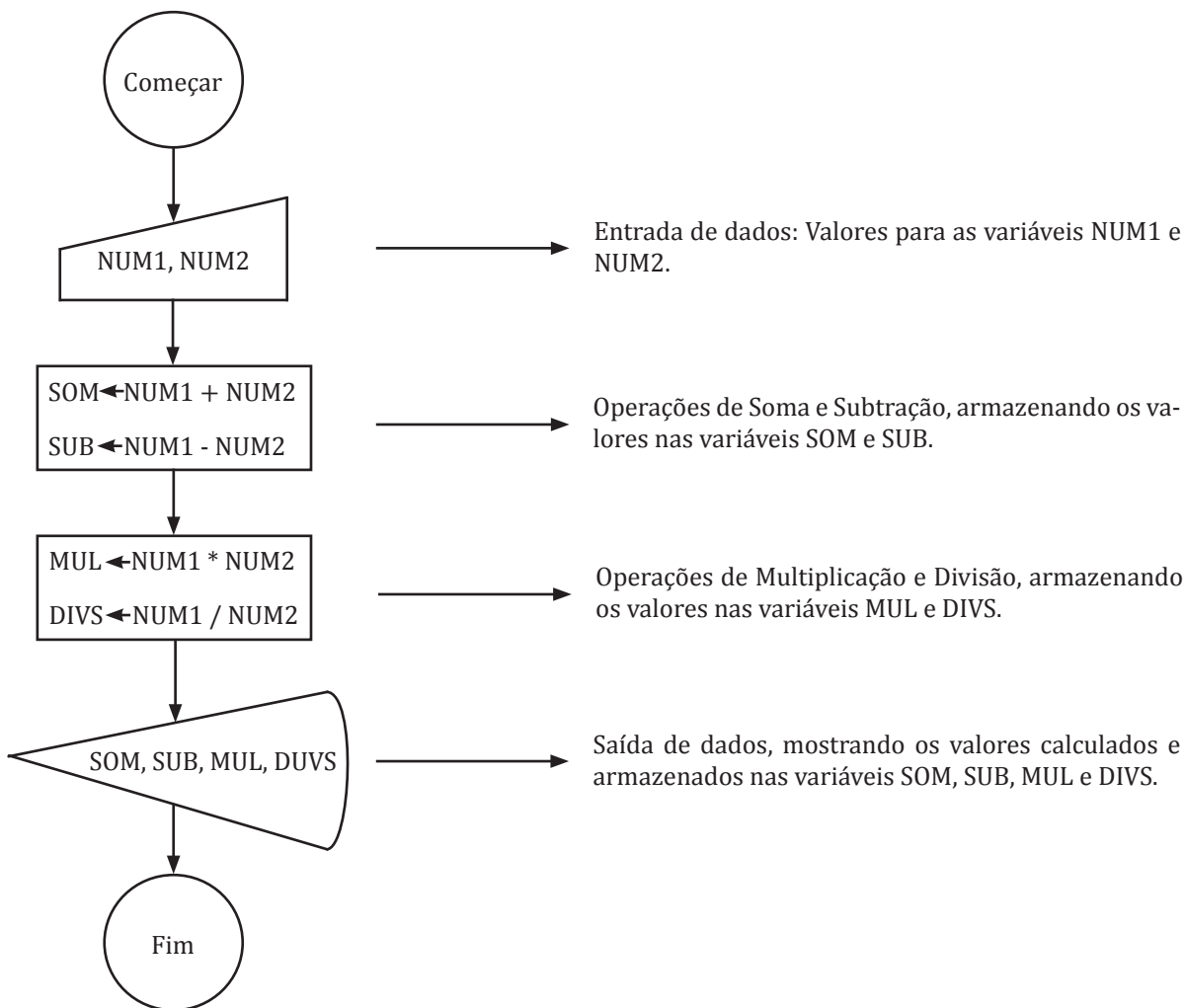
Agora definiremos a saída dos dados, constituída pelos valores armazenados nas variáveis SOM, SUB, MUL e DIVS.

Por fim, declaramos as variáveis que deverão armazenar os valores para a resolução de nosso algoritmo, que serão: SOM, SUB, MUL, DIVS, NUM1 e NUM2.

Nosso esquema de construção de algoritmo baseado na arquitetura básica do computador ficará da seguinte forma:



Construiremos agora o algoritmo representando na forma de fluxograma, com o auxílio do *FreeDFD*.



Agora usaremos o Português Estruturado para a representação do algoritmo com o auxílio da ferramenta *VisuAlg*:

algoritmo "operações"

// Função : Realizar as 4 operações com dois números quaisquer

// Autor : Luiz Carlos

// Data : 22/01/2014

var

NUM1, NUM2 : Real

SOM, SUB, MUL : Real

DIVS : Real

inicio

// Entrada de dados

Escreval ("entre com o primeiro número")

leia (NUM1)

Escreval ("entre com o segundo número")

```
//Processamento das operações
```

```
SOM:= NUM1 + NUM2
```

```
SUB:= NUM1 - NUM2
```

```
MUL:= NUM1 * NUM2
```

```
DIVS:= NUM1 div NUM2
```

```
//Saída de dados
```

```
Escreval ("A soma do número", NUM1, " com", NUM2, "é: ", SOM)
```

```
Escreval ("A subtração do número", NUM1, " com", NUM2, "é: ", SUB)
```

```
Escreval ("A multiplicação do número", NUM1, " com", NUM2, "é: ", MUL)
```

```
Escreval ("A divisão do número", NUM1, " com", NUM2, "é: ", DIVS)
```

```
finalgoritmo
```

▶ EXERCÍCIO PROPOSTO

1. Crie um algoritmo que receba uma mensagem qualquer e apresente-a na tela do computador.
2. Crie um algoritmo que receba dois números inteiros e apresente a divisão real destes.
3. Crie um algoritmo que receba um valor e calcule um número que seja o valor recebido elevado à potência dele mesmo.
4. Crie um algoritmo que calcule o valor em graus Fahrenheit (F) a partir de graus Celsius (°C), lembrando que a fórmula da conversão é $F = (9 * C + 160) / 5$.
5. Crie um algoritmo que calcule a raiz quadrada de um número digitado pelo usuário.
6. Crie um algoritmo que receba um valor em horas e informe este valor em segundos.
7. Crie um algoritmo que receba um nome e a data de nascimento de uma pessoa e informe o nome da pessoa, a idade em anos, a quantidade de meses e dias que essa pessoa já viveu, desconsiderando os anos bissextos para efeito de cálculo.
8. Crie um algoritmo que calcule determinado desconto, a ser informado pelo usuário, em cima de um valor a ser pago, também a ser informado pelo usuário.
9. Crie um algoritmo que receba um número referente a determinado dado do mundo real, depois o algoritmo deve receber um valor referente ao percentual que você deseja obter deste número. Ex.: Digamos que queremos calcular a quantidade de mulheres de uma cidade com vinte e cinco mil habitantes, sabendo que o percentual de mulheres é de 55% da população. Qual será a quantidade de mulheres?
10. Crie um algoritmo para calcular o valor de uma aplicação em reais na poupança e calcule seu valor ao final de um período em meses que deverá ser informado pelo usuário do sistema a uma taxa de juros que também deverá ser informada no momento da aplicação.

II

ESTRUTURAS DO ALGORITMO

Nesta unidade aprofundaremos nosso conhecimento sobre a abstração de problemas do mundo real para o mundo computacional, através das técnicas de construção dos algoritmos. Dentre estas técnicas temos as decisões, que permitirá que o computador tome decisões simples e as técnicas de repetição, onde aprenderemos como o computador pode repetir uma sequência de instruções. A partir destas técnicas, esperamos aprofundar o conhecimento lógico, necessário para a construção de algoritmos mais elaborados.

Objetivos:

- Apresentar a forma de como os algoritmos são construídos através das estruturas básicas de sequência, decisão e repetição.

Operações lógicas

UN 02

São as operações que realizamos com o uso dos operadores lógicos. Toda operação com operadores lógicos retorna como resposta um valor lógico. Os operadores lógicos são:

SÍMBOLO	SIGNIFICADO
>	Maior
>=	Maior ou Igual
<	Menor
<=	Menor ou Igual
=	Igual
<>	Diferente

Exemplo:

Digamos que declaramos uma variável "X" para a qual temos um valor armazenado.

EXPRESSÃO	SIGNIFICADO
(X > 3)	X é maior que 3?
(X < 4)	X é menor que 4?
(X = 5)	X é igual a 5?
(X <> 1)	X é diferente de 1?
(X >= 2)	X é maior ou igual a 2?
(X <= 0)	X é menor ou igual a 0?

Bom, dependendo do valor de X, as condições serão mostradas como falso ou verdadeiro.

Veja o exemplo na tabela abaixo:

COMPARAÇÃO	PARA X=10	RESPOSTA
X > 3	10 > 3	Verdadeiro
X < 4	10 < 4	Falso
X = 5	10 = 5	Falso
X <> 1	10 <> 1	Verdadeiro
X <= 5	10 <= 5	Falso
X >= 10	10 >= 10	Verdadeiro

EXERCÍCIO PROPOSTO

1. Responda as seguintes perguntas lógicas:

COMPARAÇÃO	CONSIDERE: A=10, B=3, C=5	RESPOSTA LÓGICA
A > B		
B <= A		
C >= B		
C <> A		
A >= B		
A >= C		
A > C		
B < C		

Desvio condicional simples e composto

UN 02

50

Um desvio condicional é uma forma simples de fazer o computador tomar uma decisão. Um desvio no fluxo de um algoritmo é essencial para que possamos oferecer escolhas, ou seja, caminhos diferentes. Uma escolha poderá conter um ou vários comandos a serem executados de acordo com a opção de escolha.

O comando “Se” é o comando no qual podemos realizar este desvio. Para que o desvio possa acontecer, o comando “Se” precisa realizar uma comparação lógica, fazendo uso das operações lógicas. A sintaxe do comando é:

Se <condição lógica> entao

Comandos

Fimse

Os comandos dentro do laço só serão executados caso a condição lógica testada seja verdade.

Este tipo de desvio condicional é simples, pois só depende de um valor lógico verdadeiro para ser executado. Observe o exemplo abaixo:

Supondo que X é uma variável, e seu valor é 100.

se (X<100) entao

escreva (“o número testado é menor que 100”)

fimse.

Neste exemplo, temos que o valor da variável X é 100 e na comparação lógica do comando “se” perguntamos se o valor é inferior a 100. O resultado lógico da comparação será falso. Como a condição para entrar no “se” deve ser verdadeira, os comandos que estão dentro do laço “se” não serão executados, ou seja, não irá escrever a mensagem.

Já no desvio condicional composto, temos opções para trabalhar os dois possíveis resultados lógicos de uma operação lógica: verdadeiro ou falso.

Para cada opção, podemos ter uma sequência de comandos a serem executados. Porém, em cada comando “se”, apenas um bloco terá os comandos executados de acordo com o resultado do valor lógico. Segue a sintaxe do desvio condicional composto:

Se <condição lógica> entao

Comandos

Comandos executados caso a condição seja verdadeira.

Senao

Comandos.

Comandos executados caso a condição seja falsa.

Fimse

Voltando a analisar nosso exemplo anterior, agora transformando o comando “se” em um desvio composto com a condição “senao”:

Supondo que X é uma variável, e seu valor é 100.

Se ($X < 100$) entao

Escreval (“o número é abaixo de 100”)

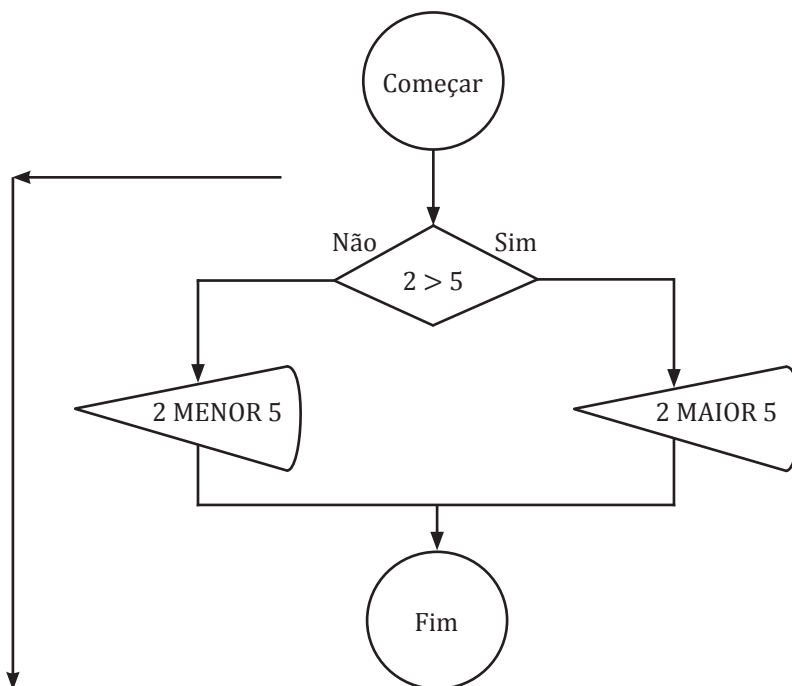
senao

Escreva (“o número é maior ou igual a 100”)

Fimse.

Aqui, atenderemos à condição do “senao” do desvio condicional, sendo a mensagem exibida “o número é maior ou igual a 100”.

Veremos outro exemplo do comando “Se”, desta vez representando o comando com o uso do diagrama de fluxo de dados:



Neste exemplo, perguntamos se o número dois é maior do que o número cinco, como expressão lógica da condição de decisão. Assim, o desvio condicional vai executar os comandos que estão na opção falso do desvio.

Perceba que há um desvio do fluxo normal da execução do algoritmo, podendo seguir pela direita (caso a condição seja verdadeira) ou pela esquerda (caso a condição seja falsa). Perceba também que uma única opção para execução dos comandos dada a resposta para a condição lógica.

▶ EXERCÍCIO RESOLVIDO

1. Construa um algoritmo que receba o nome do aluno e três notas obtidas ao longo de determinado semestre letivo. Calcule a média aritmética do aluno e informe se ele obteve aprovação - com média maior ou igual a sete -, ou reprovação, com média menor ou igual a sete.

Para resolvermos este problema, vamos analisá-lo segundo a técnica trabalhada:

Entradas: De quais entradas de dados precisamos?

Nome do aluno e as três notas obtidas

Processamento: O que deverá ser calculado?

A média baseada nas três notas

Armazenamento: Quais valores devemos armazenar?

NomeAluno, Nota1, Nota2, Nota3, Media

Saída: Quais as saídas de dados?

Nome do aluno e situação: Aprovado ou Reprovado, de acordo com a média obtida pelo aluno.

Em Português Estruturado:

algoritmo "medial"

// Função : Calcular a média de um aluno

// Autor : Luiz Carlos

// Data : 24/01/2014

var

// Armazenamento de dados

NOTA1, NOTA2, NOTA3, MEDIA : real

NOMEALUNO : caracter

inicio

// Entrada de dados

escreval ("digite o nome do aluno: ")

leia (NOMEALUNO)

escreval ("digite a primeira nota: ")

leia (NOTA1)

escreval ("digite a segunda nota: ")

leia (NOTA2)

escreval ("digite a terceira nota: ")

leia (NOTA3)

// Entrada de dados

// Processamento de dados

MEDIA:= (NOTA1 + NOTA2 + NOTA3) / 3

se (MEDIA >= 7) entao

// Saída de dados

escreval ("O aluno: ", NOMEALUNO, " ficou com a média: ", MEDIA, " Aluno Aprovado!")

senao

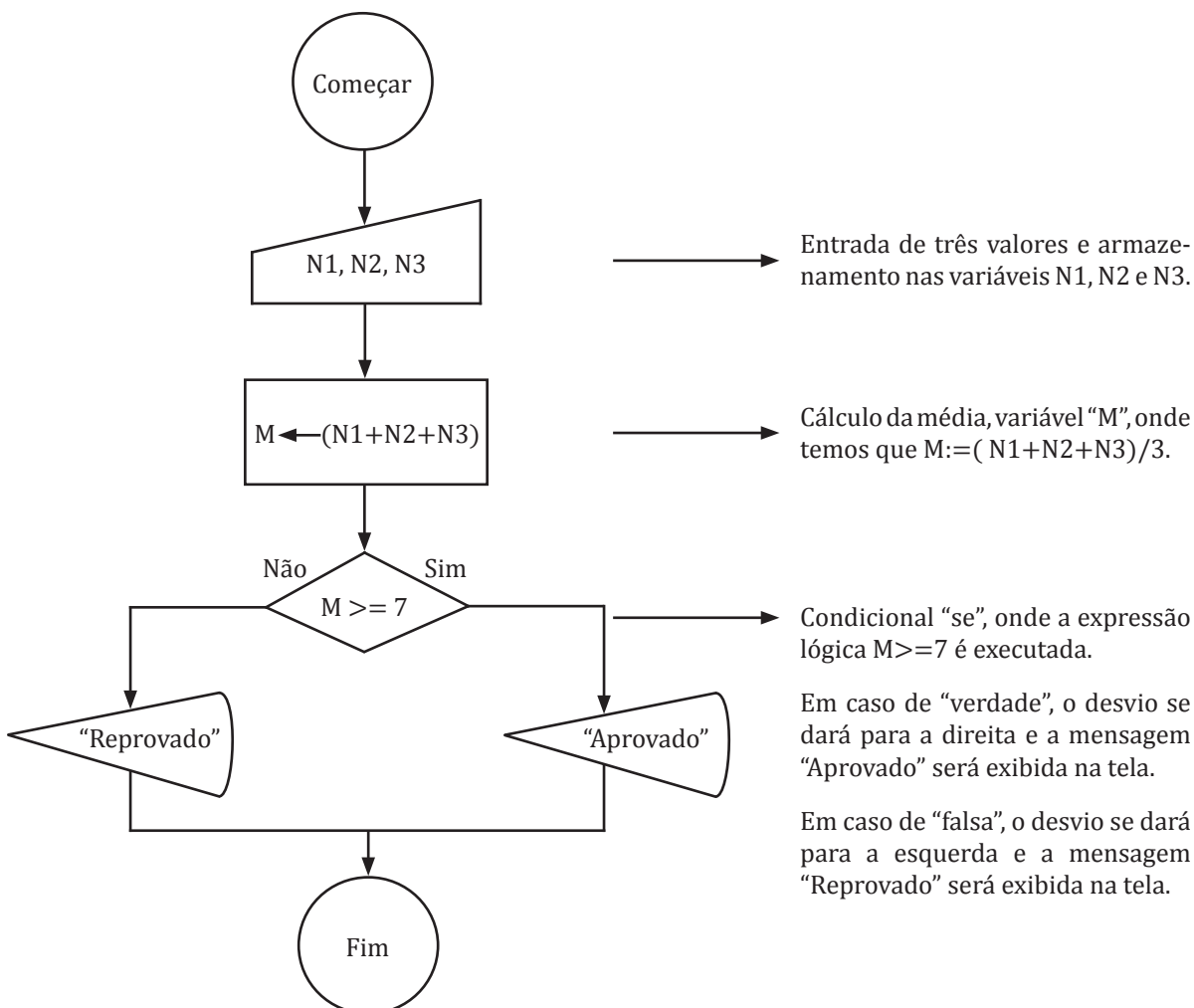
// Saída de dados

escreval (" O Aluno: ", NOMEALUNO, " ficou com a média: ", MEDIA, " Aluno Reprovado!")

fimse

finalgoritmo

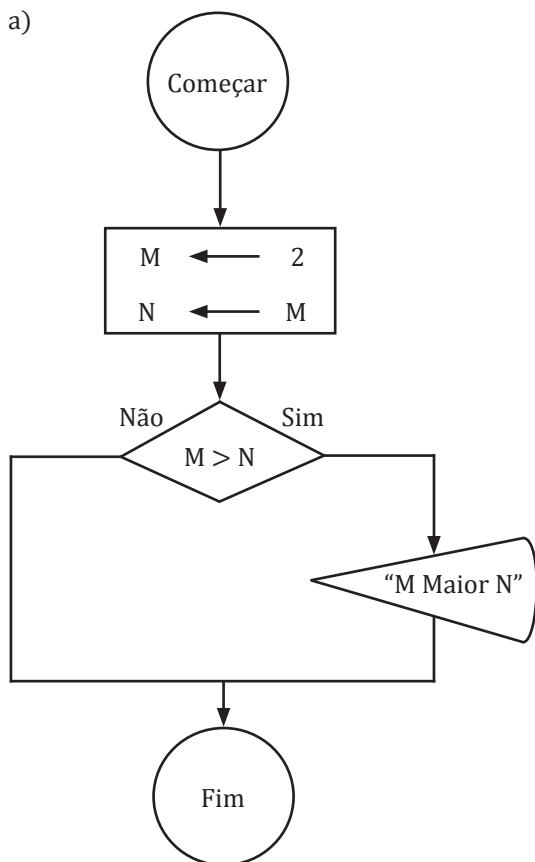
Em diagrama de fluxo de dados:



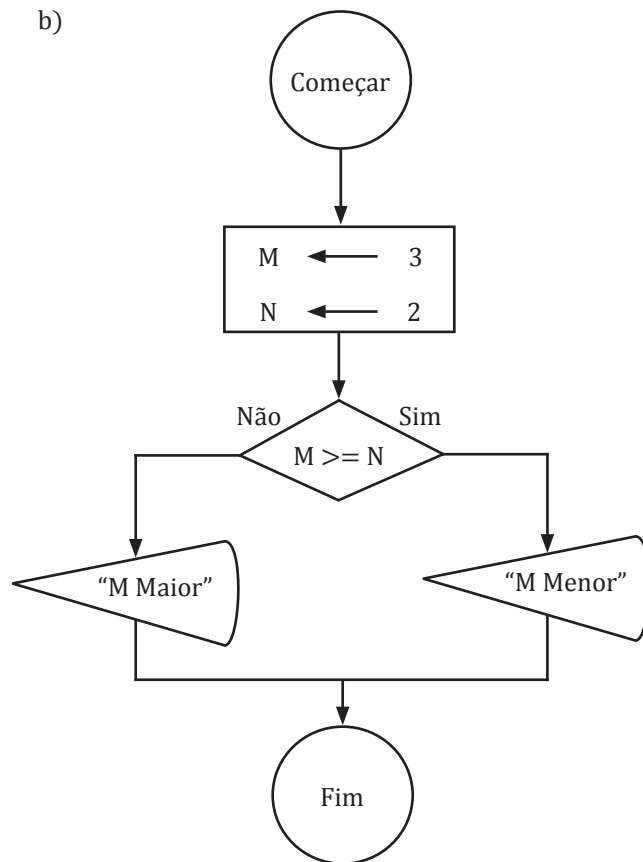
EXERCÍCIO PROPOSTO

1. Interprete os códigos mostrando o desvio do fluxo e o resultado de cada figura:

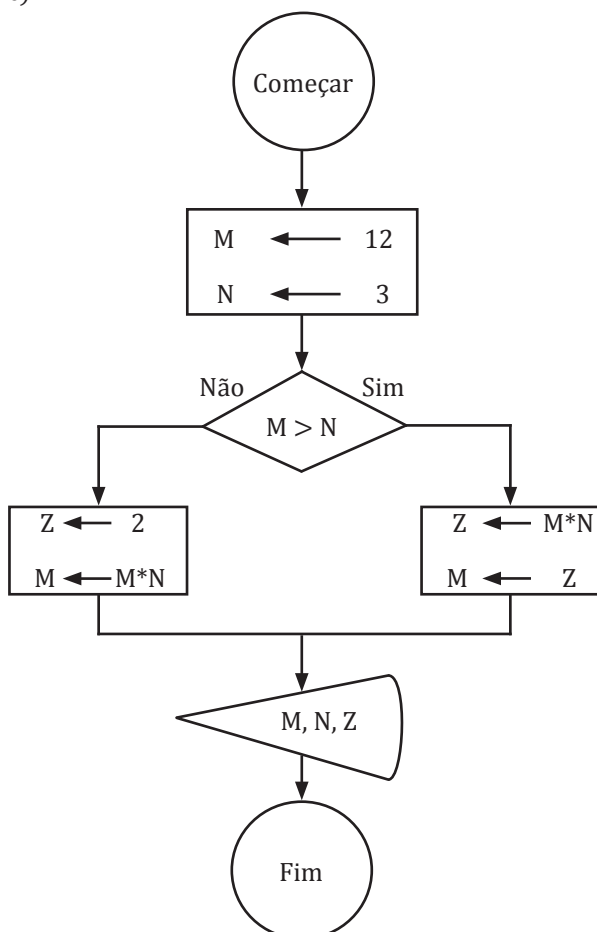
a)



b)



c)



2. Construa os seguintes algoritmos utilizando a notação do Português Estruturado:

a) Construa um algoritmo para receber um número qualquer e mostrá-lo na tela. Caso o número recebido seja inferior a zero, informe ao usuário que o número é negativo.

b) Construa um algoritmo para receber dois valores e informar qual deles é maior.

c) Construa um algoritmo para receber um número, mostrar esse número na tela e dizer se é ímpar ou par.

d) Crie um algoritmo para resolver uma equação do segundo grau no formato $AX^2+BX+C=0$, onde A terá de ser diferente de zero ($A \neq 0$), sabendo que as raízes são:

$$X1 = \frac{-B + \sqrt{B^2 - 4AC}}{2A} \text{ e } X2 = \frac{-B - \sqrt{B^2 - 4AC}}{2A}. \text{ Delta será: } B^2 - 4AC.$$

Desvio condicional aninhado

UN 02

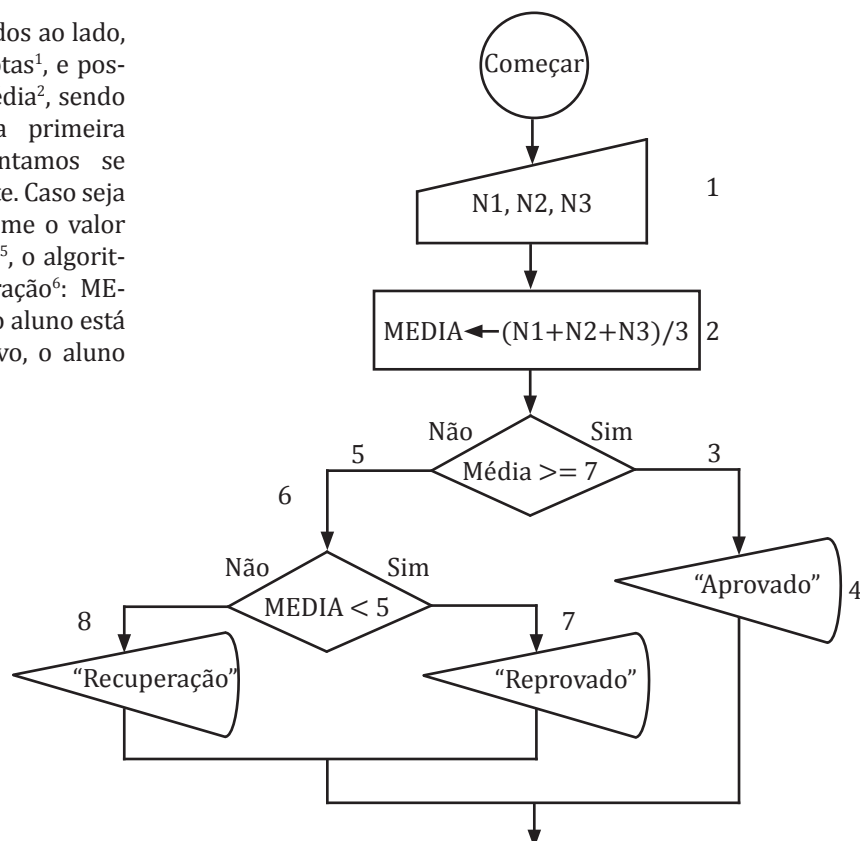
Já que podemos executar diversos comandos dentro de “se”, é de se esperar que também possamos usar outros comandos “se”, formando aninhamento de “se”. Analisaremos o exemplo anterior, onde tínhamos as notas e apenas consideramos que o aluno poderia estar “aprovado” ou “reprovado”. O que devemos fazer para colocar uma nova condição, ou seja, se a média desse aluno estiver entre 5 e 6,9, caso em que o aluno estaria em “recuperação” e faria uma quarta prova?

Para solucionarmos isto, podemos pensar da seguinte maneira:

1. Todos os alunos com média igual ou superior a sete estarão aprovados.
2. Caso o aluno tenha média inferior a sete e superior a cinco, estará em recuperação.
3. Os alunos com média inferior a cinco estarão reprovados.

Para entendermos melhor como essas decisões serão tomadas, vamos colocá-las no fluxograma. Veja abaixo:

No diagrama de fluxo de dados ao lado, temos a entrada das três notas¹, e posteriormente o cálculo da média², sendo $MEDIA = (N1+N2+N3)/3$. Na primeira comparação lógica, perguntamos se MEDIA é maior ou igual a sete. Caso seja verdade³, o algoritmo imprime o valor aprovado⁴. Em caso de falso⁵, o algoritmo fará uma nova comparação⁶: $MEDIA < 5$. Em caso afirmativo, o aluno está reprovado⁷, em caso negativo, o aluno está em recuperação⁸.



Segue o exemplo com o uso do Português Estruturado:

```

algoritmo "media2"

// Função : Calcular a média de um aluno
// Autor : Luiz Carlos
// Data : 24/01/2014
// Seção de Declarações

var

    // Armazenamento de dados
    nota1, nota2, nota3, media : real
    nomealuno : caracter

inicio

    // Entrada de dados
    escreval ("digite o nome do aluno: ")
    leia (nomealuno)
    escreval ("digite a primeira nota: ")
    leia (nota1)
    escreval ("digite a segunda nota: ")
    leia (nota2)
    escreval ("digite a terceira nota: ")
    leia (nota3)

    // Processamento de dados
    media:= (nota1 + nota2 + nota3) / 3

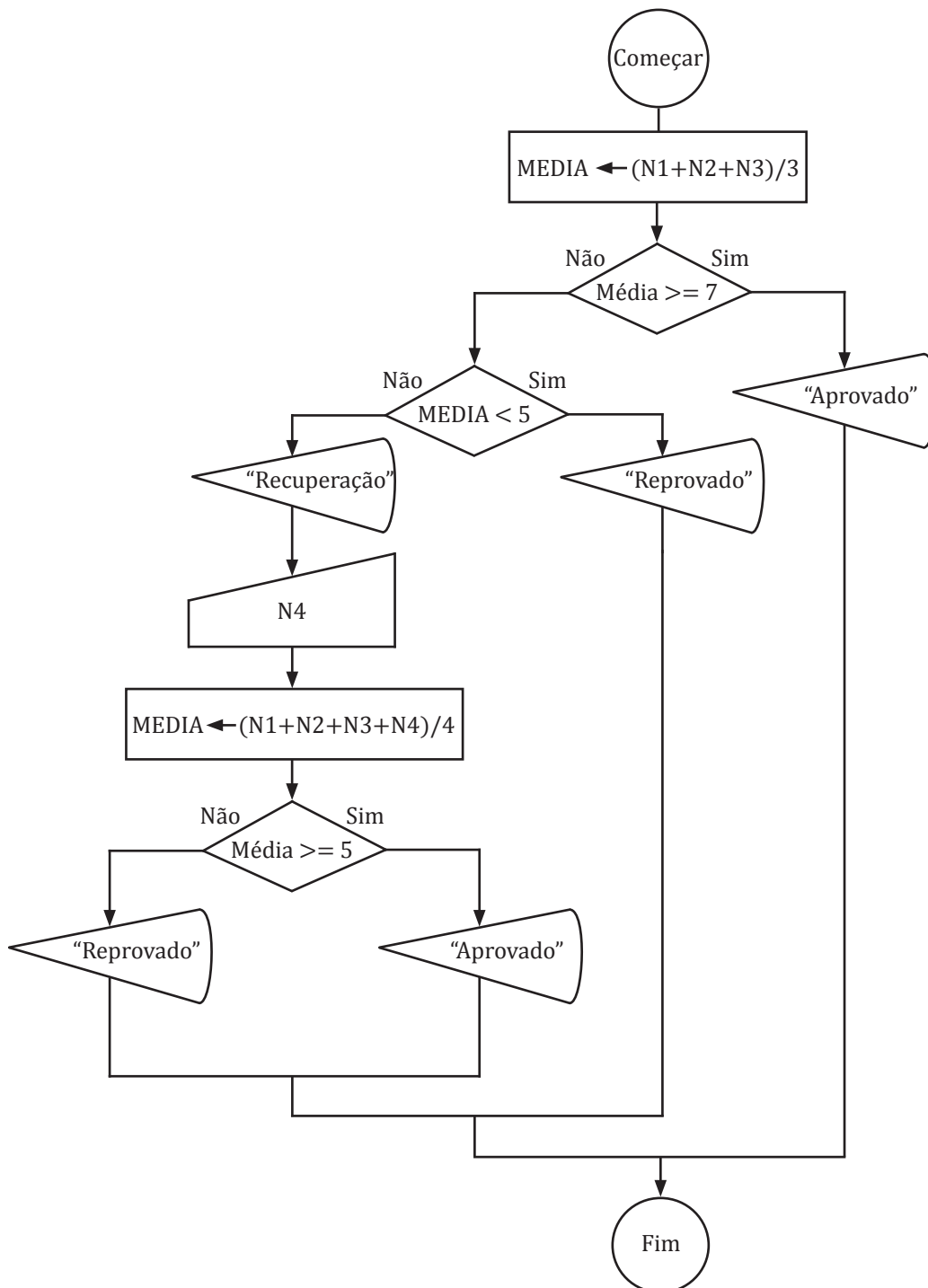
    // Saída de dados
    se (media >= 7) entao
        escreval ("O Aluno :", nomealuno, " ficou com a média: ", media, "Aluno Aprovado!")
    senao
        escreval ("O Aluno :", nomealuno, " ficou com a média: ", media, "Aluno Recuperação!")
    fimse

finalgoritmo
  
```

► EXERCÍCIO RESOLVIDO

Para completarmos o exemplo acima, iremos melhorar o algoritmo de forma que o aluno em recuperação faça uma quarta prova e conheçamos sua nova média. Caso esta média fique acima de cinco após a nota da recuperação, o aluno está aprovado. Em caso contrário, estará reprovado.

Para resolvermos esta questão, acrescentaremos novas decisões a serem tomadas na condição segundo a qual a média esteja superior a cinco e inferior a sete. Dentro desta opção, inserimos uma nova variável para receber a quarta nota, recalculando a média. Se a nova média for superior a cinco, o aluno está aprovado e se for inferior a cinco, o aluno está reprovado.



algoritmo "media3"

// Função : Calcular a média de um aluno

// Autor : Luiz Carlos

// Data : 24/01/2014

// Seção de Declarações

var

// Armazenamento de dados

nota1, nota2, nota3, nota4, media : real

nomealuno : caracter

inicio

// Entrada de dados

escreval ("digite o nome do aluno: ")

leia (nomealuno)

escreval ("digite a primeira nota: ")

leia (nota1)

escreval ("digite a segunda nota: ")

leia (nota2)

escreval ("digite a terceira nota: ")

leia (nota3)

// Processamento de dados

media:= (nota1 + nota2 + nota3) / 3

// Saída de dados

se (media >= 7) entao

escreval ("O Aluno :", nomealuno, " ficou com a média: ", media, "Aluno Aprovado!")

senao

se (media < 5) entao

escreval ("O Aluno :", nomealuno, " ficou com a média: ", media, "Aluno Reprovado!")

senao

escreval ("digite a quarta nota: ")

leia (nota4)

media:= (nota1 + nota2 + nota3 + nota4) / 4

se (media >= 5) entao

escreval ("O aluno: ", nomealuno, " ficou com a média: ", media, "Aluno Aprovado!")

senao

escreval ("O aluno: ", nomealuno, " ficou com a média: ", media, " Aluno Reprovado!")

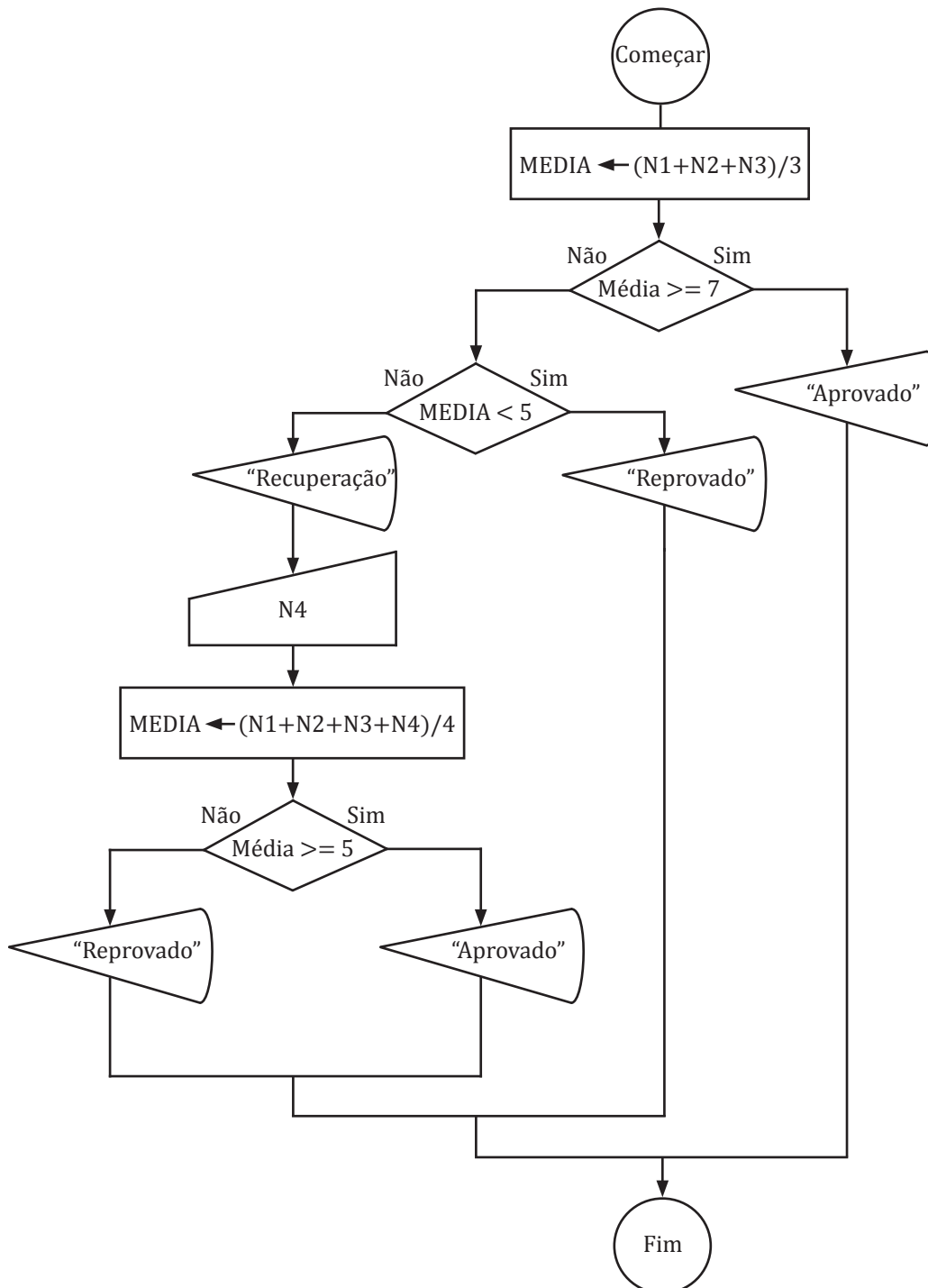
fimse

fimse

fimse

finalgoritmo

Segue a resolução com o diagrama de fluxo de dados:



Comando de decisão Escolha

UN 02

Este comando de decisão é bastante utilizado quando temos um determinado valor e queremos escolher uma ou várias opções para a execução de comandos, cabendo a nós decidir o que será executado em cada valor da variável analisada.

A sintaxe do comando é:

escolha <variável>

Caso (valor1)

<comandos>

Caso (valor2)

<comandos>

Caso (valorN)

<comandos>

Outrocaso

<comandos>

fimescolha

A variável de controle do desvio pode assumir diversos valores. Caso o valor esteja previsto em qualquer das condições dentro do bloco “escolha”, os comandos que pertencem ao bloco serão executados. Quando nenhum caso for previsto, pode ser usado o bloco “outrocaso”, que significa que qualquer valor imprevisto pode executar os comandos abaixo deste bloco. Apenas um bloco de comandos poderá ser executado por vez.

Como exemplo, vejamos um algoritmo mais elaborado para uma calculadora, com as opções de operações:

algoritmo “escolha_calculadora”

// Função : Executa as 4 operações básicas

// Autor : Luiz Carlos

// Data : 01/02/2014

var

NUM1, NUM2, RESULT : real

OP : caracter

inicio

// Entrada de dados

escreval (“+++++++ Programa Calculadora ++++++ ”)

escreval (“Entre com o primeiro número: ”)

leia (NUM1)

escreval (“Entre com o segundo número: ”)

leia (NUM2)

escreval (“Entre com a Operação: ”)

escreval (“Tecle + para SOMAR: ”)

escreval (“Tecle - para SUBTRAÇÃO: ”)

escreval (“Tecle * para MULTIPLICAÇÃO: ”)

Neste exemplo, trabalhamos as opções básicas de adição, subtração, multiplicação e divisão para o cálculo a ser realizado a partir de duas variáveis (NUM1, NUM2). Para representar as operações que faremos com as duas variáveis, criamos as condições de escolhas em que o usuário deverá entrar com os símbolos que representam tais operações (OP), analisadas no bloco de desvio “escolha”, que realizará as operações de acordo com cada símbolo. Se o usuário digitar algo diferente do previsto, o sistema não executará nenhuma operação, apresentando a frase “Opção não válida”.

```

escreval ("Tecle / para DIVISÃO: ")
leia (OP)

// Usando o comando "escolha" para as operações
escolha OP
caso "+"
    escreval ("Adição")
    RESULT:= NUM1 + NUM2
    escreval ("A soma do Número 1: ", NUM1, " com o número 2: ", NUM2, " é: ", RESULT)
caso "-"
    escreval ("Subtração")
    RESULT:= NUM1 - NUM2
    escreval ("A subtração do Número 1: ", NUM1, " com o número 2: ", NUM2, " é: ", RESULT)
caso "*"
    escreval ("Multiplicação")
    RESULT:= NUM1 * NUM2
    escreval ("A multiplicação do Número 1: ", NUM1, " com o número 2: ", NUM2, " é: ", RESULT)
caso "/"
    escreval ("Divisão")
    RESULT:= NUM1 / NUM2
    escreval ("A divisão do Número 1: ", NUM1, " com o número 2: ", NUM2, " é: ", RESULT)
outroCaso
    escreva ("Opção não válida!")
fimEscolha
finalgoritmo

```

Conectores lógicos

UN 02

Os conectores lógicos têm a função de ligar duas ou mais comparações lógicas, formando uma expressão lógica. Temos os dois conectores lógicos "E" e "OU", além do conector "Não". Veremos cada um deles abaixo.

Conector lógico "E"

O conector lógico "E" serve para ligar duas ou mais comparações lógicas. Vejamos o exemplo:

Supomos que temos uma variável X que possui valor igual a "8".

Comparação 1: Conector lógico E: Comparação 2:

$X > 6$ e $X < 9$

A comparação 1 será verdadeira, pois oito é maior do que seis. A comparação 2 também será verdadeira, pois oito é menor do que nove. Assim, toda a expressão tem resultado lógico final verdadeiro.

Vejam os outro exemplo:

Comparação 1: Conector lógico E: Comparação 2:

$X > 5$ e $X < 7$

A comparação 1 será verdadeira, pois oito é maior do que cinco. A comparação 2 será falsa, pois oito é maior do que sete (não menor). Desta forma, a expressão como um todo será falsa.

Existe uma tabela para o conector lógico “E”, onde podemos ter duas comparações lógicas. Com base nos resultados, um valor final para uma expressão lógica poderá ter:

COMPARAÇÃO 1	COMPARAÇÃO 2	RESULTADO
Verdadeiro	Verdadeiro	Verdadeiro
Verdadeiro	Falso	Falso
Falso	Verdadeiro	Falso
Falso	Falso	Falso

Assim, somente se todas as comparações forem verdadeiras, o resultado final de uma expressão lógica será verdadeiro.

Vejam os outro exemplo. Neste caso, temos duas comparações lógicas ligadas pelo conector “e” comparando o valor armazenado em uma variável denominada média, situação já mostrada no exemplo do exercício resolvido anteriormente, onde calculamos uma média de três notas. Para que um aluno esteja na condição de recuperação, sua média deverá estar no intervalo de valores entre cinco e sete.

se (Media>=5) e (Media<7) entao

Escreva(“aluno em recuperação”)

fimse

Conector lógico “ou”

O conector lógico “ou” também é usado para a ligação de duas ou mais comparações lógicas. Vejam os exemplo:

Supomos ter uma variável X que possui valor igual a “8”.

Comparação1: Conector lógico OU: Comparação 2:

$X > 6$ ou $X < 9$

A comparação 1 será verdadeira, pois oito é maior do que seis. A comparação 2 também será verdadeira, pois oito é menor do que nove. Portanto, toda a expressão é verdadeira.

Comparação 1: Conector lógico E: Comparação 2:

$X > 5$ ou $X < 7$

No comando “se”, temos duas comparações lógicas a avaliar:

1. A média é maior ou igual a cinco?
2. A média é menor do que sete?

O resultado será verdadeiro se ambas as comparações forem verdadeiras, o que significa que o aluno estará em recuperação.

A comparação 1 será verdadeira, pois oito é maior do que cinco. A comparação 2 será Falsa, pois oito não é maior do que sete. Mesmo assim, a expressão lógica é verdadeira, pois uma ou a outra comparação é verdadeira.

Para o conector lógico “ou”, também temos uma tabela para obter os resultados lógicos de duas comparações lógicas:

COMPARAÇÃO 1	COMPARAÇÃO 2	RESULTADO
Verdadeiro	Verdadeiro	Verdadeiro
Verdadeiro	Falso	Falso
Falso	Verdadeiro	Falso
Falso	Falso	Falso

Assim, apenas se todos os resultados das comparações lógicas forem falsos o valor da expressão lógica será falso.

Vejam os exemplos. Supomos que um algoritmo deve analisar duas variáveis, uma representando se o estado de luz de um carro está ligado ou desligado e outra mostrando se a porta do veículo está aberta ou fechada. Se qualquer uma das situações acima for verdade, o algoritmo deve emitir um alerta ao motorista. Segue parte do código deste algoritmo:

se (luzligada=”sim”) ou (portaaberta=”sim”) entao

Escreva(“Atenção verifique luzes e portas”)

fimse

No comando “se”,
temos duas comparações
lógicas a avaliar:
3. Luz do carro está ligada?
4. A porta do carro está aberta?
O resultado será verdadeiro se
qualquer uma das comparações
for verdadeira, emitindo um
alerta ao motorista.

63

Conector lógico “nao”

O conector lógico “nao” serve para alterar o valor lógico de uma condição lógica. Exemplo: Supomos uma variável X de valor X=10.

(X=10) ? -> Verdadeiro

Nao (X=10) ? -> Falso

Assim, usamos o conector “nao” para inverter o resultado de uma comparação uma lógica.

EXERCÍCIO PROPOSTO

1. Analise as expressões lógicas abaixo e responda quais serão os valores das variáveis X, Y e Z ao final de cada bloco de execução. Inicialmente considere X:= 2, Y:=4 e Z:= 6.

a)

```

Se (X>6) e (X> Z) entao
  X:=5
  Y:=Z
Senao
  X:=3
  Z:=Y
Fimse
Escreva(X, Y, Z)

```

b)

```

Se (X>6) ou (X> Z) entao
  X:=5
  Y:=Z
Senao
  X:=3
  Z:=Y
Fimse
Escreva(X, Y, Z)

```

c)

```

Se (X>Y) ou nao(X> Z) entao
  X:=4
  Y:=5
Senao
  X:=Z
  Z:=X
Fimse
Escreva(X, Y, Z)

```

d)

```

Se (X>6) ou (X> Z) entao
  Se (Y>=2) entao
    Y:=8
  Senao
    Y:=12
  fimse
Senao
  X:=3
  Z:=Y
Fimse
Escreva(X, Y, Z)

```

e)

```

Se (X<=6) ou (X> Z) entao
  Se (Y>=2) entao
    Y:=40
  Senao
    Y:=15
  fimse
Senao
  Se (Z>=X) entao
    Y:=Z
  fimse
  Z:=Y
Fimse
Escreva(X, Y, Z)

```

f)

```

Se nao(X<=6) ou (X> Z) entao
  Se nao(Y>=2) entao
    Y:=20
  Senao
    Y:=1
  fimse
Senao
  Se (Z>=X) entao
    Y:=Z
  fimse
  Z:=Y
Fimse
Escreva(X, Y, Z)

```

2. Responda os algoritmos abaixo por meio de diagrama de fluxo de dados e Português Estruturado, teste-os usando os *softwares* adequados.

a) Construa um algoritmo para receber três valores quaisquer e ordenar os valores do maior para o menor, mostrando a ordenação na tela do computador.

b) Construa um algoritmo para descobrir se um número é par ou ímpar. Dica: Use o comando MOD como resto de uma divisão de um número qualquer por dois.

c) Construa um algoritmo que receba uma faixa de valores, com um valor mínimo e um máximo, e receba um número qualquer. O algoritmo deve informar se o número recebido está dentro ou fora da faixa de valores.

d) Construa um algoritmo para reajustar o salário de uma pessoa em 10% se ele receber até R\$ 800,00, em 8% se ele receber até R\$ 1.200,00 e em 5% se ele receber acima de R\$ 1.200,00.

e) Construa um algoritmo para medir o Índice de Massa Corporal de uma pessoa (IMC) sabendo que o cálculo do $IMC = PESO / (ALTURA)^2$ e informe o resultado de acordo com o valor do IMC seguindo a tabela abaixo:

VALOR DO IMC	MENSAGEM
Abaixo de 17	Muito abaixo do peso
Entre 17 e 18,5	Abaixo do peso
Entre 18,6 e 24,99	Peso Normal
Entre 25 e 29,99	Acima do peso
Entre 30 e 34,99	Obesidade I
Entre 35 e 39,99	Obesidade II (Crítica)
Acima de 40	Obesidade III (Mórbida)

Laços de repetição

UN 02

Laços de repetição são as estruturas que executam a repetição de um ou mais comandos. Vejamos a seguinte situação: digamos que precisamos de que o computador apresente na tela uma contagem regressiva de dez números (10, 9, 8, 7...1). Como faríamos? Até onde sabemos, teríamos de inserir um a um os números que deverão ser mostrados na tela.

É para este tipo de situação que existem os laços de repetição, que diminuem a repetição de escrita de comandos, deixando o algoritmo com menor tamanho e maior simplicidade.

Assim, um laço de repetição permite que um ou vários comandos dentro da estrutura da repetição sejam repetidos pela quantidade de vezes desejada ou por uma quantidade indeterminada de vezes.

São três os laços que trabalharemos neste material: Comando Enquanto, Repita e Para.

Enquanto

O laço “enquanto” é um comando de repetição que faz uso de uma expressão lógica para entrada no laço e execução das repetições previstas dentro do laço. Sua sintaxe é a seguinte:

Enquanto (condição-Logica) faca

<comando1>

<comando2>

<comandoN>

Fimenquanto

O laço “enquanto” precisa de uma variável de controle a fim de controlar a entrada no laço e a permanência dentro deste, realizando os comandos a serem repetidos. Assim, é pertinente usar uma variável exclusiva para o controle deste laço.

Vejamos o exemplo citado no início deste conteúdo, como resolveríamos usando o laço “enquanto” usando a representação do português estruturado com o *VisuAlg*:

[algoritmo](#) “Decremento”

// Função : Uso do Repita com contagem regressiva

// Autor : Luiz Carlos

// Data : 28/01/2014

Não há uma condição lógica para que o laço “repita” seja ativado, ou seja, de toda forma, ao menos uma vez, os comandos dentro do laço serão executados. A condição lógica de teste final deverá ser falsa para que o laço continue se repetindo. Quando a condição for verdadeira, a sequência de repetição para.

65

Este algoritmo apresenta a impressão de um valor iniciando em 10 até o valor 0.

Usamos o laço “repita” para repetir os comandos de impressão e cálculo do decremento do valor inicial de X, que a cada execução terá seu valor diminuído -1. Quando o valor for 0, o valor da variável que controla o laço é alterado para falso, o que causa o fim das repetições.

// Seção de Declarações

var

RESP : real // Variável para controle do laço

X : inteiro // Variável para atribuir valores

inicio

X:= 10 // Iniciando a variável X com o valor 10

RESP:= verdadeiro // Iniciando a variável RESP para controle do laço

repita // Início do laço sem condição de entrada

escreval (“O valor de X é: “, X) // Ecreve o valor de x

se X < 1 entao // Condição para mudar expressão lógica e sair do laço

RESP:= falso // Alterando o valor lógico da variável RESP

fimse

X:= X - 1 // Decrementando o valor de X em -1 a cada vez que o laço se repetir

ate RESP=falso // Condição de saída do laço

fimalgoritmo

Repita

O laço “repita” é um comando de repetição segundo o qual os comandos dentro do laço serão executados independentemente da condição de permanência no laço, pois o teste da condição de permanência no laço só é realizado ao final.

Não há uma condição lógica para que o laço “repita” seja ativado, ou seja, de toda forma, ao menos uma vez, os comandos dentro do laço serão executados. A condição lógica de teste final deverá ser falsa para que o laço continue se repetindo. Quando a condição for verdadeira, a sequência de repetição para.

Repita

<comando1>

<comando2>

<comandoN>

Ate(Condição-Logica)

O laço “repita” também precisa de uma variável de controle a fim de controlar a saída do laço. A permanência dentro do laço exige que a condição de teste seja falsa. Também devemos usar uma variável exclusiva para o controle deste laço.

Vejamos o exemplo citado no início deste conteúdo, que mostra como resolveríamos este algoritmo usando o laço repita por meio da representação do Português Estruturado com o *VisuAlg*:

algoritmo “Decremento”

// Função : Uso do Repita com contagem regressiva

// Autor : Luiz Carlos

// Data : 28/01/2014

// Seção de Declarações

var

RESP : logico // Variável para controle do laço

```

X: inteiro           // Variável para atribuir valores
inicio
X:= 10               // Iniciando a variável X com o valor 10
RESP:= verdadeiro    // Iniciando a variável RESP para controle do laço
repita               // Início do laço sem condição de entrada
    escreval ("O valor de X é: ", X)    // Ecreve o valor de x
    se X < 1 entao    // Condição para mudar expressão lógica e sair do laço
        RESP:= falso // Alterando o valor lógico da variável RESP
    fimse
    X:= X - 1        // Decrementando o valor de X em -1 a cada vez que o laço se repetir
ate RESP=falso      // Condição de saída do laço

```

Este algoritmo apresenta a impressão de um valor iniciando em 10 até o valor 0. Usamos o laço "repita" para repetir os comandos de impressão e cálculo do decremento do valor inicial de X, que a cada execução terá seu valor diminuído -1. Quando o valor for 0, o valor da variável que controla o laço é alterado para falso, o que causa o fim das repetições.

fimalgoritmo

Para

O comando "para" é usado quando sabemos exatamente quantas vezes queremos repetir os comandos dentro de um laço. Sua sintaxe é:

Para (variável) de <valor-inicial> ate <valor-final> faca

<comando1>

<comando2>

<comandoN>

Fimpara

Devemos usar uma variável para este laço e definir um valor inicial e final. A cada vez que o laço é executado, o comando "para" se encarrega de incrementar esta variável, comparando se ela atingiu o valor final desejado, ou seja, o número de vezes que queremos repetir os comandos internos dentro do laço.

Assim, para o comando de repetição "para" devemos usar uma variável exclusiva para controle do laço, que terá um valor definido como início e um valor final a ser atingido, sendo o laço executado na quantidade de vezes que estabelecemos da diferença entre seus valores inicial e final.

algoritmo "Decremento"

// Função : Uso do Para com contagem regressiva

// Autor : Luiz Carlos

// Data : 28/01/2014

// Seção de Declarações

var

RESP: real // Variável para controle do laço

X: inteiro // Variável para atribuir valores

inicio

X:= 10 // Iniciando a variável X com o valor 10

para RESP de 1 ate 11 faca // Início do laço com valor inicial e final do controle

Este algoritmo apresenta a impressão de um valor iniciando em 10 até o valor 0. A variável RESP será inicializada com o valor 1 quando a primeira repetição for executada, repetindo por 10 vezes os comandos de saída de dados com o valor de X, inicialmente 10 e decrescendo a cada vez que o laço for executado.

```
escreval ("O valor de X é: ", X) // Ecreve o valor de x
```

```
X := X - 1 // Decrementando o valor de X em - 1 a cada vez que o laço repetir
```

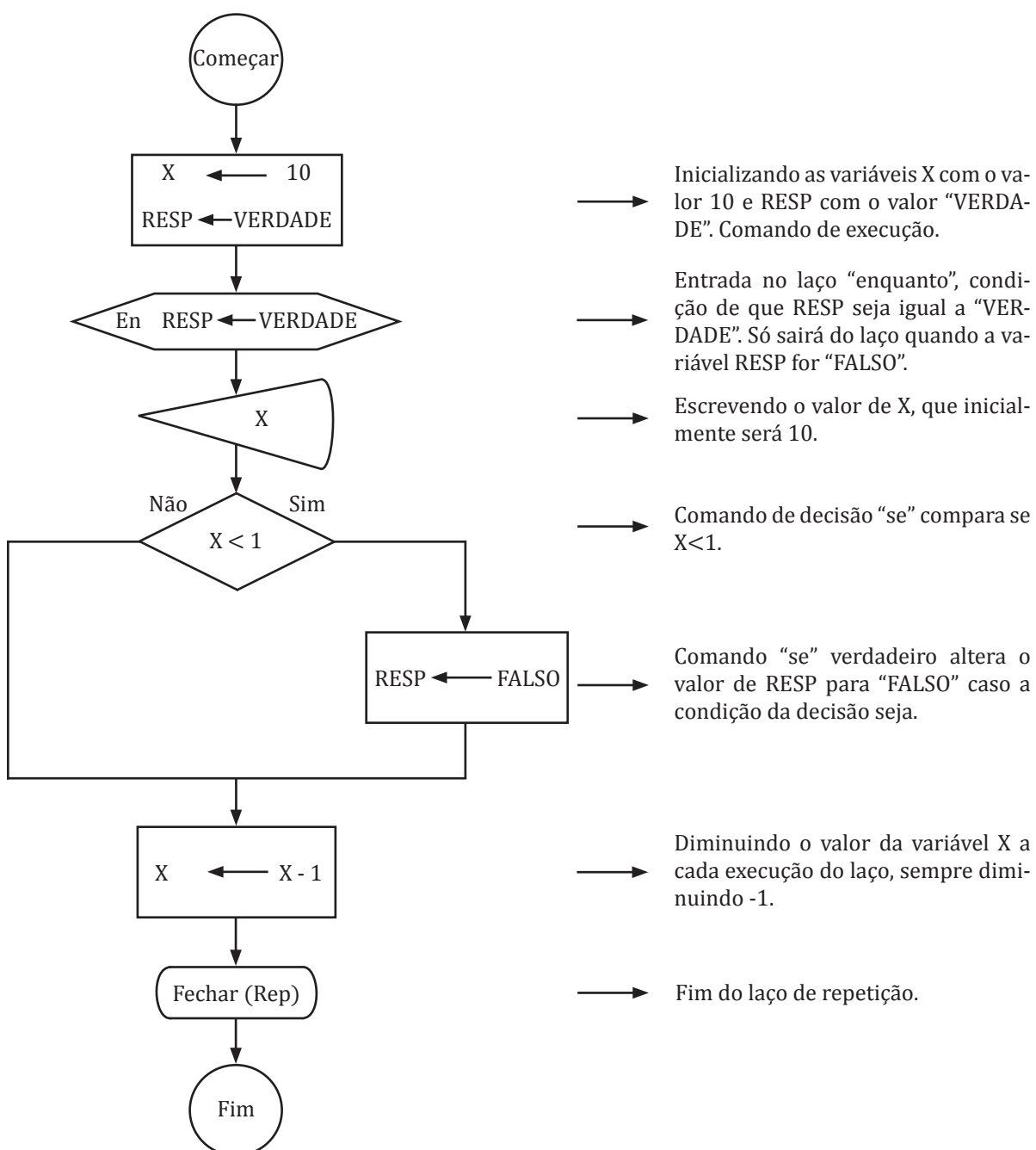
```
fimse // Fim do Laço
```

```
fimalgoritmo
```

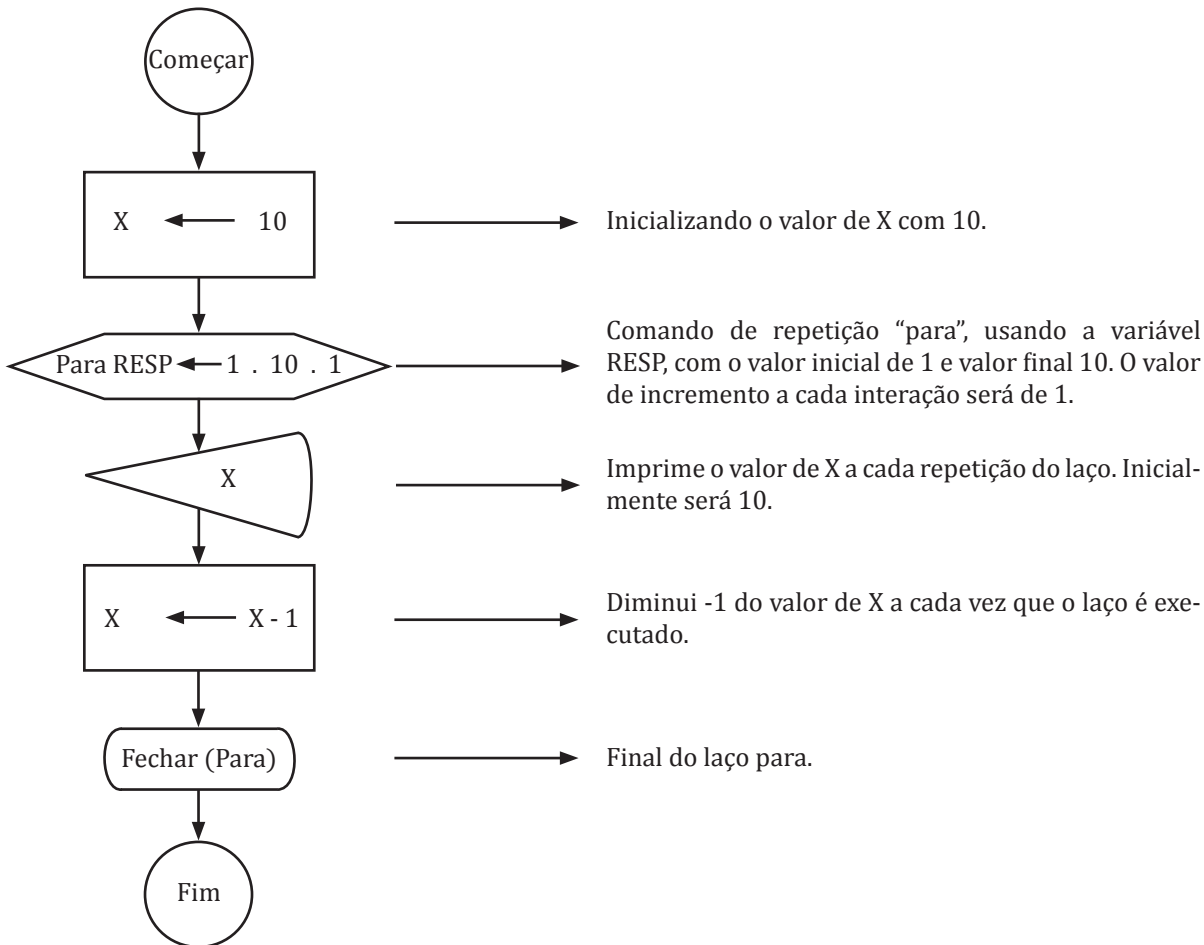
Comandos de repetição usando o Diagrama de Fluxo de Dados

Na representação dos laços pelo diagrama de fluxo de dados usando o FreeDFD, temos apenas dois laços: “enquanto” e “Para”. Assim, mostramos os dois laços do problema trabalhado neste tópico de repetição, comentando-os.

Usando o Laço “enquanto”



Usando o Laço “para”



SAIBA MAIS

Programação Estruturada - A programação estruturada é baseada na ideia de que qualquer programa poderá ser escrito com base em três estruturas básicas (WIKI, 2013):

Estruturas de Sequência: Os comandos devem ser alocados de forma sequencial de acordo com a lógica obedecida.

Estrutura de Decisão: Estruturas que permitem ao computador tomar decisões a partir de comparações lógicas das variáveis.

Estrutura de Repetição: Estrutura que permite que os computadores repitam operações de acordo com as condições das repetições.

A programação estruturada evoluiu para a programação em blocos, onde os blocos contêm uma sequência de comandos ordenados para executar determinada função, sendo que os blocos podem ser usados em diversas partes do programa, sem necessidade de reescrever os comandos para a execução de determinada função já solucionada pelo bloco.

Assim, a programação estruturada evoluiu continuamente até chegar à Programação Orientada a Objetos, a qual, embora seja uma forma diferente de programar, utiliza-se da programação estruturada para a construção das classes dos objetos. Você verá a Programação Orientada a Objetos em outras disciplinas neste curso.

01. Quando usar os comandos de condição “Se” e “Escolha”?

Dependerá bastante de qual problema estamos tentando resolver. É sensato usar o comando “escolha” quando queremos limitar as opções de escolhas, como, por exemplo, escolhas de um menu ou escolhas de controles de funções a serem executadas. O comando “se” se torna mais poderoso quando usado com os conectores lógicos, e em um único comando “se” podemos atender a milhares de situações possíveis, sendo mais indicado quando não sabemos exatamente o valor de uma variável para decisão, mas conhecemos uma faixa de valores possíveis a se trabalhar na escolha.



Banco de imagens/NEAD

02. Quando usar o laço “Enquanto” e “Repita”?

Devemos usar o laço “enquanto” quando queremos repetir os comandos do laço somente se a condição que queremos atender for verdadeira. Já o laço “repita” deve ser usado quando queremos repetir, ao menos uma vez, os comandos que poderão estar dentro do laço.

03. É possível criar um laço infinito?

Sim, é possível, desde que realmente queiramos que isso ocorra ou quando, por engano, não trabalhamos a entrada e saída de um laço de repetição. Porém, devemos sempre ter uma condição de saída de todos os laços.

▶ EXERCÍCIO RESOLVIDO

Crie um algoritmo que receba um número qualquer do usuário e multiplique este número pelo valor de 1 até o valor dele mesmo (valor final é o número digitado pelo usuário), mostrando na tela esta multiplicação e o resultado da soma das multiplicações, ou seja, se o usuário entrar com o número 5, o sistema deverá multiplicar e o número 5 ficará: ($5*1 = 5$; $5*2 = 10$; $5*3 = 15$; $5*4 = 20$; $5*5 = 25$) apresentando ao final a soma da multiplicação, que no caso seria o número 75.

Usando o *VisuAlg*

algoritmo “multiplicanumeros”

// Função : Receba um número e multiplica pelos antecessores

// Autor : Luiz Carlos

// Data : 29/01/2014

// Seção de Declarações

var

NUM, X, RESULT, CALC : real

inicio

// entrada de um número qualquer

escreva (“digite um número qualquer”)

leia (NUM)

// Cálculo da multiplicação do número digitado pelos antecessores

para X de 1 ate NUM faca **1**

CALC:= NUM * X **2**

escreva (“O Numero “, NUM, “multiplicado por “, X, “resultando: “, CALC)

3

Usamos quatro variáveis:
NUM: Recebe um valor digitado.
X: Controle do laço de repetição.
RESULT: Soma dos valores digitados – acumulador.
CALC: Multiplicação do número pelo antecessor.

RESULT:= RESULT + CALC 4

fimpara

escreva ("O Numero ", NUM, "multiplicado por ", X, " resultando: ", CALC)

fimalgoritmo

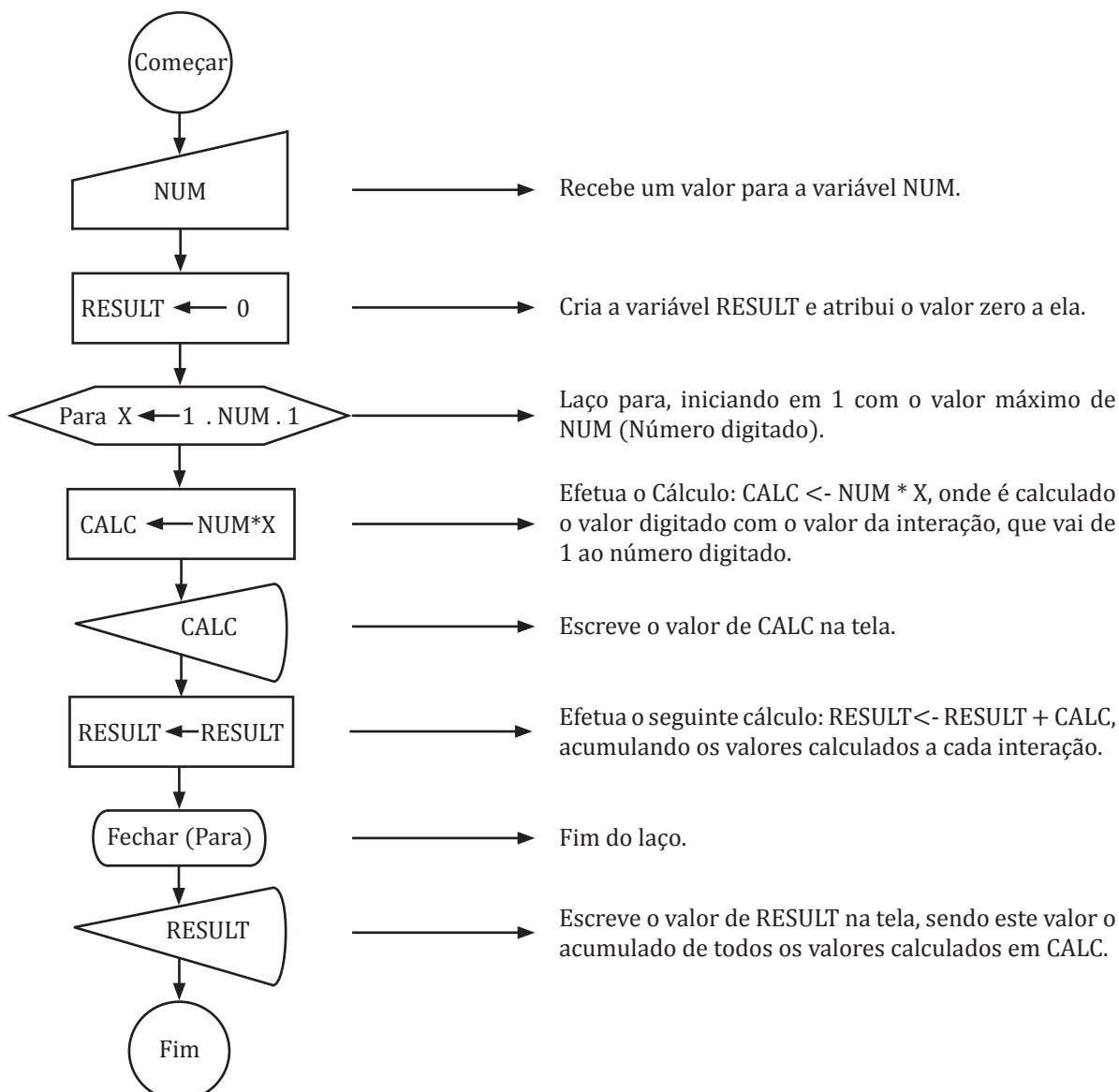
1. Como sabemos a quantidade de repetições do laço, usamos o comando "para". A quantidade máxima de repetições será o próprio número digitado (NUM). A cada interação do laço, calculamos o número digitado (NUM) pelo valor da interação (X). "X" vai iniciar no número 1 até o valor da variável NUM de forma sequencial, ou seja: 1, 2, 3...NUM.

2. O primeiro comando do laço é uma operação com a variável CALC que resultará no valor da multiplicação do número digitado (NUM) pelo valor da interação X. Como sabemos que o valor máximo de X será o próprio número digitado (NUM), asseguramos que o número digitado (NUM) será multiplicado do valor 1 até ele mesmo de forma sequencial a cada interação do laço.

3. Escrevo o resultado da multiplicação do número digitado (NUM) pelo valor da interação (X), mostrando a variável CALC que conterá o valor da operação.

4. A última operação do laço é a soma dos valores encontrados a cada operação de multiplicação (CALC), sendo somada ao valor acumulado a variável RESULT. Esta técnica de acumulação de valores é bastante utilizada.

Usando o *FreeDFD*, teremos a seguinte representação:



▶ EXERCÍCIO PROPOSTO

1. Interprete os trechos dos algoritmos:

a) algoritmo “Laços”

var

X, Y : Inteiro

inicio

Y:= 3

Para X de 1 ate 5 faca

Y:= Y + X

fimpara

escreva (Y)

fimalgoritmo

b) algoritmo “Laços”

var

X, Y : Inteiro

inicio

Y:= 3

Enquanto (Y > 1) faca

X:= X + Y

Y:= Y - 1

fimenquanto

escreva (X)

fimalgoritmo

c) algoritmo “Laços”

var

X, Y : Inteiro

inicio

Y:= 3

Repita

X:= X + Y

Y:= Y - 1

ate (Y <= 3)

escreva (X)

fimalgoritmo

d) algoritmo “Laços”

var

X, Y, Z : Inteiro

inicio

para X da 1 ate 3 faca

para Y de 1 ate 3 faca

Z := Z + 1

fimpara

fimpara (Z)

fimalgoritmo

2. Crie os seguintes algoritmos:

a) Construa um algoritmo que receba um número. Caso este seja par, escreva os dez próximos números pares do número digitado. Caso o valor digitado seja ímpar, escreva os dez números ímpares antecessores ao número digitado.

b) Construa um programa que receba um número qualquer e escreva os dez números subsequentes a este.

c) Construa um programa que receba um número qualquer e escreva os dez números subsequentes e os dez números antecedentes a ele.

d) Construa um programa que some os valores digitados pelo usuário, sejam quantos ele desejar. Ao final, apresente a quantidade de valores digitados e a soma de todos os valores digitados pelo usuário.

e) Construa um algoritmo para escrever do número 500 ao número 1.500 na tela do computador usando o comando de repetição “para”.

f) Construa um algoritmo para apresentar a tabela de multiplicação dos números um ao dez.

g) Construa um algoritmo para calcular o fatorial de um número qualquer, lembrando que o fatorial é a multiplicação do número escolhido por todos os seus antecessores, excluindo o zero. Assim, o fatorial de cinco seria: $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$.

III

ESTRUTURAS DE ARMAZENAMENTOS E ESTRUTURA DE BLOCOS

Nesta unidade, apresentamos as estruturas de armazenamento de dados, onde podemos criar estruturas que irão nos permitir manipular grandes quantidades de variáveis em um algoritmo. Também trabalharemos as estruturas de blocos e suas aplicações práticas, onde tais estruturas simplificam o processo de criação de grandes algoritmos.

Objetivos:

- Apresentar as estruturas complexas de armazenamento de dados e as estruturas de blocos de algoritmos.

Matrizes

UN 03

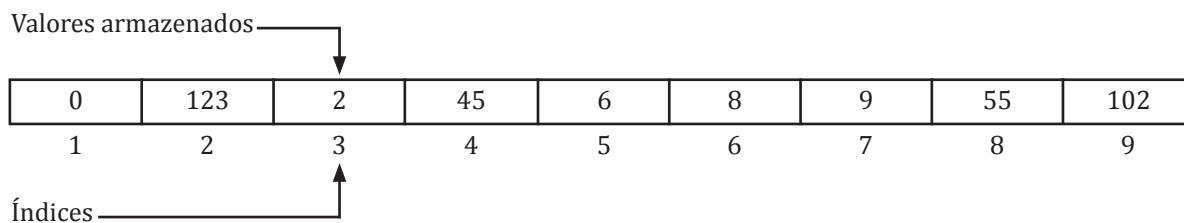
Até o momento, nos preocupamos em tratar das estruturas básicas da programação estruturada e de sua lógica na construção de algoritmos. Deixamos de lado algo muito importante: as variáveis, que armazenam dados. Pois bem, vejamos a seguinte situação:

Digamos que eu precise armazenar duzentos valores numéricos. Teremos então de criar duzentas variáveis, certo? Até este momento, sim. Mas para casos como este é que temos a possibilidade de trabalhar com os conjuntos de variáveis.

Um desses conjuntos de variáveis são as matrizes, ou conjunto de dados, assunto de que trataremos a partir deste tópico.

Matrizes de uma dimensão

Uma matriz de uma dimensão, também chamada vetor ou conjunto, é uma variável que pode armazenar diversos valores, os quais podem ser manipulados por meio de um índice, conforme figura abaixo:



De tal modo, temos uma variável que pode conter diversos valores, cada um deles referenciado por meio de um índice.

Ex.:

Variável [4] = 45 Variável [7] = 9

Sempre que vamos usar os valores de uma variável do tipo matriz, devemos fazer referência ao nome da variável e à posição que desejamos trabalhar. Desta forma: NomeVariavel[posição].

A declaração de uma variável deste tipo com a representação no Português Estruturado pode ser:

Var

NomeDaVariavel: vetor [1..?] de <tipo>

Digamos que queremos declarar uma variável vetor com dez posições do tipo inteiro.

A declaração seria desta forma:

VET1: vetor [1..10] de inteiro

Na seção de declaração das variáveis, escolhemos um nome para a variável que representará o vetor (NomeDaVariavel) e a declaramos como um vetor, criando a quantidade de posições que esta deve ter por meio dos índices da mesma [1..?], que devemos iniciar com a posição 1 e terminar com a posição desejada. Depois de determinarmos os índices, escolhemos o tipo de valor que desejamos manipular.

77

Ao declaramos a variável VET1, criamos uma estrutura que permite armazenar até dez números do tipo inteiro. Perceba que a variável só foi criada na memória. Assim, para alocarmos valores dentro dela devemos fazer as devidas referências. Exemplo: Digamos que quiséssemos armazenar o valor dez na primeira posição. Teríamos:

VET1[1] := 10

Dessa forma, esta variável estaria exatamente assim na memória do computador:

10				
----	--	--	--	--

Para mostrarmos este valor armazenado na memória dentro da variável VET1, devemos proceder da seguinte maneira:

Escreva (VET1[1])

Como precisamos mostrar sempre os valores indexados nas variáveis do tipo vetor, os valores do índice podem ser substituídos por uma variável. Vejamos o seguinte exemplo, no qual desejamos preencher um vetor de cinco posições com valores digitados pelo usuário e posteriormente mostrar os valores armazenados na tela do computador.

algoritmo “declaramatriz”

// Função : Declarar uma matriz de uma dimensão preenchendo e mostrando valores

// Autor : Luiz Carlos

// Data : 30/01/2014

// Seção de Declarações

var

MAT : vetor [1..5] de inteiro

X : inteiro

inicio

para X de 1 ate 5 faca

escreval (“entre com o valor a ser armazenado na posição: “ , X)

leia (MAT[X])

fimpara

para X de 1 ate 5 faca

escreva (MAT[X])

fimpara

fimalgoritmo

Definimos um vetor de cinco posições com o nome de MAT. Usaremos uma variável X para controle dos laços e índices.

O Laço “para” vai repetir os comandos cinco vezes, e X vai se iniciar com o valor 1 até o valor 5, valor limite para armazenar dados no vetor que criamos. A cada interação do laço, o comando “escreva” pede que seja inserido um valor para a posição X, que armazena na variável MAT com o valor do índice X. No segundo Laço, X também irá se iniciar com o valor 1 a cada repetição do laço, X será incrementado mais 1 a X até o valor final 5. A cada laço, é escrito o valor que estará armazenado na variável MAT com o valor de índice igual a X.

A partir deste momento, deixamos de usar a ferramenta FreeDFD, pois ela não dará suporte à criação de variáveis indexadas (vetores e matrizes) ou variáveis compostas, passando a trabalhar apenas com o *VisuAlg*, que ainda suporta este e outros assuntos que veremos adiante.

Na seção de comandos, estamos pedindo para que seja armazenado o valor 10 na posição 1 da variável VET1.

Usamos o comando escreva fazendo referência à variável VET1, que é um vetor e deve ser colocada sempre com a referência da posição à qual nos reportamos quanto ao valor que queremos mostrar. Assim, temos que o valor a ser mostrado é o da posição 1.

SAIBA MAIS

Resolvendo grandes problemas - Até o presente momento, trabalhamos com exemplos de resolução de problemas simples, de forma direta. Tais exemplos são essenciais para entendermos o funcionamento das estruturas necessárias à construção de algoritmos.

Já falamos um pouco sobre a programação estruturada e que todos os problemas podem ser resolvidos usando as estruturas básicas da programação estruturada, que são: Sequência, Decisão e Repetição.

Deste modo, por maior que seja o problema descrito neste material, já sabemos que podemos resolvê-lo com o uso destas estruturas.

Uma técnica bastante usada para a resolução de grandes problemas é dividir para conquistar, que consiste em dividir um grande problema em pequenos problemas e resolvê-los um a um. Após isso, já conhecemos a solução, bastando uni-los em uma única estrutura sequencial. O exemplo abaixo mostra um pouco desta técnica.

▶ EXERCÍCIO RESOLVIDO

Construa um algoritmo de um número qualquer e armazene os dez números pares subsequentes em um vetor de dez posições. Em outro vetor, armazene os dez números ímpares subsequentes ao número digitado. Ao final, apresente a soma dos dez números pares e a soma dos dez números ímpares, assim como todos os números armazenados nos respectivos vetores.

É um exemplo complexo, mas que pode ser resolvido dividindo o problema em várias partes.

1. Definição das variáveis:

Preciso definir dois vetores de tamanho 10, uma variável para receber um número fornecido pelo usuário e uma variável para laço de controle. Teremos:

// Seção de Declarações

var

PAR, IMPAR: vetor [1..10] de inteiro

NUM, X: inteiro

2. Inicialmente, preciso receber um número do usuário. Temos:

Início

// Seção de Comandos

escreval("entre com um número")

leia (NUM)

3. Descobrir se o número digitado é par ou ímpar para prosseguir com os cálculos:

// Seção de Comandos

se (NUM MOD 2=0) entao

 escreval ("numero digitado é par")

senao

 escreval ("numero digitado é impar")

fimse

4. Caso o número seja par, temos que proceder ao armazenamento dos próximos dez números pares. Para isso, devemos criar uma nova variável, que será o próximo número a ser armazenado, a chamaremos de PAR. Se o número digitado é par e somarmos mais dois a ele obteremos o próximo par, armazenando-o na primeira posição da variável. Ao somarmos dois ao par encontrado anteriormente, teremos o próximo, armazenando-o no próximo espaço da variável, continuando até o décimo número. Como o procedimento se repete, devemos usar uma estrutura de repetição, reproduzindo estes passos do 1 até o 10.

Ao adicionarmos 1 ao número par digitado, encontramos o próximo ímpar a ser armazenado. Depois de encontrado o primeiro ímpar, os próximos dez serão encontrados ao acrescentarmos dois a cada interação a partir da primeira e armazenarmos no local desejado.

Temos então o seguinte código:

```
//Seção de declarações
se (NUM MOD 2=0) entao
    escreval ("numero digitado é par")
    PPAR:= NUM           //Se o Numeo é par, precisamos armazenar os 10 próximos
    PIMPAR:=NUM+1        //Se o número é par, o próximo é ímpar..
    para X de 1 ate 10 faca
        PPAR:= PPAR+2    //Adiciona +2 aos próximos 10 números
        PAR[X]:= PPAR    //Armazenar o valor PAR no vetor
        IMPAR[X]:= PIMPAR //Armazena os próximos 10 números ímpares
        PIMPAR :=PIMPAR+2 //Vai para o próximo número ímpar a cada interação
    fimpara
senao
    escreval ("numero digitado é ímpar")
fimse
```

5. Caso o número digitado inicialmente seja ímpar, a lógica é semelhante à usada anteriormente, segundo a qual devemos apenas descobrir o primeiro próximo par, somando um ao número inicial, e o primeiro próximo ímpar, somando dois ao número inicial. O restante segue a mesma lógica, ficando então o bloco desta forma:

```
//Condição em que o número digitado é ímpar..
se (NUM MOD 2=0) entao
    escreval ("numero digitado é par")
senao
    escreval ("numero digitado é ímpar")
    PPAR:= NUM +1        //O próximo Par será o número +1
    PIMPAR:=NUM          //O número é ímpar..
    para X de 1 ate 10 faca
```

```

PAR[X]:= PPAR           //Armazenar o valor PAR no vetor
PPAR:= PPAR+2           //Adiciona ao primeiro PAR para os próximos 10 números
PIMPAR :=PIMPAR+2       //Descobre o próximo número IMPAR
IMPAR[X]:= PIMPAR       //Armazena os próximos 10 números impares
fimpara
fimse

```

6. Agora veremos como somar os números já armazenados nos vetores par e ímpar. Nesta operação, será necessário usar uma variável para acumular os valores armazenados nos vetores. Novamente, usaremos um comando de repetição para ler todos os valores armazenados nos vetores, acumulando-os nas variáveis. Criaremos duas variáveis (SOMAP, SOMAI) para armazenar os valores armazenados em cada um dos vetores, de forma que a cada interação do laço um valor de um dos vetores seja adicionado à variável. Teremos:

```

//Comandos
Para X de 1 Ate 10 faca
    SOMAP:=SOMAP + PAR[X]           //Soma todos os números armazenados em PAR
    SOMAI:=SOMAI + IMPAR[X]        //Soma todos os números armazenados em PAR
fimpara

```

7. Só nos falta apresentar os valores somados de cada vetor e as variáveis que apresentam as somas dos valores armazenados. A fim de obtermos melhor visualização, usaremos dois laços de repetição, um para cada vetor. Teremos:

```

//escrevendo os Valores armazenados
escreval(" números pares armazenados a partir do número ", NUM, " : ")
para X de 1 ate 10 faca
    escreva(PAR[X])
fimpara
escreval(" A soma de todos os números pares foi: ", SOMAP)
escreval(" números impares armazenados a partir do número ", NUM, " : ")
para X de 1 ate 10 faca
    escreva(IMPARG[X])
fimpara
escreval(" A soma de todos os números impares foi: ", IMPAR)

```

Assim, juntando as partes, teremos:

algoritmo “**declaramatriz**”

// Função : Criar vetores para receber valores pares e impares subsequentes

// Autor : Luiz Carlos

// Data : 30/01/2014

// Seção de Declarações

var

PAR, IMPAR : **vetor** [1..10] de **inteiro**

NUM, X : **inteiro**

PPAR, PIMPAR : **inteiro** // Variaveis para calcular valores a serem armazenados

SOMAP, SOMAI : **inteiro** // Variaveis para armazenar valores totais dos vetores

inicio

escreval (“entre com um número”)

leia (NUM)

se (NUM MOD 2=0) **entao**

escreval (“numero digitado é par”)

PPAR:= NUM // Se o número é par, precisamos armazenar os 10 próximos

PIMPAR:= NUM+1 // Se o número é par, o próximo é impar...

para X de 1 ate 10 **faca** //

PPAR:= PPAR+2 // Adiciona +2 aos próximos 10 números

PAR[X]:= PPAR // Armazena o valor PAR no vetor

IMPAR[X]:= PIMPAR // Armazena os próximos 10 números ímpares

PIMPAR[X]:= PIMPAR+2 // Vai para o próximo número impar a cada interação

fimpara

senao

escreval (“numero digitado é impar”)

PPAR:= NUM+1 // O próximo Par será o número +1

PIMPAR:= NUM // O número é impar...

para X de 1 ate 10 **faca** //

PPAR:= PPAR // Adiciona +2 aos próximos 10 números

PAR[X]:= PPAR+2 // Armazena o valor PAR no vetor

PIMPAR:= PIMPAR+2 // Armazena os próximos 10 números ímpares

IMPAR[X]:= PIMPAR // Vai para o próximo número impar a cada interação

fimpara

fimse

```
// Somando valores armazenados
para X de 1 ate 10 faca      //
    SOMAP:= SOMAP + PAR[X]  // Soma todos os números armazenados em PAR
    SOMAI + IMPAR[X]        // Soma todos os números armazenados em PAR
fimpara

// Escrevendo os valores armazenados
escreval ("números pares armazenados a partir do número ", NUM, " : ")
para X de 1 ate 10 faca      //
    escreva (PAR[X])
fimpara
escreval ("A soma de todos os números pares foi: ", SOMAP)
escreval (" números impares armazenados a partir do número ", NUM, " : ")
para X de 1 ate 10 faca
    escreva (PAR[X])
fimpara
escreval ("A soma de todos os números ímpares foi: ", IMPAR)
```

fimalgoritmo

83

▶ EXERCÍCIO PROPOSTO

1. Interprete os códigos dos algoritmos abaixo:

a) algoritmo "Exercicio"

var

X, Y : inteiro

VET : vetor [1..10] de inteiro

inicio

Para X de 1 ate 10 faca

 Leia (Y)

 VET[X]:= Y * X

fimpara

fimalgoritmo

I - Se Y=4 e X=1, o que acontecerá?

II - Se Y=10 e X= 5, o que acontecerá?

III - Se Y=6 e X= 10, o que acontecerá?

IV - Considerando os valores acima, escreva os valores armazenados em VET conforme suas respectivas posições:

b) algoritmo "Exercicio"

var

X, Y : inteiro

VET1, VET2 : vetor [1..10] de inteiro

inicio

Y := 2

Para X de 1 ate 10 faca

VET1[X] := Y * X

fimpara

Para X de 1 ate 10 faca

VET2[X] := VET1[X] * Y

fimpara

finalgoritmo

I - Quais os valores armazenados nas variáveis VET1 e VET2 ao final da execução do algoritmo?

VET1

--	--	--	--	--	--	--	--	--	--

VET2

--	--	--	--	--	--	--	--	--	--

2. Crie algoritmos para as seguintes situações:

a) Construa um algoritmo para preencher um vetor de cinco posições com números fornecidos pelo usuário. Após o armazenamento, apresente os valores.

b) Construa um algoritmo para preencher um vetor com sete posições somente com números pares fornecidos pelo usuário. Caso o usuário forneça números ímpares, o algoritmo não deve armazená-los, solicitando um novo número. Ao final, apresente os valores armazenados.

c) Construa um algoritmo para preencher um vetor de cinco posições com números que o usuário desejar. Após os dados serem armazenados, crie um novo vetor, também com cinco posições, no qual os valores armazenados serão o valor armazenado no primeiro vetor multiplicado por cinco.

Ex.:

Vetor 1: Valores digitados pelos usuários

A	B	C	D	E
---	---	---	---	---

Vetor 2: Criado automaticamente pelo sistema, sendo os valores

5 * A	5 * B	5 * C	5 * D	5 * E
-------	-------	-------	-------	-------

d) Crie um vetor para receber dez nomes quaisquer, digitados pelo usuário. Depois de preenchido o vetor, crie um sistema de busca para seu algoritmo no qual o usuário poderá fazer quantas buscas quiser digitando um nome (laço controlado pelo usuário). Se o nome estiver armazenado no vetor, o sistema deverá informar a posição do vetor no qual o nome estiver inserido.

Matrizes com mais de uma dimensão

As matrizes são estruturas para armazenamento de dados capazes de receber vários dados de determinado tipo. Observe a imagem abaixo que representa uma matriz de cinco linhas por quatro colunas, ou seja, uma matriz 5x4.

	Colunas			
	1	2	3	4
Linhas	1	0	0	0
	0	1	0	0
	0	0	1	0
	0	0	0	1
	1	0	0	0

Desta forma, podemos ter uma variável do tipo matriz para armazenamento de diversos valores, onde cada valor será referenciado por um valor numérico que representará a linha e outro valor numérico para representar a coluna.

Ex.:

Variavel [1,1] = 1

Variavel [3,2] = 0

Para declararmos uma variável do tipo matriz utilizando a representação do Português Estruturado, temos:

Var

NomeDaVariavel: vetor [1..?, 1..?] de <tipo>

Digamos que queremos declarar um variável do tipo matriz, com o nome de MAT1, com cinco linhas e quatro colunas. Teríamos:

MAT1: vetor [1..5, 1..4] de inteiro

Ao declararmos a variável MAT1, criamos uma estrutura que permite armazenar até vinte números do tipo inteiro. Perceba que a variável só foi criada na memória. Assim, para alocarmos valores dentro dela, devemos fazer as devidas operações de manipulação.

Exemplo: Digamos que queremos armazenar o valor 10 na primeira posição da variável. Teríamos:

Sempre que vamos usar os valores de uma variável do tipo matriz, devemos fazer referência ao nome da variável e informar entre colchetes a posição da linha e da sua coluna. O exemplo ao lado mostra a referência à matriz na posição de linha 1 e coluna 1. No exemplo abaixo, temos linha 3 e coluna 3.

Na seção de declaração das variáveis, escolhemos um nome para a variável que representará a matriz (NomeDaVariavel) e o declaramos como vetor, criando os índices para a linha e para a coluna [1..?, 1..?], que devem se iniciar com a posição 1 e terminar com a posição desejada. Depois de determinarmos os índices, escolheremos os tipos de dados a ser trabalhados.

Na seção de comandos, estamos pedindo que seja armazenado o valor 10 na posição de linha 1 e coluna 1 da variável MAT1.

$\text{MAT1}[1,1] := 10$

Assim, podemos representar a variável do tipo matriz na memória da seguinte forma:

		Colunas			
		1	2	3	4
Linhas	1	10			
	2				
	3				
	4				
	5				

Para mostrarmos este valor armazenado na memória dentro da variável MAT1, devemos proceder desta forma:

Escreva ($\text{MAT1}[1,1]$)

Assim, para manipularmos os valores dos índices nas variáveis do tipo matriz de linha e de coluna, podemos substituir os valores por variáveis, o que permite operar de forma mais dinâmica as operações com uma matriz de grande dimensão. Vejamos o seguinte exemplo, no qual desejamos preencher uma matriz de três linhas por três colunas com valores digitados pelo usuário, e posteriormente mostrar os valores armazenados na tela.

algoritmo “**declaramatriz**”

// Função : Preencher uma matriz 3 por 3

// Autor : Luiz Carlos

// Data : 30/01/2014

// Seção de Declarações

var

MAT : vetor [1..3, 1..3] de inteiro

X : inteiro

// Variável para controlar linha e coluna

inicio

para L de 1 ate 3 faça

para C de 1 ate 3 faça

escreval (“entre com o valor a ser armazenado na posição: “ , L, C)

leia (MAT[L, C])

fimpara

Definimos uma matriz de três linhas por três colunas com o nome MAT. Usaremos uma variável L para controle do laço da linha. Usaremos uma variável C para controle do laço da coluna.

O primeiro laço que irá controlar as repetições para a variável “L”, que representa as linhas, será repetido três vezes. A cada repetição, o segundo laço deverá ser executado três vezes, referentes à variável C, que controla a coluna. O “escreva” pede que seja inserido um valor para a posição L,C, da variável MAT. O comando Leia esperará um valor a ser armazenado na posição da matriz correspondente aos valores de L e C, controlados pelos laços.


```

para L de 1 ate 3 faca
    para C de 1 ate 3 faca
        escreva (MAT[L, C])
    fimpara
finalgoritmo

```

No segundo conjunto de laço, temos o mesmo procedimento para controle das linhas e das colunas, mostrando os valores armazenados na variável MAT [L,C].

Como no exemplo, podemos ver que é possível armazenar diversos valores nas variáveis do tipo Matriz, bem como é possível manipular seus valores.

EXERCÍCIO RESOLVIDO

Construa um algoritmo para preencher uma matriz de duas linhas por quatro colunas a partir de números digitados pelo usuário. Em seguida, leia toda a matriz e apresente o maior e o menor número armazenados na matriz.

algoritmo “declaramatriz”

// Função : Preencher uma matriz e descobrir o maior e menor número

// Autor : Luiz Carlos

// Data : 14/02/2014

var

Mat1 : **vetor** [1..2, 1..4] de **inteiro**

L, C, NUM : **inteiro**

Maior, Menor : **inteiro**

inicio

// Preenchimento da Matriz

para L de 1 ate 2 faca

para C de 1 ate 4 faca

escreval (“Entre com um numero a ser armazenado em: “, L, C)

leia (Mat1[L, C])

fimpara

fimpara

// atribuindo valores iniciais ao maior e menor

Maior:= Mat1[1, 1]

Menor:= Mat1[1, 1]

// Procurando o maior e menor número - Lendo a matriz

para L de 1 ate 2 faca

para C de 1 ate 4 faca

se (Mat1[L, C] > Maior) entao

Maior:= Mat1[L, C]

Nas variáveis, definimos a matriz (Mat1 2x4) e para controlar as linhas e colunas as variáveis L e C. Para armazenar os números do maior e menor número armazenados na matriz, criamos as variáveis Maior e Menor.

Usamos o laço para controle da Linha com a variável L, que vai de 1 até 2, e outro comando para controlar a coluna de 1 até 4, variável C. Depois de inserimos o valor digitado diretamente na matriz com a linha e a coluna definida pelos laços de controle.

Usamos o pressuposto de que um número deve ser o maior e o menor dentre os digitados. Pegamos o primeiro número armazenado para ser o maior e o menor. Assim, temos um número para começar a comparar aos demais armazenados.

Usamos novamente os comandos de repetições para controle de linha e coluna para lermos toda a matriz preenchida e vamos comparar cada valor da matriz aos valores já armazenados para descobrir se é maior ou menor do que o valor já armazenado nas variáveis Maior e Menor. Assim, encontraremos o maior e o menor número armazenados na Matriz depois de lê-la por inteiro e comparar todos os seus valores.

Para imprimirmos a matriz, devemos saltar uma linha a cada vez que escrevermos os valores presentes nas colunas. Assim, colocamos um “escreval” dentro do laço da linha. No laço da coluna, colocamos só o comando escreva e uma vírgula que aparecerá a cada número impresso. Assim, escrevemos a matriz de forma parecida como ela é representada graficamente.

```
para L de 1 ate 2 faca
  para C de 1 ate 4 faca
    se (Mat1[L, C] > Maior) entao
      Maior:= Mat1[L, C]
    fimse
    se (Mat1[L, C] < Menor) entao
      Menor:= Mat[L, C]
    fimse
  fimpara
fimpara
escreval ("O maior valor foi: ", Maior, " E o menor foi: ", Menor)
// Escrevendo a matriz em forma de Matriz na tela
para L de 1 ate 2 faca
  escreval (" ")
  para C de 1 ate 4 faca
    escreva (Mat1[L, C], ", ")
  fimpara
fimpara
finalgoritmo
```

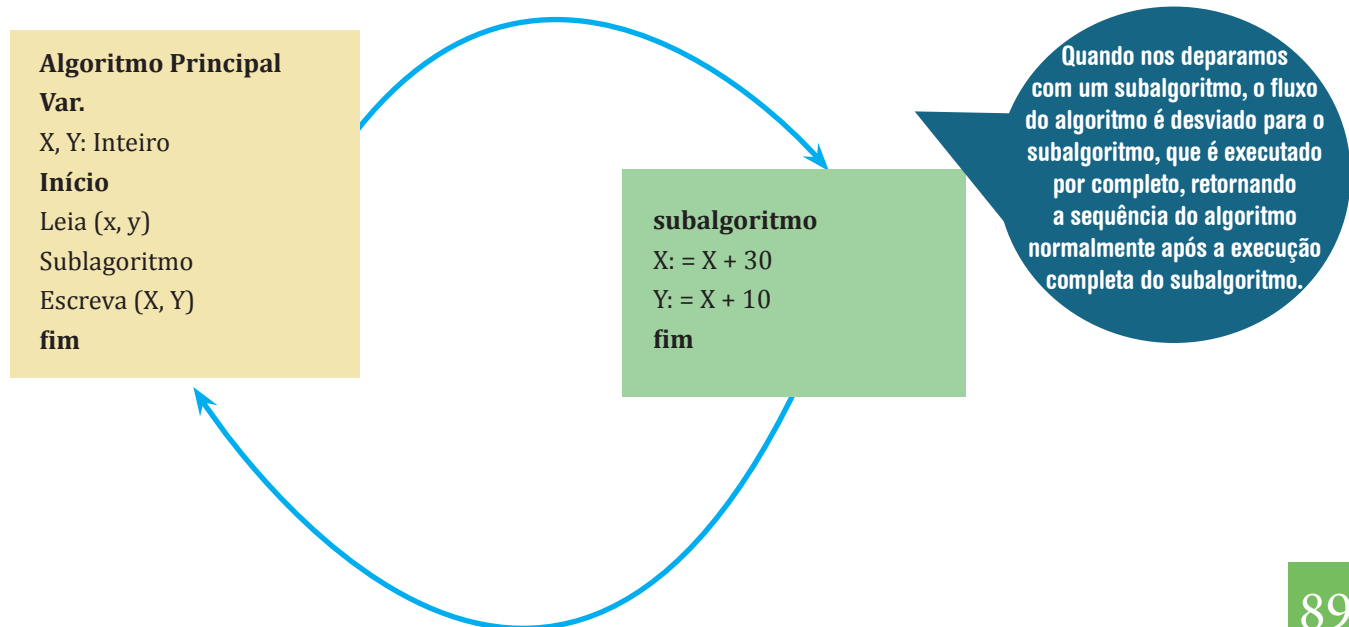
▶ EXERCÍCIO PROPOSTO

1. Construção dos algoritmos:

- Construa um algoritmo para preencher uma matriz de três linhas por três colunas na qual o usuário digite todos os valores. Ao final, apresente na tela os valores armazenados na matriz.
- Construa um algoritmo para preencher uma matriz quatro por quatro. Ao final, o algoritmo deve ler toda a matriz e mostrar quantos números ímpares e quantos números pares estão armazenados na matriz.
- Construa um algoritmo que preencha uma matriz três por três. Após o preenchimento, o algoritmo deverá mostrar a soma dos números armazenados em cada linha da matriz.
- Construa um algoritmo que preencha uma matriz três por três. Após o preenchimento, o algoritmo deverá mostrar a soma dos números armazenados em cada coluna da matriz.
- Construa um algoritmo que preencha uma matriz três por três e apresente a soma apenas dos elementos nos quais o número da linha seja igual ao número da coluna.
- Construa um algoritmo para preencher uma matriz dez por dez a partir de um número digitado. Os valores que comporão a matriz serão os valores digitados multiplicados pelo número da linha e somados ao número que representa a coluna.
- Construa um algoritmo que preencha duas matrizes três por três de forma manual. O algoritmo deverá criar, de forma automática, uma terceira matriz três por três e preencher os valores com a soma dos elementos das duas matrizes. Em outras palavras, na matriz três a posição de linha um e coluna um será igual à matriz um posição de linha um e coluna um somada ao valor armazenado na matriz duas posição de linha um e coluna um, e assim sucessivamente para todos os elementos das matrizes.

Programação em bloco (Subalgoritmos)

Programação em bloco ou programação com subalgoritmos é um conceito utilizado para agrupar um ou vários comandos em uma estrutura, semelhante a um bloco, o qual permite ser chamado no corpo de um algoritmo, desviando a execução do algoritmo para o bloco de comandos em questão e, após a execução do bloco, retorna ao ponto de desvio no corpo do algoritmo (MANZANO; OLIVEIRA, 2010).



São dois os tipos de subalgoritmos que podemos ter:

Procedimentos: É um bloco de comandos que ao ser chamado executa uma série determinada de comandos.

Função: É semelhante ao procedimento, podendo retornar, porém, um valor ao comando que chamou a função.

A grande diferença entre procedimento e função é que a função pode retornar um valor ao algoritmo principal após a execução de seus comandos, ao passo que o procedimento não retorna valores.

Tanto os procedimentos quanto as funções podem ter variáveis próprias, chamadas variáveis locais, que só existirão enquanto a função estiver sendo executada, não sendo utilizada em outras funções ou no corpo do algoritmo. Também podem trabalhar com variáveis do tipo global, ou seja, uma variável definida no algoritmo que poderá ser utilizada nos procedimentos e funções.

Para entendermos melhor, vamos estudar cada um destes subalgoritmos.

Procedimento

Um procedimento deve ser declarado entre a seção de declaração das variáveis e o início do programa. Assim, podemos declarar da seguinte forma:

algoritmo DeclaraProcedimento**var**

Var1, Var2.. VarX: <tipo>

procedimento <nome_Procedimento>**var**

VarA, VarB.. VarZ: <tipo>

Inicio

<comandos>

fimprocedimento**inicio**

<comandos>

<nome_Procedimento>

fimalgoritmo

Um procedimento pode ter suas próprias variáveis (locais), bem como usar as variáveis definidas no algoritmo principal (Global).

Quando queremos chamar um procedimento, fazemos a citação ao seu nome no corpo do algoritmo, assim como fazemos com os comandos. Como ela está definida acima, o algoritmo sabe que se trata de um bloco de comandos.

Vejamos a seguinte aplicação prática:

Vamos construir um algoritmo para executar a soma de dois números, usando um procedimento para a execução dessa soma:

algoritmo “ProcedimentoSOMA”

// Função : Apresentar o uso do procedimento

// Autor : Luiz Carlos

// Data : 15/02/2014

varNUM1, NUM2, S: inteiro // Declarando um procedimento para a somaprocedimento somavar

// Declarando uma variável local para cálculos

AUX: inteiroinicio

// NUM1, NUM2 e S são variáveis globais

AUX := NUM1 + NUM2 // NUM1, NUM2 valores armazenados

S := AUX // Transferência de valores: Local (AUX) para Global(S)

fimprocedimento

inicio

escreva ("entre com o 1 valor: ")

leia (NUM1)

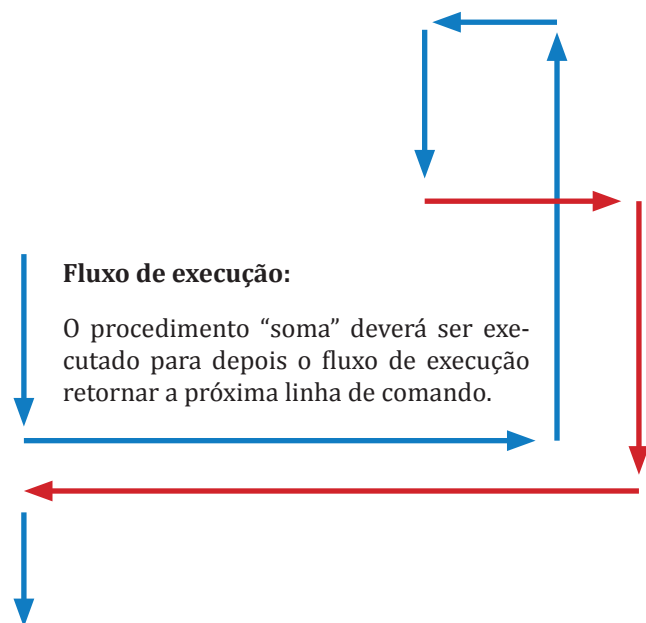
escreva ("entre com o 2 valor: ")

leia (NUM2)

soma // Chama o procedimento soma

escreva (S)

fimprocedimento



EXERCÍCIO RESOLVIDO

1. Construção dos algoritmos:

- Construa um algoritmo para preencher uma matriz de três linhas por três colunas na qual o usuário digite todos os valores. Ao final, apresente na tela os valores armazenados na matriz.
- Construa um algoritmo para preencher uma matriz quatro por quatro. Ao final, o algoritmo deve ler toda a matriz e mostrar quantos números ímpares e quantos números pares estão armazenados na matriz.
- Construa um algoritmo que preencha uma matriz três por três. Após o preenchimento, o algoritmo deverá mostrar a soma dos números armazenados em cada linha da matriz.
- Construa um algoritmo que preencha uma matriz três por três. Após o preenchimento, o algoritmo deverá mostrar a soma dos números armazenados em cada coluna da matriz.
- Construa um algoritmo que preencha uma matriz três por três e apresente a soma apenas dos elementos nos quais o número da linha seja igual ao número da coluna.
- Construa um algoritmo para preencher uma matriz dez por dez a partir de um número digitado. Os valores que comporão a matriz serão os valores digitados multiplicados pelo número da linha e somados ao número que representa a coluna.
- Construa um algoritmo que preencha duas matrizes três por três de forma manual. O algoritmo deverá criar, de forma automática, uma terceira matriz três por três e preencher os valores com a soma dos elementos das duas matrizes. Em outras palavras, na matriz três a posição de linha um e coluna um será igual à matriz um posição de linha um e coluna um somada ao valor armazenado na matriz duas posição de linha um e coluna um, e assim sucessivamente para todos os elementos das matrizes.

algoritmo "PalpiteLoteria"

// Função : Gera números para palpite de loterias

// Autor : Luiz Carlos

// Data : 15/02/2014

var

APOSTA: vetor [1..6] de inteiro

NUM, X: Inteiro

NOVO: caracter

Optamos por
usar um vetor de
06 posições para
armazenar o palpite
gerado pelo
sistema.

Neste procedimento, apenas geramos um número aleatório e passamos para a variável global "NUM" a cada execução do procedimento.

Neste procedimento, colocamos a rotina de impressão das apostas por meio do comando de repetição para com a variável local "y" para controlar a leitura do vetor da aposta "APOSTA"

```

procedimento gerapalpite // Procedimento para gerar um palpite
var
AUX: inteiro // Declarando uma variável local para cálculo do palpite
inicio
AUX := RANDI (60) // Função RANDI () - gera números aleatórios de 0 a 60
NUM:= AUX // Armazena o número aleatório gerado na variável global NUM
fimprocedimento

procedimento imprimepalpite // Procedimento para impressão do palpite
var
Y: inteiro
inicio
escreval ()
para Y de 1 ate 6 faca
    escreva (APOSTA[Y])
fimpara
escreval ()
fimprocedimento

inicio // Início do programa principal
NOVO:= "s"
repita // Repete para gerar quantas apostas desejar
    escreva ("Este programa vai gerar um palpite para aposta na megasena")
    para X de 1 ate 6 faca // Armazena os números gerados aleatoriamente
        gerapalpite // Chama o procedimento para gerar um número aleatório
        APOSTA[X]:= NUM
    fimpara
    imprimepalpite // Chama o procedimento para escrever o palpite gerado
    escreva ("tecle 'S' para gerar um novo palpite")
    leia (NOVO)
    ate (NOVO <> "s")
fimalgoritmo

```

O comando repita do programa principal irá permitir que sejam geradas quantas apostas o usuário desejar.

O programa principal apenas controla o armazenamento dos números gerados aleatoriamente por meio da chamada da função "gerapalpite" a cada interação do laço para, que trará um número aleatório a cada interação do laço e armazenará na posição do vetor.

EXERCÍCIO PROPOSTO

1. Melhore o programa do exercício resolvido fazendo as seguintes alterações:

- Alterar o procedimento “gerapalpite” para que sejam armazenados os seis palpites no procedimento, evitando que ele seja chamado seis vezes durante a execução do programa;
- Garanta que não serão gerados números repetidos nos palpites, pois na forma como o programa está pode ocorrer que um número seja sugerido repetidas vezes.
- Crie um novo procedimento para organizar os números gerados por ordem crescente, pois na forma atual os números não são armazenados em ordem crescente. Crie um procedimento para isto.
- Melhore o visual da apresentação das apostas, colocando mais detalhes na apresentação das apostas ao usuário. Crie um procedimento para isto.

2. Crie um algoritmo usando procedimentos para uma calculadora com as quatro funções aritméticas básicas, de forma que o usuário poderá executar quantas operações desejar, saindo do algoritmo quando assim desejar.

Funções

Uma função também é um bloco contendo diversas instruções. Porém, diferentemente dos procedimentos, uma função pode receber valores por parâmetro e retornar um valor para o comando que a chamou. Sua sintaxe é apresentada abaixo:

algoritmo “NomeAlgoritmo”

var

VAR1, VAR2.. VARN: <tipo>

funcao <Nome_Função>(parâmetros): <Tipo_Função>

var

VARA, VARB.. VARX: <tipo>

inicio

<comandos>

retorne (VALOR)

fimfuncao

inicio

<comandos>

<Nome_Função (parâmetros)>

fimalgoritmo

Envia valores para o comando que chamou.

Envia valores para a função.

Na declaração de uma função, obrigatoriamente devemos informar o tipo de valor que a função retornará, definido o tipo da função.

Ao chamarmos a função, podemos enviar e receber valores pela própria chamada.

Vejamos o exemplo a seguir usando uma função que retornará determinado valor:

```

algoritmo "função"

// Função : Apresenta o uso de uma função
// Autor : Luiz Carlos
// Data : 15/02/2014

var

    NUM1, NUM2: inteiro
    RESULT: inteiro
    // Declarando uma função
    funcao calculo: inteiro
    var
        X: inteiro
    inicio
        X <- NUM1 * NUM2
    retorne X
    fimfuncao

// Programa Principal

inicio
    escreval ("entre com o 1 valor ")
    leia (NUM1)
    escreval ("entre com o 2 valor ")
    leia (NUM2)
    RESULT <- calculo
    escreva (RESULT)
fimalgoritmo
  
```

Declaração da Função do tipo inteiro, ou seja, retorna ("retorne") um valor inteiro ao comando que chamou. No caso, retornará o valor de X, que é uma variável local.

Na variável "RESULT", atribuímos o valor da função "calculo", que terá o valor de seu retorno "retorne" como valor de "X".

Agora, observe este novo exemplo enviando valores para a função:

```

algoritmo "função"

// Função : Apresenta o uso de uma função
// Autor : Luiz Carlos
// Data : 15/02/2014

var

    NUM1, NUM2: inteiro
    RESULT: inteiro
    // Declarando uma função
  
```

Recebemos os valores de NUM1 e NUM2 e os armazenamos temporariamente em X e Y. Desta forma, manipulamos apenas os valores de X e Y, preservando as variáveis NUM1 e NUM2.


```
funcao calculo (X, Y: inteiro): inteiro
```

```
var
```

```
Z: inteiro
```

```
inicio
```

```
Z: X + Y
```

```
retorne Z
```

```
fimfuncao
```

```
// Programa Principal
```

```
inicio
```

```
escreval ("entre com o 1 valor ")
```

```
leia (NUM1)
```

```
escreval ("entre com o 2 valor ")
```

```
leia (NUM2)
```

```
RESULT <- calculo (NUM1, NUM2)
```

```
escreva (RESULT)
```

```
fimalgoritmo
```

Retorno do valor calculado na variável local Z para a chamada da função.

Ao chamar a função, passamos os valores de NUM1 e NUM2 para a Função e recebemos o valor calculado "retorne" da função.

A passagem de valor com parâmetros para referência permite que os valores de uma ou mais variáveis sejam preservados, pois criamos uma imagem das variáveis contendo o mesmo valor das originais para trabalho dentro da função em questão. Assim, podemos manipular valores sem comprometer os valores originais de uma ou mais variáveis.

EXERCÍCIO RESOLVIDO

1. Crie um algoritmo usando função para a ativação de comandos em um robô, sendo as seguintes funções básicas:

- a) Andar: Move o robô por passos nas coordenadas para frente(X) e para trás (Y);
- b) Rotacional: O robô se move segundo valores nas coordenadas por passos à direita (X) e à esquerda (Y).

```
algoritmo "Controlrobô"
```

```
// Função : Apresentar o uso de uma função no controle de um robô
```

```
// Autor : Luiz Carlos
```

```
// Data : 15/02/2014
```

```
var
```

```
X, Y: inteiro
```

```
OP, RESULT: caracter
```

```
// Função para mover o robô para frente e para trás
```

```
funcao andar (A, B: inteiro): caracter
```

Esta função recebe dois valores inteiros por parâmetro e retorna um valor do tipo caractere pelo "retorne", que terá o valor de "RESP".

```

var
RESP: caracter

inicio
se (X > 0) entao
    escreval ("movendo para frente...")
    RESP:= "o robô se moveu para frente - comando concluído"
    retorne RESP
fimse

se (Y > 0) entao
    escreval ("movendo para trás")
    RESP:= "o robô se moveu para trás - comando concluído"
    retorne RESP
fimse

fimfuncao

// Função para rotacionar o robô para esquerda e direita
funcao rotacionar (A, B: inteiro): caracter
var
RESP: caracter
inicio
se (X > 0) entao
    escreval ("movendo para esquerda")
    RESP:= "o robô se moveu para esquerda - comando concluído"
    retorne RESP
fimse

se (Y > 0) entao
    escreval ("movendo para direita")
    RESP:= "o robô se moveu para direita - comando concluído"
    retorne RESP
fimse

fimfuncao

// Programa principal

```

Esta função recebe dois valores inteiros por parâmetro e retorna um valor do tipo caractere pelo "retorne", que terá o valor de "RESP".

inicio

repita

```

escreval ("o que você deseja fazer")
escreval ("A para andar")
escreval ("B para se mover")
escreval ("Tecle outro valor para sair do programa")

```

```
leia (OP)
```

```
escolha (OP)
```

```
caso="A"
```

```
    escreval ("Deseja mover para frente quantos passos")
```

```
    leia (X)
```

```
    escreval ("Deseja mover para trás quantos passos")
```

```
    leia (Y)
```

```
    RESULT:= andar (X, Y)
```

```
caso= "B"
```

```
    escreval ("Deseja rotacionar o robô quantos giros a direita")
```

```
    leia (X)
```

```
    escreval ("Deseja rotacionar o robô quantos giros a esquerda")
```

```
    leia (Y)
```

```
    RESULT:= rotacionar (X, Y)
```

```
outrocaso
```

```
    escreval ("saindo do programa")
```

```
fimescolha
```

```
escreval (RESULT)
```

```
escreval ()
```

```
ate (OP <> "A") e (OP <> "B")
```

fimalgoritmo

No programa principal, criamos um laço de repetição para o comando do nosso robô imaginário. O usuário pode sair do programa ao digitar uma opção não prevista pelo comando escolha.

Chamamos uma função passando os valores de X e Y para ela e aguardamos um retorno como resposta, armazenando a mensagem de retorno na variável "RESULT".

Note que neste exemplo criamos um algoritmo que pode representar os comandos básicos de movimento de um robô, uma abstração da realidade, mas com uma lógica que realmente funciona, ou seja, podemos comandar um robô de forma bem parecida com a deste exemplo que criamos para demonstrar o uso de funções.

▶ EXERCÍCIO PROPOSTO

1. Construa um algoritmo no qual o usuário entre com um número que será avaliado por uma função, retornando uma mensagem que informa se o número é par ou ímpar.
2. Construa um algoritmo para uma calculadora com as funções aritméticas básicas usando funções para cada operação.
3. Crie um algoritmo no qual desejamos calcular uma função do tipo: $F(X) = X^2 + 5X - 3$ onde o usuário entra com o valor de X e o algoritmo calcula o valor de F(X) em uma função informando o resultado.
4. Crie um algoritmo no qual o usuário entrará com três valores para as variáveis X, Y e Z, onde estes valores serão passados para uma função como parâmetros por referência, onde será realizado o cálculo da seguinte função G, tendo como retorno o valor segundo a equação: $G(X, Y, Z) = 3X^2 + 5Y + Z$.
5. Construa um algoritmo que preencha uma matriz de dez linhas por dez colunas com números aleatórios (usando a função `randl(1000)`) por meio de um procedimento próprio para esta tarefa. Crie um menu com as seguintes funções a serem executadas:
 - i. **Maior:** Procura o maior número armazenado na matriz;
 - ii. **Menor:** Procura o menor número armazenado na matriz;
 - iii. **Soma:** Soma todos os elementos da matriz.

01. Por que usar subalgoritmos?

A vantagem do uso dos subalgoritmos é termos blocos de comandos que podem ser chamados em qualquer parte de um programa, evitando a reescrita de parte dos códigos sempre que preciso, economizando em linhas de programação e tornando a execução dos programas mais rápida. Também podemos citar a organização do programa, pois temos um algoritmo mais legível.



Banco de Imagens/NEAD

02. Como definir entre o uso de um procedimento ou uma função?

Quando desejamos que o subalgoritmo retorne um valor para o algoritmo, devemos usar uma função. Em outros casos, podemos usar tanto uma função como um procedimento.

03. É possível passar valores para parâmetros em um procedimento?

Sim. Isto pode acontecer da mesma forma como passamos valores para uma função. A única diferença é que não podemos enviar um valor de resposta como em uma função, mas podemos mudar valores em variáveis usadas no algoritmo. Vejamos o exemplo abaixo:

algoritmo "ProcedimentoSOMA"

// Função : Apresentar o uso do procedimento

// Autor : Luiz Carlos

// Data : 15/02/2014

var

NUM1, NUM2, S: inteiro

// Declarando um procedimento para a soma

procedimento soma (A, B: inteiro)

var

// Declarando uma variavel local para cálculos

AUX: **inteiro**

inicio

AUX:= A = B // A e B passado por referencia de NUM1 e NUM2

S:= AUX // Transferência de valores: Local (AUX) para Global (S)

fimprocedimento

// Seção de Comando do programa principal

inicio

escreva ("entre com o 1 valor ")

leia (NUM1)

escreva ("entre com o 2 valor ")

leia (NUM2)

soma (NUM1, NUM2) // Chama o procedimento soma passando valores por referencia

escreva (S)

fimalgoritmo

OBSERVAÇÃO:

A partir deste momento, deixamos o uso da ferramenta *VisuAlg*, pois ela não suporta mais nosso próximo assunto: estrutura de dados heterogêneas. Esperamos que o uso dessa ferramenta tenha sido útil em seu aprendizado como foi útil para testarmos os algoritmos apresentados aqui até o momento.

Registros

UN 03

Até este ponto, aprendemos sobre as estruturas homogêneas para armazenar grandes quantidades de dados nas matrizes. No entanto, poderíamos apenas armazenar um tipo de dado de acordo com o que foi definido. Neste tópico, apresentamos os registros, que são estruturas de armazenamento de dados heterogêneos e podem armazenar diversos tipos de dados em um mesmo local, ou seja, em um registro.

Imaginemos uma situação na qual precisemos armazenar o nome de um aluno, suas quatro notas e sua média. Com os conhecimentos até o presente momento, isto seria possível, mas só para armazenar os valores para um aluno. E se tivéssemos de armazenar os valores para cinquenta alunos? Mesmo usando as variáveis do tipo Matriz, seria complexo resolvermos tal problema. Para esta situação, criamos os registros de dados.

Para declararmos um registro, usamos uma seção especial acima da seção de declaração das variáveis. Vejamos:

algoritmo DeclaraRegistro**tipo**

Nome_Registro = REGISTRO

Var1: Tipo1

Var2: Tipo 2

VarN: TipoN

FimRegistro

var**Nome_Variavel:** Nome_Registro

Um exemplo típico do uso dos registros é a do boletim escolar do aluno com sua média, conforme podemos ver na imagem abaixo:

REGISTRO DO ALUNO					
Nome do Aluno					
Turma					
Disciplina	Nota 1	Nota 2	Nota 3	Nota 4	Média

Um registro específico com os campos preenchidos poderia ser:

REGISTRO DO ALUNO					
Nome do Aluno	Francisco Cipriano				
Turma	3-A				
Disciplina	Nota 1	Nota 2	Nota 3	Nota 4	Média
Português	10,0	7,0	8,0	7,0	8,0

O exemplo abaixo ilustra a criação do registro mostrado, bem como o preenchimento dos campos por meio do comando de inserção de dados leia.

Programa “Registro”**Tipo**

BOLETIM = registro

Nome, Turma, Disciplina: Caracter

Nota1, Nota2: Real

Nota3, Nota4: Real

Media: Real

FimRegistro

Declaração da variável do tipo registro, onde cada uma das variáveis é um campo do registro.

Var

ALUNO: BOLETIM

Declaramos a variável “ALUNO” do tipo registro, que herdará todas as características do seu tipo, ou seja, todos os campos do registro.

Início

Escreva (“Entre com os dados do registro”)

Escreva (“Entre com o nome do aluno”)

Leia (ALUNO.Nome)

Escreva (“Entre com a turma do aluno”)

Leia (ALUNO.Turma)

As operações com registros são realizadas sempre usando o nome da variável e o campo ao qual devemos nos referir.

variavel.<nome_do_Campo>

Escreva (“Entre com a disciplina ”)

Leia (ALUNO. Disciplina)

Escreva (“Entre com as quatro notas”)

Leia (ALUNO.Nota1)

Leia (ALUNO.Nota2)

Leia (ALUNO.Nota3)

Leia (ALUNO.Nota4)

$$\text{ALUNO. Media} := (\text{ALUNO. Nota1} + \text{ALUNO. Nota2} + \text{ALUNO. Nota3} + \text{ALUNO. Nota4}) / 4$$
FimAlgoritmo

Caso desejássemos escrever os valores do registro na tela, procederíamos da mesma forma:

escreva(ALUNO.Media)

Bom, até aqui tudo bem. Porém, nunca temos só um registro para armazenar, mas vários registros. Como exemplo, podemos citar os registros de contato do seu celular, que permitem o armazenamento de centenas de contatos contendo o nome da pessoa, celular e e-mail.

Para fazermos um algoritmo de registro dos contatos de um celular, utilizamos um vetor de um registro. O exemplo abaixo apresenta como isto pode ser feito:

Programa “ContatosTelefone”

Tipo

RAGENDA = registro

Nome, Email, Cel: Carater

FimRegistro

Var

AGENDA:vetor [1..500] de RAGENDA

Criando um vetor de registros

X: inteiro

RESP: Carater

Início

repita

escreva ("Programa Agenda")

escreva ("Deseja Inserir um número na agenda, tecla 'S' ")

leia (RESP)

se (RESP=S) entao

X:=X+1

Escreva ("entre com o nome da pessoa")

Leia (AGENDA[X].Nome)

Escreva ("entre com o e-mail da pessoa")

Leia (AGENDA[X].Email)

Escreva ("entre com o celular da pessoa")

Leia (AGENDA[X].Cel)

fimse

ate (X<>"s")

Caso o usuário deseje incluir um valor no registro, isso deverá ser feito indicando em qual posição deverá ser armazenado o registro no vetor (usamos a variável X para isso).

FimAlgoritmo

Neste exemplo, criamos um laço repita para armazenarmos os contatos telefônicos em uma agenda, na qual um usuário poderá inserir quantos valores desejar. Como usamos um vetor de registro, devemos ter cuidado com o índice que representa tal vetor, posição na qual iremos armazenar os registros.

EXERCÍCIO RESOLVIDO

1. Crie um algoritmo para armazenar os registros de uma pessoa, contendo os campos nome, sexo, endereço, e-mail, CPF, Tel1, Tel2, Tel3, de modo que exista um menu para inserção de cadastro, consulta e exclusão dos registros.

Programa "RegistroPessoas"

Tipo

VTEL: vetor [1..3] de caracter

REGCONTATO = registro

Nome, Email: Carcter

Sexo, Endereco:Carcter

Tel: VTEL

FimRegistro

Var

CONTATO:vetor [1..50] de REGCONTATO

CAD: vetor [1..50] de Carcter

Criando um vetor de 3 posições para armazenar os telefones de contato.

Criando um registro com o vetor: O campo "Tel" do registro recebe o vetor "VTEL"

Criando variável para armazenar até cinquenta registros.

CAD é um vetor para cuidar do armazenamento nos registros.

INDICE, OPINDICE: inteiro

RESP, SAIR: Carater

Procedimento LimpaReg

var

Z: inteiro

Inicio

Para Z de 1 ate 50 faca

CAD[Z]:="S"

fimpara

fimprocedimento

Procedimento NIndice

var

Z: inteiro

inicio

Para Z de 1 ate 50 faca

Se CAD[Z]<>"S" entao

INDICE:=Z

interrompa

Fimse

Se Z=50 entao

Escreva("todos os registros ocupados")

INDICE:=51

fimse

Fimpara

fimprocedimento

Procedimento Incluir

var

Z: inteiro

inicio

NIndice

Se (INDICE<=50) entao

Este procedimento limpa a variável "CAD", que controla os índices de armazenamento no Cadastro. Usaremos o caracter "S" para dizer que o índice no registro poderá ser ocupado.

Este procedimento vai buscar um valor para o índice de armazenamento no registro. Se o vetor CAD estiver vazio, na primeira interação do comando "para" o valor de "Z", que será igual a 1, passa para a variável global "Indice", indicando que é o valor que está disponível para armazenamento. Isto será feito até se que encontre um valor disponível. Caso não haja, o procedimento informará que os registros estão cheios. O comando "interrompa" para a execução do procedimento, o que acontecerá tão logo seja encontrado um valor de índice disponível para armazenamento no cadastro "CONTATO".

Este procedimento é responsável por incluir um cadastro. Inicialmente, devemos chamar o procedimento que nos mostra se há um índice disponível (procedimento VIndice). Caso haja, será feito o cadastro no índice disponível. Se não houver índice, o procedimento é cancelado, pois o valor da variável "INDICE" será de 51, conforme condição no procedimento "VIndice". Após o cadastro, atualizamos o vetor dos índices disponíveis por meio da mudança de valor se "S" para "N", deixando o índice não mais disponível, comando: CAD[INDICE]:="N".

```

Escreva("digite o Nome da pessoa, e-mail, sexo e endereço de cadastro")

Leia (CONTATO[INDICE].Nome, CONTATO[INDICE].Endereco)

Leia (CONTATO[INDICE].Email, CONTATO[INDICE].Sexo)

Para Z de 1 ate 3 faca

    Escreva("Entre com o numero de telefone", Z)

    Leia (CONTATO[INDICE].Tel[Z])

Fimpara

CAD[INDICE]:= "N"

Senao

    Escreva("Procedimento de inclusão abortado- registro de conatos cheio")

fimpara

```

```

fimprocedimento

```

```

Procedimento Pesquisar

```

```

var

```

```

    Z: inteiro

```

```

    PNome: Caracter

```

```

inicio

```

```

escreva("Digite o nome do contato que você quer pesquisar")

```

```

leia (PNome)

```

```

para Z de 1 ate 50 faca

```

```

    se (PNome=CONTATO[Z].Nome) entao

```

```

        OPIndice=Z

```

```

        interrompa

```

```

    fimse

```

```

    se Z=50 entao

```

```

        escreva("nome de não encontrado no registro")

```

```

        OPIndice=51

```

```

    Fimse

```

```

fimpara

```

```

fimprocedimento

```

Neste procedimento, podemos pesquisar qualquer dado armazenado no registro "CONTATO" e obter seu índice, variável "OPINDICE", que permite a operação de exclusão do registro (caso deseje) ou somente a visualização do registro.

Procedimento Mostrar

var

Z, Y: inteiro

OP: Caracter

inicio

escreva ("Tecla 'A' Para mostrar todos os registros na tela ")

escreva ("Tecla 'B' Para mostrar apenas 1 registro - pesquisar ")

leia (OP)

escolha (OP)

caso = A

Para Z de 1 ate 50 faca

escreva (CONTATO[INDICE].Nome, CONTATO[INDICE].Endereco)

escreva (CONTATO[INDICE].Email, CONTATO[INDICE].Sexo)

Para Y de 1 ate 3 faca

Leia (CONTATO[INDICE].Tel[Z])

fimpara

Fimpara

Caso = B

Pesquisar

Se (OPINDICE<=50) entao

escreva(CONTATO[OPINDICE].Nome, CONTATO[OPINDICE].Endereco)

escreva (CONTATO[OPINDICE].Email, CONTATO[OPINDICE].Sexo)

Para Y de 1 ate 3 faca

Leia (CONTATO[OPINDICE].Tel[Z])

Fimpara

senao

Escreva("não há registros para impressão")

outrocaso

Escreva("Opção inválida - retorno ao programa principal")

fimescolha

fimprocedimento

Neste procedimento, apresentaremos as informações armazenadas nos registros "CONTATOS". Podemos mostrar todos os registros armazenados ou apenas um registro, por meio da chamada ao procedimento "Pesquisar", que retornará o índice do registro no qual existe o cadastro desejado.

Procedimento excluir

var

 Z: inteiro

 OP: Caracter

inicio

 escreva ("Tecle 'A' Para excluir todos os registros ")

 escreva ("Tecle 'B' Para excluir apenas 1 registro - pesquisar ")

 leia (OP)

 escolha (OP)

 caso = A

 LimpaReg

 Caso = B

 Pesquisar

 Se (OPINDICE<=50) entao

 CAD[INDICE]:= "S"

 senao

 Escreva("não há registros para exclusão")

 Fimse

 outrocaso

 Escreva("Opção inválida – retorno ao programa principal")

 fimescolha

fimprocedimento

Inicio

SAIR:="N"

repita

 escreva ("+++++++ Programa Contatos ++++++")

 escreva (" O que você deseja fazer? ")

 escreva (" Tecle A – Para incluir um novo contato ")

 escreva (" Tecle B – Para mostrar um contato registrado ")

 escreva (" Tecle C – Para Excluir um contato ")

 escreva (" Tecle D – Para sair do programa ")

 leia (RESP)

 escolha (RESP)

Neste procedimento, apagaremos um ou todos os registros. Na verdade, o processo de apagar não ocorrerá fisicamente, pois mudaremos apenas a situação do índice de controle, o que permitirá reescrever os dados por cima na variável de armazenamento dos registros "CONTATOS".

```

    caso= A
        Incluir
    caso= B
        Mostrar
    caso= C
        Excluir
    caso= D
        SAIR:="S"

outrocaso

        Escreva( "Opção inválida – retorno ao programa principal")

    fimescolha
ate (SAIR="S")
FimAlgoritmo

```

Comentário sobre o algoritmo:

Observe que este algoritmo contém alguns problemas que podem ser tratados para deixá-lo ainda melhor, tais como a exclusão do registro, feita de forma inapropriada, e alguns problemas sobre a visualização e pesquisa. Porém, serve a lição para apresentarmos o conceito de registros e reforçar a questão da modularização com os subalgoritmos, mostrando um problema grande resolvido em partes.

▶ EXERCÍCIO PROPOSTO

1. Crie um algoritmo para cadastrar vinte funcionários de uma empresa. O cadastro deverá conter os seguintes campos: nome do funcionário, idade, salário, endereço de contato e setor de trabalho. Construa menus para inserir, apagar e pesquisar os dados, assim como uma pesquisa especial de funcionários lotados em um mesmo setor de trabalho.
2. Crie um algoritmo para cadastrar carros. O cadastro deve conter as seguintes informações: placa do carro, modelo, marca, cor e um campo para multas. Um carro poderá registrar até dez multas. Construa um menu para cadastrar os carros, as multas por carro e consultas dos veículos cadastrados com as informações de cada veículo.

SAIBA MAIS

O que temos pra frente - Diversas linguagens de programação suportam a criação e manipulação dos registros. Dentre elas, podemos citar a Linguagem C e COBOL. Quando uma linguagem suporta a criação de registros, ela implementa funções para facilitar o uso de registros, como as funções de apagar ou gravar um registro.

Porém, vimos que trabalhar com registros é um tanto quanto trabalhoso e limitado, pois devemos especificar a quantidade máxima de registros a implementar. Porém, existe forma de trabalhar na qual nos desprendemos desta limitação e podemos criar registros tanto quanto temos memória para suportar seu armazenamento.

Em 1975, surgiram os primeiros programas de Banco de Dados. Banco de Dados, disciplina que você verá adiante neste curso, é também a solução para a manipulação eficiente de grande quantidade de registros. Com o uso dos bancos de dados, não nos preocupamos com o armazenamento e manipulação dos registros, nos preocupando apenas com o programa em si, o que facilita a criação de programas com registros.

CONCLUSÃO

Chegamos ao fim desta disciplina e esperamos que você tenha aprendido um pouco de lógica e construção de algoritmos. É bem verdade que a prática o ajudará a melhorar sua lógica e fará com que você construa algoritmos mentalmente, sem a ajuda do computador.

De agora em diante, veremos técnicas avançadas de algoritmo e resolução de problemas mais complexos, assim como o uso de uma linguagem de programação para o desenvolvimento de programas baseados nos algoritmos desenvolvidos ao longo desta disciplina.

Parabéns a você que concluiu com êxito esta disciplina e realizou com sucesso todos os exercícios propostos neste caderno. A quem não obteve êxito, espero que consiga superar as dificuldades enfrentadas.

REFERÊNCIAS

MANZANO, J. A. N. G.; Oliveira, J. F. **Algoritmos: lógica para o desenvolvimento de programação de computadores**. São Paulo: Editora Érica, 2010.

Site do Brasil Escola. **Filosofia, o que é lógica**. Disponível em: <<http://www.brasilescola.com/filosofia/o-que-logica.htm>>. Acessado em nov. 2013.

Site do Brasil Escola. **Fluxogramas**. Disponível em: <<http://www.infoescola.com/administracao/tipos-de-fluxogramas/>>. Acessado em nov. de 2013.

FORBELLONE, A. L. V.; EBERSPACHER, H. F. **Lógica de Programação: a construção de algoritmos e estrutura de dados**. São Paulo: Prentice Hall, 2005.

AGUILAR, L. J. **Fundamentos de Programação: algoritmos e sua estrutura de dados e objetos**. Porto Alegre: Mcgraw Hill Brasil, 2008.

Dicionário On-Line – **Abstração**. Disponível em: <<http://www.dicio.com.br/abstracao/>>. Acessado em dez. 2013.

Site da Wikipédia. **Programação Estruturada**. Disponível em: <http://pt.wikipedia.org/wiki/Programa%C3%A7%C3%A3o_estruturada>. Acessado em jan. 2014.

[illegible]

[illegible]

[illegible]

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

EDITORIA

EdUFERSA - Editora da Universidade Federal Rural do Semi-Árido
Campus Leste da UFERSA
Av. Francisco Mota, 572 - Bairro Costa e Silva
Mossoró-RN | CEP: 59.625-900
edufersa@ufersa.edu.br

IMPRESSÃO

Imprima Soluções Gráfica Ltda/ME
Rua Capitão Lima, 170 - Santo Amaro
Recife-PE | CEP: 50040-080
Telefone: (91) 3061 6411

COMPOSIÇÃO

Formato: 21cm x 29,7cm
Capa: Couchê, plastificada, alceado e grampeado
Papel: Couchê liso
Número de páginas: 116
Tiragem: 400

Agência Brasileira do ISBN
ISBN 978-85-63145-58-1



9 788563 145581



UNIVERSIDADE FEDERAL
UFERSA
RURAL DO SEMI-ÁRIDO



PROGRAD
PRÓ-REITORIA DE
GRADUAÇÃO



Ministério
da Educação
GOVERNO FEDERAL
BRASIL
PAÍS RICO E PAÍS SEM POBREZA